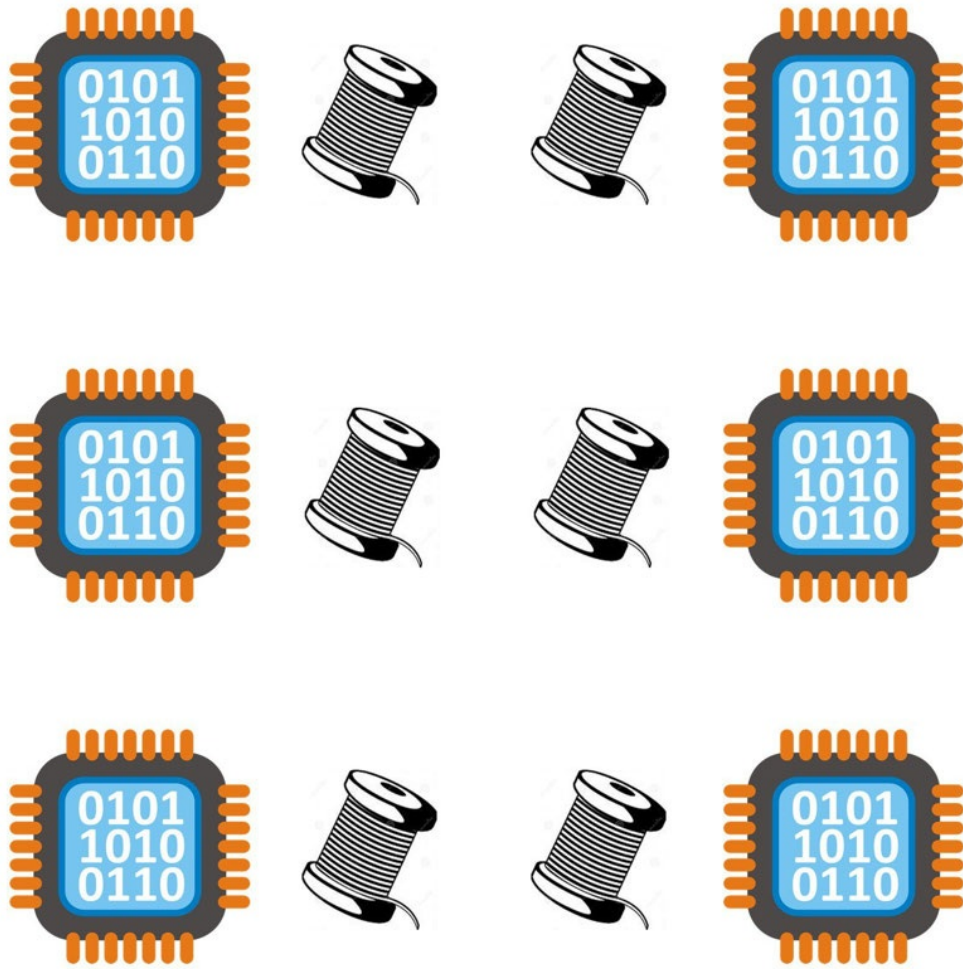
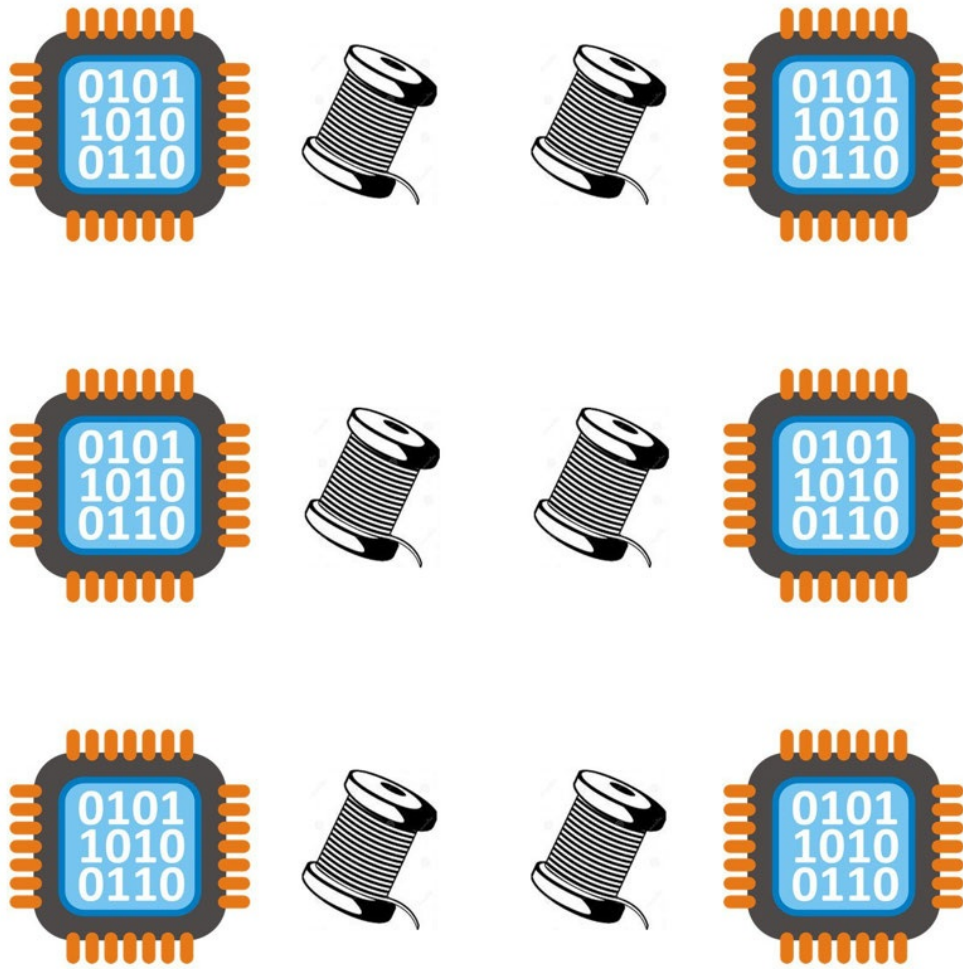




Learn Multithreading with Modern C++

James Raynard



Learn
Multithreading
with Modern C++

James Raynard

© James Raynard 2022

Published by James Raynard

© 2022 Pulborough

All rights reserved. No part of this book may be reproduced or modified in any form, including photocopying, recording or by any information storage and retrieval system, without permission in writing from the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the text, and the publisher was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Chapter One

Concurrency Introduction

What does concurrency mean?

Concurrency

Concurrency generally means two or more things happening at the same time. For example, you could be reading this book while listening to some music.

In computing, concurrency normally means that the activities are conceptually different. Some programs take a long time to run, perhaps because they are fetching data from the Internet, or doing a lengthy calculation. Often, the program will provide some kind of feedback to the user, such as an “hourglass” or a progress counter. This lets the user know that the program is still running, and also gives some idea of what the program is doing. A program which performs these two activities at the same time is an example of computational concurrency.

At the operating system level, concurrency has been standard in operating systems for many years. For example, when I record videos on my laptop, a program records my words of wisdom. PowerPoint is open and waiting for me to go to the next slide in the lecture. The mail program occasionally checks for new messages, and so on. The computer is able to run several programs at the same time, or at least it appears to.

However, until fairly recently, most computers only had one processing unit. And if you only have one processing unit, how can you run different programs at the same time? The answer lies in something called “time slicing”.

Time Slicing

With time slicing, each program gets to run on the processor for a little bit of time. Then it stops, and another program gets to run. Then that program stops to allow other program to run, and so on.

If, for example, I am working on a document while waiting for a compilation to finish, we could have a scenario like this:

- The compiler runs for a short period of time, then pauses
- The mail program runs for a short period of time, then pauses
- The compiler runs again from where it left off, then pauses
- The word processor runs for a short period of time, then pauses

If all this switching between tasks happens quickly enough, it will appear that all these programs are running at the same time.

Cinema films (movies) rely on a similar illusion: they present a sequence of static images so quickly that the human eye interprets them as continuous motion.

Task Switching

A cynic will tell you that “there is no such thing as a free lunch”, and unfortunately this applies to task switching.

The reason is that when a program is running, the processor needs to have some information about it. The processor contains memory cells, known as “registers”; these contain the values of local variables, the program’s current stack position, which instruction the program is currently executing, and so on. When the processor starts running another program, all this “context” has to be loaded up for the new task.

Modern processors also have “cache” memory, which stores instructions and data which are being used, in larger quantities than would fit into the registers. If the program has not run recently, this information will not be present in the cache and needs to be loaded up again.

These “context switches” take time, during which the processor cannot execute any instructions for any program. This will slow down the program.

Hardware Concurrency

In today's world, computers usually have several processors. In fact, even phones now have multiple processors. These are known as “cores” or “processor cores”.

These multiple processor cores can all be working away, doing different things at the same time. They could even be running different parts of the same program at the same time.

These cores all execute independently of each other. They are known as “hardware threads”, because each processor core follows its own execution path or “thread” through the code. There is one hardware thread per processor core.

Software Concurrency

Modern operating systems allow a program to perform multiple activities at the same time at the software level, using time slicing. These are known as “software threads”.

Programmers often refer to software threads as “threads” and hardware threads as “cores”.

Computers often have more software threads executing than there are processor cores. This is generally not a problem, because usually threads do not run all the time.

Quite often, a thread has to stop and wait for some event to occur or some result to become available. For example, a thread may be waiting for another thread to finish a calculation, or it may be waiting for the user to press a key. While one thread is waiting, another thread can run and do some useful work.

C++ Support for Concurrency

C++ did not have any standard support for concurrency until C++11. As with many standard C++ features, it only provides the basics for programmers to write their own library; however, what it does have is very efficient. C++20 offers some more sophisticated features, which are not yet fully supported by compilers.

C++'s concurrency implementation is fairly low-level, compared to other languages. The basic building block is the `std::thread` class; this maps onto a software thread which is managed by the operating system.

`std::thread` is similar to the `thread` class in the Boost library, but there are some important differences:

- No thread cancellation
- Different argument semantics
- Different behaviour on thread destruction

Threads and Tasks

The terms “thread” and “task” are often used interchangeably. In this book, “thread” will be used to mean either a software thread, or an object of the C++ `std::thread` class.

A “task” is a higher-level abstraction, representing some work that should be performed concurrently. This work could be done by a single thread, or possibly by several threads.

Chapter Two

Concurrency Motivation

Why is concurrency important?

Concurrency Motivation

Concurrency first appeared on mainframes in the 1960's, but it was not until the 1990's that the world of workstations and personal computers began to regard it as an important technology. This was due to a number of developments in the world of computing, which were happening around that time.

The Rise of the Internet

Previously, most servers were connected to local networks which served offices or departments; typically, they were only required to handle a few dozen or maybe a few hundred clients at most.

Internet servers can have thousands or even millions of simultaneous users. Clearly, the amount of work that a server could do at any one time needed to increase dramatically.

Internet servers use threading to increase server throughput.

The Rise of Windows

Users on personal computers could now access several running programs at the same time. They expected to be able to interact with any program at any time, even if the program was busy doing something else.

This meant that Windows programs had to be able to respond to user interaction events while continuing with their work.

Windows applications use threading to provide “separation of concerns”.

Fast Numeric Computing

Games and multimedia applications were becoming very popular at this time. These typically involve a lot of mathematical calculations. If the game cannot update its display fast enough, the game will appear “jerky”, and the audio and visual may suffer from gaps or distortion.

Scientists and engineers were increasingly using computer simulations, which also require rapid mathematical computation.

Games and simulations use threading to provide separation of concerns and to speed up calculations.

The Rise of Multi-core Processors

Computers began to appear which had multiple processors.

Programmers use threading to make effective use of the hardware.

Single-threaded Servers

Before multithreading, a server performed each activity as a separate “process” (a process is a single instance of an executing program)

- The server waits for a client to connect to it
- When a client connects, the server creates a new “ child ” process to handle the client
- The server waits for the next connection

There is a single server process, and a number of child processes which are dealing with clients. These processes are all separate running applications, each with their own address space. There is no direct communication between these processes.

Usually, the server usually needs to communicate with the child process, perhaps to exchange data with it or to control it in some way. This requires establishing

some form of communication channel between the two processes.

Setting up these communication channels has overhead, and there is also a certain amount of overhead when creating a new process. On Windows systems in particular, creating a new process is slow and resource intensive. This overhead will reduce the amount of work that the system can do, which reduces its scalability.

Separation of Concerns

Imagine you are editing a large document, using a single-threaded application, and you start an action which takes a long time to complete. This could be compiling an index for the document, or calculating page numbers, or even - and remember this is the 1990's! - saving the document to a floppy disk.

While this is happening, the user interface of the program will be completely unresponsive. It will not respond to keystrokes or mouse clicks. If you cover up its window with another application and then uncover it, the program window will have been replaced by a grey box.

The Windows system is sending events to the program, telling it that the user is interacting with it, but the program cannot respond, because it is busy performing the action. All the GUI events are stuck in a queue, waiting for the program to respond to them.

Eventually, the action ends. The program reads all the events in the queue, and processes them instantly. Depending on what you did when you were frantically trying to get the program to respond, all sorts of things could happen. You may find that you have erased your document, or overwritten it with garbage and saved it. This is obviously not desirable!

Separation of concerns improves the user's experience of GUI programs. More generally, it may improve performance, particularly on multi-processor systems. It also follows good coding practice, because it keeps all the code which does one thing in the same place.

In this example, all the GUI event handling code is in a separate thread from the document processing code. This makes the code easier to understand and work with; it also allows different programmers to work on different parts of the code

at the same time, without interfering with each other's work.

Fast Numeric Computation

Numeric computation was mainly restricted to people who worked in Universities or Research Institutes and could justify buying a “supercomputer” or “transputer”. These were specialized hardware devices with large numbers of processors. They had a parallel processing architecture which allowed the processors to coordinate with each other.

These computers were programmed using specialized languages, such as Occam or Parallel Fortran. These divided up the program and ran different parts of it on different processors at the same time. The computers, and indeed the languages they used, were extremely expensive.

Nowadays, we can buy a normal computer, or even a phone, which supports parallel programming in hardware, and use a standard language to write parallel code.

Effective Use of Hardware

In the 1990's, chip designers reached the limit of what they could achieve by making bigger and more complex processors which ran faster and faster. They were starting to be affected by the limitations of the laws of Physics.

Electrons move through silicon at around 5-10% of the speed of light (roughly 4,000 miles or 7,000km in a second.) With a processor cycle time of 1 nanosecond (a frequency of 1 GHz or 1,000 million cycles per second), this translates to a quarter of an inch or half a centimetre per cycle.

Given that modern processor chips can be 3 inches or 5cm across, then it will take several cycles for an electron to go from one side of the processor chip to the other. This limits the speed at which data can move around and instructions can be processed.

The other problem is heat generation. When electrons move through silicon, they generate heat, and this increases as the chip gets bigger and faster. If designers had continued making single chip processors, then chips would now be running

at the temperature of molten steel or the surface of the sun. Which is really not something you want to keep on your desk!

Instead, designers started producing “cores”, in which there are several processors on the same silicon chip.

A single-threaded program can only run on one core at a time. If you have a modern computer which only runs one program, and it only uses one of the available cores, then you are clearly not making full use of the system’s power.

Concurrency is the Answer!

Concurrency gives our program the ability to do more than one thing at the same time.

For servers, we can have a child thread which runs inside the server process. This avoids having to create a new process every time a client makes a connection. Because the server and the child are part of the same process, they share the same address space. They can directly access each other’s data, if they choose to share it.

This removes all the overhead related to interprocess communication and process creation. It does add overhead from thread creation, but this is less than the overhead from process creation. It also adds overhead, because the data must be shared safely, but that is a different question.

For separation of concerns, we can write a program which is able to respond to GUI events promptly, while it continues to perform a long-running task. We do that by providing separate threads for the GUI and for the task.

For numeric computation and effective use of hardware, we can write programs which can execute across different cores at the same time.

Concurrency addresses all these problems. However, it does add some problems of its own, which we will cover later. Also, there are some programs where concurrency does not help, or even makes things worse.

Concurrency and Parallelism

Finally, there is a small but important distinction between concurrency and parallelism.

Strictly speaking, “concurrency” is when conceptually different tasks run at the same time.

- A server and a child are different concepts
- Processing a GUI event and indexing a document are different concepts

“Parallelism” means that the same tasks run at the same time.

- Performing different parts of the same vector addition on different cores at the same time is an example of parallelism

Chapter Three

Concurrency Overview

Adding a little more detail

Application-level Concurrency

At the application level, concurrency can involve

- single-threading: separate executing program instances perform each activity
- multi-threading: a single executing program instance in which different activities are performed

Single-threaded concurrency

Single-threading is the oldest form of concurrency, with a separate process for each activity. Each process has its own execution path through the code, which is known as a “thread”.

In an ancient Greek legend, Theseus found his way out of the Minotaur's labyrinth by following a silken thread. Perhaps this inspired the use of the word “thread” to describe the way that the program finds its way through the maze of the source code?

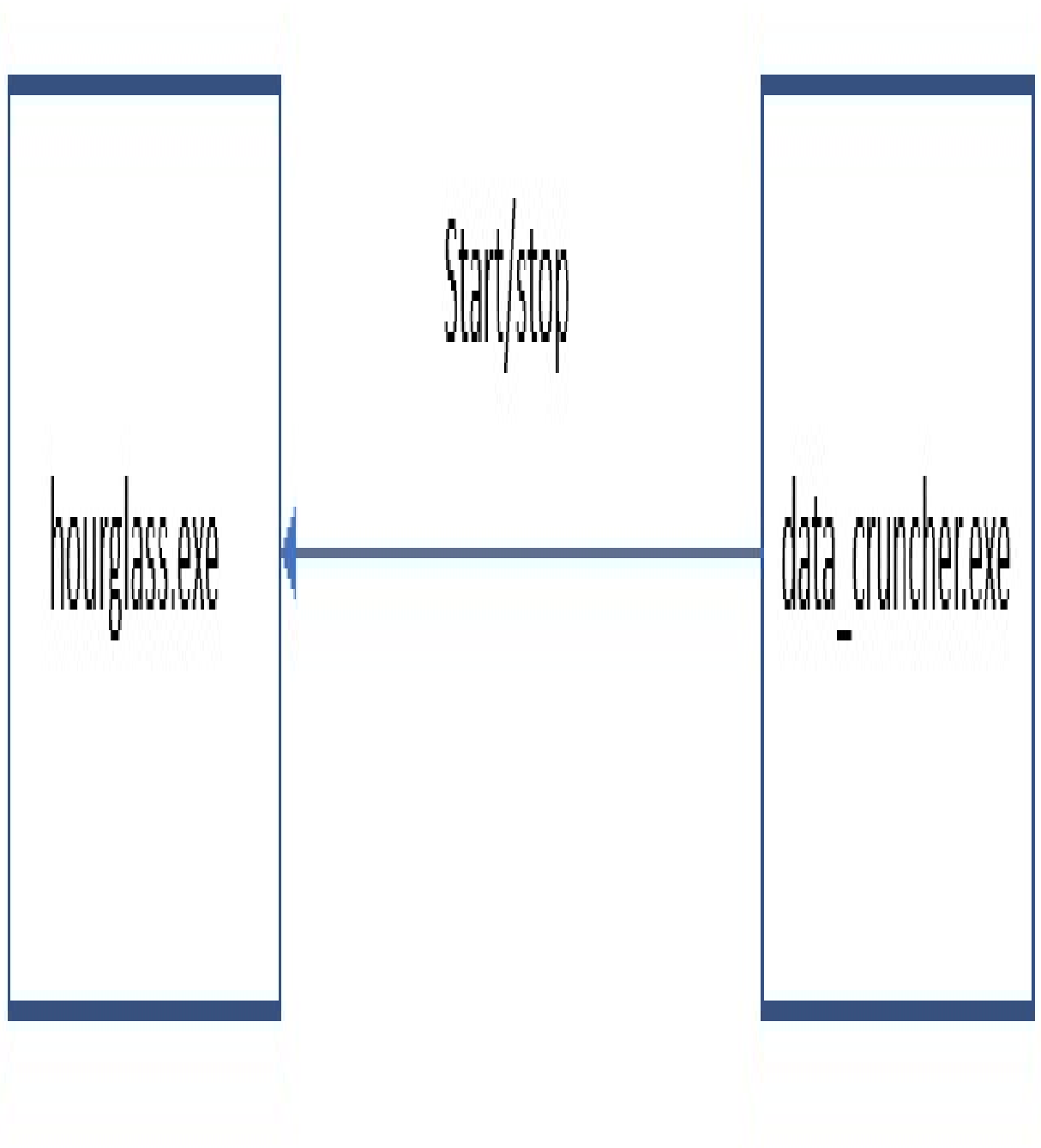
In single threading, each process has a single thread. The only way through the code is to start at the beginning of `main()`, execute instructions in sequence until the end of `main()`, and then stop. Each process also has its own memory space, so it is not possible for another process to access its memory directly.

We start up a new process for each activity that we want to run concurrently. The operating system will organize this for us.

If any of our activities need to exchange data, or communicate in some way with each other, we need to establish some kind of interprocess communication.

There are various ways of doing this: message queues, semaphores, shared memory, network sockets, and so on.

For example, if we have a program `data_cruncher.exe` which takes a long time to do a computation, and we want to display an hourglass to the user, we will need to start up a separate program `hourglass.exe`. We will also need to set up a communication channel between the two processes, to make `hourglass.exe` stop when `data_cruncher.exe` is ready to interact with users again.



Two separate processes which interact via interprocess communication

One advantage of this is that the processes can be run on different computers and communicate over a network, distributing the processing load. However, creating a new process can be slow on some systems. Interprocess communication adds complexity to the programs and can be slow to set up.

C++ does not provide direct support for interprocess communication. Programmers must either make system-specific API calls, or use a third-party library.

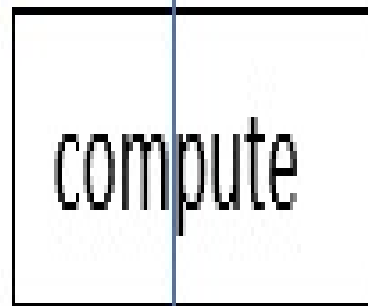
Multi-threading Concurrency

With multi-threading concurrency, all the activities run in a single process. Each activity has its own execution path or “thread” through the code. So you can imagine there are several different threads making their way through the program’s code, at the same time, taking different paths, doing different things, or even doing the same thing. To start a new activity, we start a new thread by creating an `std::thread` object.

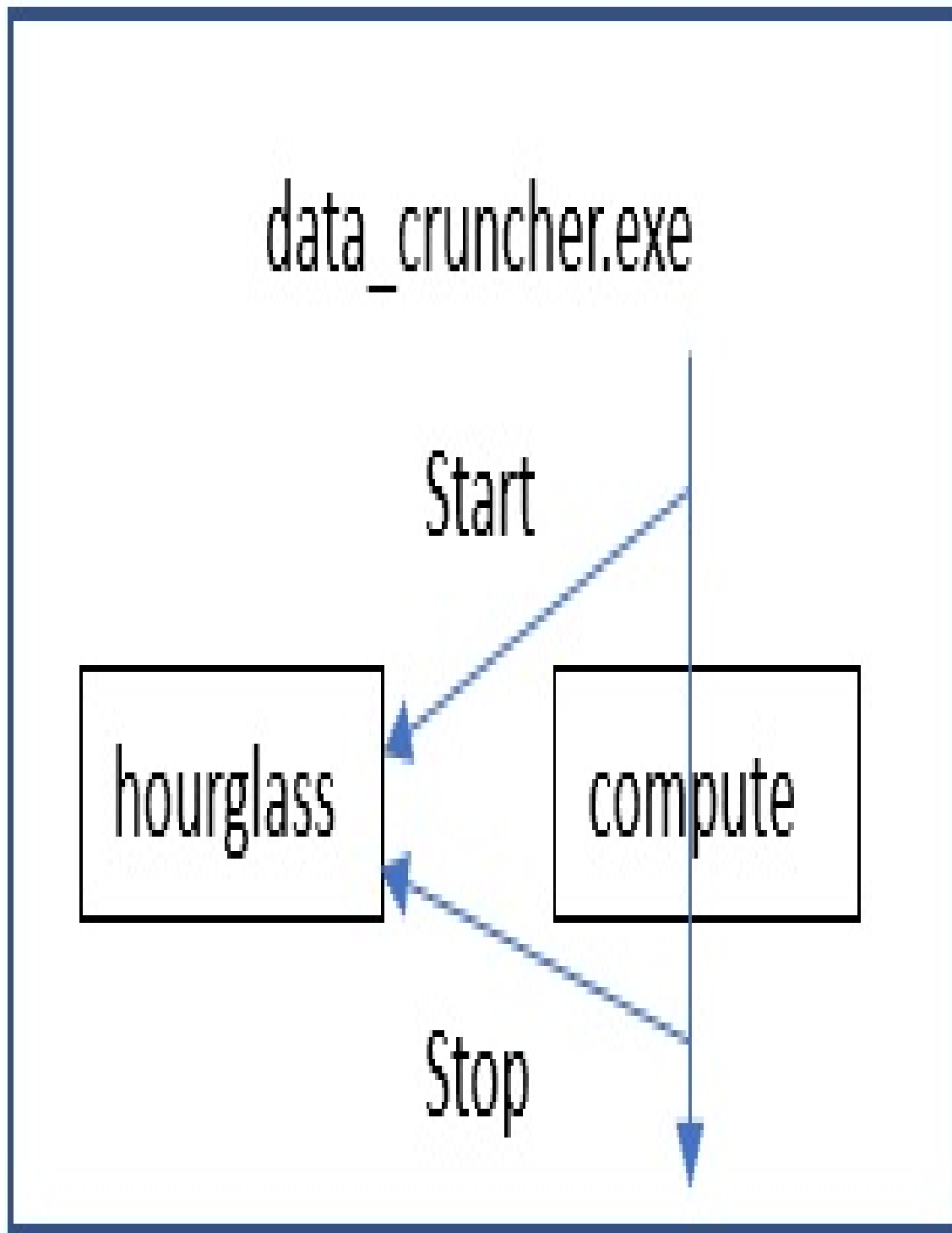
All the threads share the memory space of the process they are running in. If any data is “global” in this process, meaning not in any class or function, then all the threads can see it: they can access it and modify it. It is also possible for threads to send certain kinds of objects from one to another.

data_cruncher.exe

Start



Stop



Two separate threads which interact directly within the same process

In our data crunching example, we now have one program that does everything. This program will start in `main()`, executing its main thread of execution. When it is about to start a lengthy computation, it starts a new thread which displays the hourglass. When it completes the computation, it terminates the hourglass thread and carries on.

Threads are implemented using so-called “light weight processes”. These are provided by the operating system.

Creating a thread involves less overhead than creating a new process. There is also less overhead when task switching threads, because there is less context.

Sharing data between threads and communicating between threads is much easier than it is for processes. On the other hand, there is a lack of data protection. Working with threads can require careful programming, to avoid inconsistent behaviour and data corruption. This is a big problem in threading.

Advantages of Concurrency

The program will feel more responsive: if the program is busy doing something, it can still communicate with the user and respond to commands. The user is never left staring at a “stuck” program.

Concurrency can improve throughput by allowing large amounts of data processing or calculations to be done at the same time. This reduces the overall program execution time.

Concurrency allows logically distinct operations to be kept separate from each other. For example, if you are using an email program to compose a new message, other threads can run, perhaps to fetch incoming mail or to archive your old messages.

Concurrency allows programmers to take full advantage of modern hardware. Threads can be distributed across processor cores, making better use of the available computational power.

Disadvantages of Concurrency

Concurrency adds complexity to programs. Writing a threaded program requires more thought, and the resulting code is harder to understand and maintain. It also introduces new categories of bugs, which are often timing-dependent. These can be very hard to replicate and debug.

Concurrency may not necessarily result in faster programs. Protecting shared data may increase execution time by more than the amount that threading saves.

Concurrency should only be used when the benefits outweigh the costs.

Chapter Four

Concurrent Applications

What does it mean to make an Application Concurrent?

Server Applications

Server applications such as a database server naturally lend themselves to threading. Typically, a server receives lots of requests from different clients, so it is quite natural to start a new thread to handle each request.

There is a high degree of commonality of code. Usually there are only a few basic operations. For example, a database server may be asked to return some data, or create a new table and populate it. It is easy to define the concurrent tasks and decide how to implement them.

Data which is shared between threads can be protected easily. A database is designed to support transactions, which prevent operations from interfering with each other. Other servers can use a database for this.

Client Applications

For client applications or standalone user applications, things are not so simple. Usually, there is only one user for each program instance, so there are no opportunities to gain concurrency between users.

The users tend to follow a sequential workflow, which does not lend itself easily to threading. For example, open a document, make some changes, save the document and then print it out.

The data that might be useful for sharing between threads is often hidden away inside functions or objects in the application, unlike servers, where it is openly exposed.

Concurrency and Responsiveness

As we discussed earlier, single-thread GUI applications tend to “block” when performing an operation that takes a long time. The application does not respond to GUI messages which tell it, for example, to redraw itself or respond to a mouse click.

In a multi-threaded GUI application, the user interface can run in its own thread. This thread is always in the foreground and responds promptly to user actions, regardless of what else the application is doing.

Each task, such as saving a document or preparing a print preview, is run as a separate thread. These threads run in the background, asynchronously. They can communicate with the user interface thread, for example through message queues. The user thread can respond by changing state when appropriate: for example, it might display an hourglass, or disable some menu items.

Concurrency and Shared Data

All threads have access to the program’s memory space and can easily share data. In C++ terms, this can be done using global or static variables. However, different threads can access the same data concurrently; if different threads try to modify the same variable at the same time, this can result in data corruption, so the variable needs to be protected.

Traditionally, threaded code used “mutexes” to “lock” a variable so that only one thread could access it at a time.

Another technique is to make the variable “atomic”, so that operations on the variable cannot be interrupted by other threads.

In this book, we will concentrate on the traditional approach with locks, with some coverage of atomics.

Overview of Threaded Client Application

Imagine we have an application such as Word, which is about to index a document. It will start a new thread to do the indexing.

First, the GUI thread disables most of the user interface functionality. Many menu items will be greyed out, so the user cannot click on them, but the user can still do a few things, such as moving the window around and have it redraw itself. The GUI thread then displays a dialogue box, which allows the user to cancel the indexing.

The indexing thread will then start to work out where all the items are and how to index them.

Every now and then, the indexing thread will send a message to the GUI thread saying, “I have completed X%”, and the GUI will display a progress bar, showing that X% has been completed.

Finally, the indexing thread will send a message to the GUI thread, saying “I have finished” and then it will terminate. The GUI thread will close the dialogue box, then re-enable the user interface, and the disabled menu items will be available again.

Chapter Five

A Brief History of C++ Concurrency

C++ Concurrency Through the Ages

Single-threaded Concurrency

In the very early days, when the C language was developed for writing the Unix operating system, a program could create a new process by making the `fork()` system call. This causes the operating system to create a new “child” process which is an exact copy of its “parent”, apart from a few book-keeping details.

`fork()` returns in both the parent process and the child. In the parent, it returns the process ID of the child. In the child, it returns 0.

The parent can also create a “pipe” before forking, which allows data to flow between the parent and the child. This allows the two processes to communicate.

Optionally, the child process can make the `exec()` system call and load up a different executable. This makes it run a different program from the parent. In fact, this is how the Unix command-line shell is implemented.

By default, the child process will be a “daemon” which executes independently of the parent. However, the parent can stop and wait for the child process to terminate.

Many client-server systems were implemented this way.

Multi-threading using C API's

For many years, C++ did not provide direct support for threads. Programmers had to use threads through C interfaces:

- The pthreads library for POSIX systems (UNIX-like)

- The Windows API provides access to threads through the “Base Services”
- Various Unix API’s provide system calls to access the operating system’s thread support. This varies between different versions of Unix
- OpenMP allows programs to process C-style array loops in parallel

Multi-threading using C++ API’s

Eventually, C++ threading libraries began to appear.

ACE is a cross-platform library, mainly intended for networking. Large, clunky, and written in typical 1990’s C++.

The Boost project was set up to develop new features for C++, many of which appeared in C++11. Its thread support consisted of a wrapper around the operating system’s threading facilities. This gave a consistent interface, so programmers could write the same threading code on any platform.

Poco is a more modern and lightweight version of ACE. Again, it is mainly intended for networking, but does have thread support.

Multi-threading using the C++ Standard Library

C++ did not officially acknowledge the existence of threads until 2011. This is rather ironic, as the first serious code that Bjarne Stroustrup wrote with his new “C with Classes” was a task management library with coroutines. This was in 1980. It took another 31 years to get threading into the standard!

C++11 adds features for creating and managing threads

With C++11, we now have a thread-safe standard library, so we can use all the standard containers and algorithms in threaded code. Again, this requires some care, but no more than for other types of data.

C++11 also provides a memory model. This means that programmers can write threaded code which is portable and efficient. The code will have well-defined behaviour, unless of course the programmer writes unsafe code.

Finally, atomic operations allow the programmer to have fine-grained control over operations at low levels, at the cost of making the code harder to understand.

Threading Developments in the C++ Standard Library

Since 2011, C++ has added some more threading features to the standard library:

- C++14 added read-write locks
- C++17 supports parallel execution of many standard algorithms
- C++20 adds joining threads, coroutine support and improved synchronization facilities

At the time of writing (July 2022), compilers do not fully support C++20. Some features will not be properly usable until C++23.

This book will only cover standard C++ features up to and including C++17.

Chapter Six

Launching a Thread

Hello, Thread!

Structure of a Single-Threaded C++ Program

Before we find out how to launch a thread in C++, we need to learn about the structure of threaded programs.

With a single-threaded program, there is a single thread of execution, known as the “main thread”.

main()



main
thread

A single-threaded program has only one thread of execution

The main thread will call the `main()` function, which is the entry point to the program. `main()` may itself call other functions.

When the `main()` function returns, the main thread terminates. The program terminates when the main thread terminates.

Structure of a Multi-threaded C++ Program

A multi-threaded program still has a main thread of execution, which calls the `main()` function.

However, the main thread can also create further threads. Each of these threads has a function which is the entry point for the thread.

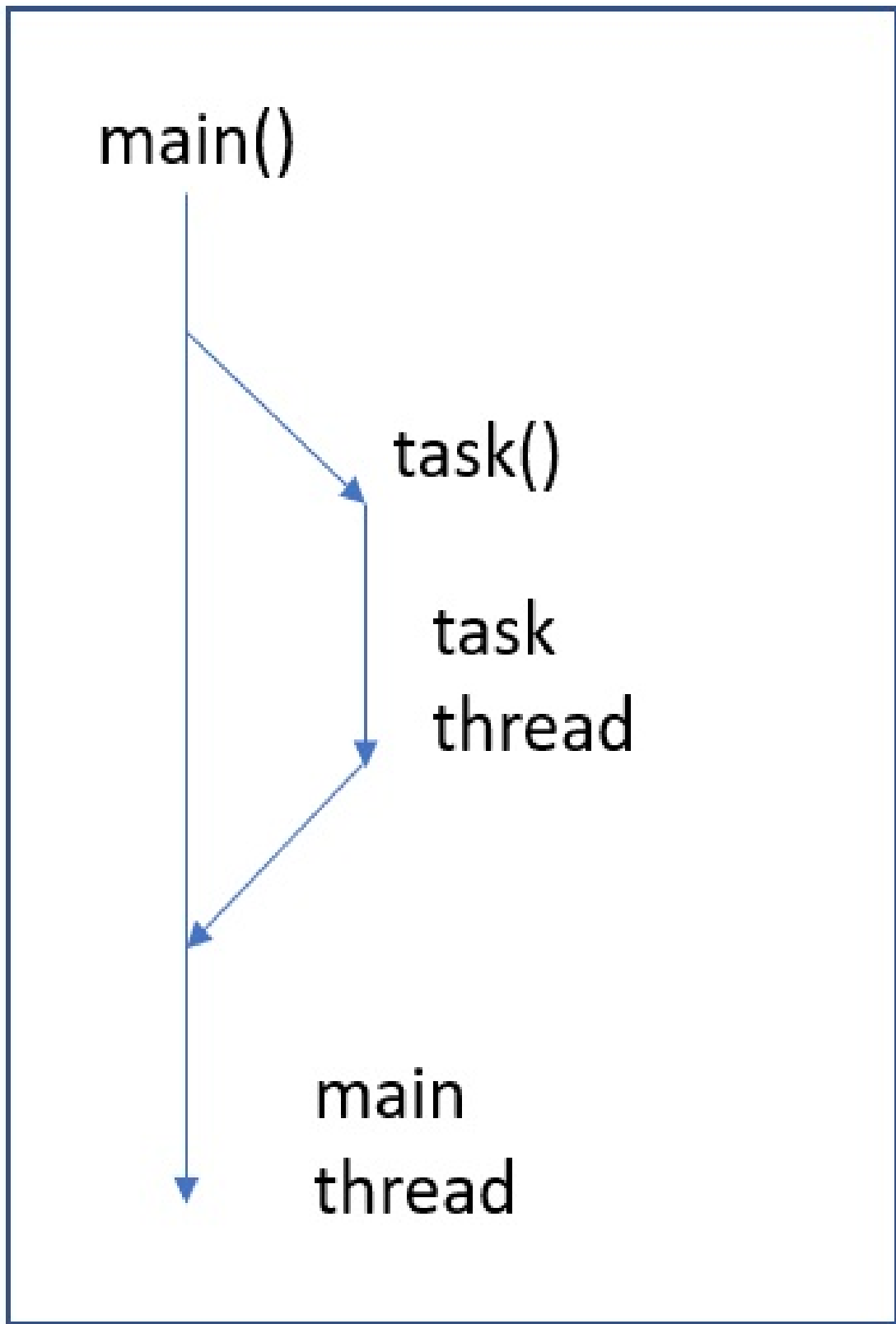
When a thread starts executing, its entry point function is called. When this entry point function returns, the thread terminates.

main()

task()

task
thread

main
thread



The main thread creates a task thread. The two threads execute independently of each other

Meanwhile, the main thread will continue with its own execution path. It does not stop and wait for the other thread, unless it is explicitly told to "join" it.

In the diagram, the main thread creates a thread whose entry point is the task() function. When the new thread starts, it executes the code in task(). This task() is just like any other function. It can call other functions; it can also start other threads. When task() returns, that thread will terminate.

Thread Creation

The C++ library provides an `std::thread` class, which represents a thread implemented in the operating system. It is defined in the `<thread>` header.

Like many resource management classes in C++, an `std::thread` object cannot be copied, but it can be moved.

`std::thread`'s constructor takes a callable object as its first argument. This represents the entry point function for the thread. This callable object could be a function pointer, a lambda expression, and so on. It contains the code that the thread will execute when it runs.

This entry point function must not be overloaded. It can take arguments by value, by reference or by move. These are passed as additional arguments to the `std::thread` constructor.

If the entry point function returns a value, it will be ignored. (We will find out later how to get a result from a thread.)

Thread Execution

Once we create an `std::thread` object, the thread will start running immediately, by calling the entry point function with the given arguments.

If we want the parent thread to stop and wait for this thread, it must call the `join()` member function on the thread. For example, if the main thread creates an

std::thread object t, and the main thread calls t.join(), the main thread will not execute any more instructions until t terminates.

It is very important to do this. If the main thread does not call t.join(), then the program could terminate while t is still running. This will cause the thread to be terminated in mid-execution, which is not good.

Hello, Thread program

This is the threaded version of the famous “Hello, World!” program.

The thread entry point will be a function which prints out “Hello, Thread!”. It takes no arguments, and returns nothing.

The main() function creates an std::thread object and passes a pointer to the entry point function as the argument to the constructor.

```
#include <thread>
```

```
#include <iostream>
```

```
// Thread entry point function
```

```
void hello()
```

```
{
```

```
    std::cout << "Hello, Thread!\n";
```

```
}
```

```
int main()
```

```
{
```

```
    // Create an std::thread object and initialize it
```



```
// with a callable object

// A function pointer to the entry point function

std::thread t{hello};


// Wait for the thread to complete

t.join();

}
```

When the program runs, the output is

Hello, Thread!

Thread Function with Arguments

If we want to pass arguments to the entry point function, we list them as extra arguments to the thread constructor.

As an example, we will now pass the string to be printed as an argument to hello().

```
#include <thread>

#include <iostream>

#include <string>


// Thread entry point now takes an std::string argument

void hello(std::string str)
```

```

{
    std::cout << str << "\n";
}

int main()
{
    // Create an std::thread object and initialize it
    // with the entry point function
    // Pass the argument to hello()
    std::thread t{hello, "Hello, Thread"};

    // Wait for the thread to complete
    t.join();
}

```

Again, the output from the program is

Hello, Thread!

Thread Function with Reference Argument

If we want to pass an argument by reference, we wrap it in a call to `std::ref()`, or `std::cref()` for a const reference.

This time, `hello()` will take its argument by reference. We will create an

std::string object in main(), then pass it to hello().

To check that this works, we will modify the argument in hello(), then print it out in main() after the thread completes.

```
#include <thread>
```

```
#include <iostream>
```

```
#include <string>
```

```
// Thread entry point now takes its argument by reference
```

```
void hello(std::string& s)
```

```
{
```

```
    s = "xyz";
```

```
}
```

```
int main()
```

```
{
```

```
    // This will be the argument to hello()
```

```
    std::string str{"abc"};
```

```
    // Pass the argument by reference
```

```
    std::thread t{hello, std::ref(str)};
```

```
// Wait for the thread to complete

t.join();

// Verify that the string has been modified

std::cout << "str is now " << str << "\n";

}
```

When the program runs, the output is

```
str is now xyz
```

Thread Function with Pass by Move

We can move an argument into the function by calling `std::move()`. This requires that the argument's type has a move constructor. The function will then take ownership of the data in the argument.

This time, we use `std::move()` to cast the argument to an rvalue reference.

```
#include <thread>

#include <iostream>

#include <string>

// Thread entry point

void func(std::string s)

{
```

```
std::cout << "All your string are belong to us: " << s << "\n";  
}
```

```
int main()  
{  
    // This will be the argument to hello()  
    std::string str{"abc"};  
  
    // Pass the argument by move  
    std::thread t{func, std::move(str)};  
  
    // Wait for the thread to complete  
    t.join();  
  
    // Verify that the string has been modified  
    std::cout << "str is now " << str << "\n";  
}
```

When the program runs, the output is

All your string are belong to us: abc

str is now

The final value of str is an empty string, because its data has been moved into the argument to func().

(Strictly speaking, str is an undefined state after being moved from, and reading from it is undefined behaviour. However, here we are only trying to demonstrate that the data has been moved from the variable.)

Hello, Thread with Member Function

We can also use a member function as the thread entry point.

When calling through a member function pointer, we have to pass an object of the class as the first argument. This object argument will provide the this pointer inside the member function body.

```
#include <thread>
```

```
#include <iostream>
```

```
#include <string>
```

```
class greeter {
```

```
    public:
```

```
        // Thread entry point
```

```
        void hello()
```

```
        {
```

```
            std::cout << "Hello, Thread!\n";
```

```
        }
```

```
};
```

```
int main()
{
    // Create an object of the class

    greeter greet;

    // Use the member function pointer

    // Pass an object as the argument

    // to the member function call

    std::thread t{&greeter::hello, &greet};

    // Wait for the thread to complete

    t.join();
}
```

When the program runs, the output is

Hello, Thread!

Hello, Thread with Lambda Expression

Finally, we can also use a lambda expression as the thread entry point. The lambda expression is written as the first argument to the thread constructor.

```
#include <thread>
```

```
#include <iostream>
```

```
#include <string>
```

```
int main()
```

```
{
```

```
    // Use a lambda expression
```

```
    std::thread t{ []()
```

```
    {
```

```
        std::cout << "Hello, Thread!\n";
```

```
    };
```

```
    t.join();
```

```
    // Lambda expression with capture by reference
```

```
    std::string str{"abc"};
```

```
    std::thread t2{ [&]()
```

```
    {
```

```
        str = "xyz";
```

```
    };
```



```
t2.join();  
  
std::cout << "str is now " << str << "\n";  
  
}
```

When the program runs, the output is

Hello, Thread!

str is now xyz

Chapter Seven

The C++ Thread Class

std::thread in More Detail

Pausing Threads

We can tell a thread “do not execute any instructions for a certain period of time”. This is known as “pausing” the thread or “making the thread sleep”.

To do this, we call `std::this_thread::sleep_for()`

This takes an argument of type `std::chrono::duration`, which specifies the time the thread should sleep for.

```
// Make the current thread pause for 50 milliseconds
```

```
this_thread::sleep_for(std::chrono::milliseconds(50));
```

C++14 has suffixes for duration literals, which make this code much more readable. We can bring these in by using the `std::literals` namespace:

```
using namespace std::literals;
```

```
// Make the current thread pause for 50 milliseconds
```

```
std::this_thread::sleep_for(50ms);
```

`std::sleep_for()` also works for single-threaded programs, where it will pause the main thread. Apart from `#include <thread>`, no other thread-related code is required.

Now we no longer need to make calls to the operating system API to make a program pause or sleep.

Thread Identifiers

Each individual thread object has an identifier, of type `std::thread::id`. This is used internally by the threading library. The actual value depends on the implementation, and has no significance to the programmer.

These identifiers are guaranteed to be unique. If several threads are running concurrently, they will have distinct identifiers. If two thread object instances have the same identifier, then they are the same object (this can happen with references to thread objects.)

However, the identifiers can be re-used: if a thread terminates and a new thread starts up, the new thread could have the same identifier as the old thread.

We call `get_id()` to find a thread's identifier. This is available as a free function, and also as a member function.

// Get the current thread's identifier

```
std::this_thread::get_id();
```

// Get the identifier for the std::thread object t

```
t.get_id();
```

Native Handle

Operating systems usually provide a thread API with C functions. These take a thread “handle” as the first argument. This is a pointer to a data structure which contains information about the system thread; it is opaque and does not contain information which is useful to an application.

`std::thread` has a `native_handle()` member function. This returns the handle for the system thread which is associated with the `std::thread` object. The handle can then be used to access the operating system API: Windows Base API, Linux kernel threads and so on.

`std::thread` is designed to provide a "lowest common denominator" interface for all operating systems which support threads and C++. However, many operating systems provide useful extra features which are not directly available through `std::thread`, for example:

- Thread priority . A thread can be given a greater or lesser share of processor time
- Thread affinity . Make a thread run on a specific processor core

Thread affinity, also known as “pinning”, is very important in high-performance computing, where low latency is required. For example, some investment banks have systems where each processor core is reserved for processing a different stock's data. This means that the thread for the stock is never switched onto another core and other threads will not be switched onto this code. The thread will not be delayed by context unloading and reloading.

Moving `std::thread` Objects

An `std::thread` object has ownership of the underlying system thread, similar to the way that an `std::unique_ptr` has ownership of the allocated memory.

We can only have one thread object for a given execution thread at any one time; copying thread objects is not allowed. In the `std::thread` class definition, the copy operators are declared as `=delete`.

However, it is possible to move the execution thread from one `std::thread` object to another. The moved-from object releases its ownership of the execution thread, and the moved-to object acquires ownership of it.

Passing `std::thread` Objects to Functions

Thread objects can be passed by move. We can either pass a temporary object, or use `std::move()` to cast a named object to an rvalue reference.

Supposing the thread's entry function is `f()`:

// Function taking a thread object by move

```
void func(std::thread t);
```

```
// Pass a temporary thread object
```

```
// which has f() as its entry point
```

```
func(std::thread{ f });
```

```
// Pass a named object by move
```

```
std::thread thr{ f };
```

```
func(std::move(thr));
```

The function argument `t` now has ownership of the execution thread. `func()` is responsible for calling `join()` on the thread.

The caller no longer owns the execution thread and cannot call `join()`.

Returning `std::thread` Objects from Functions

As well as passing `std::thread` objects to functions, we can also return them from functions. The compiler will automatically move a returned local variable if possible, so we do not cast it:

```
// Function returning a temporary thread object
```

```
std::thread func(std::thread t)
```

```
{
```

```
    // Return a temporary thread object
```

```
    return std::thread{ f };
```

```
}
```

// Function returning a named object

```
std::thread func(std::thread t)
{
    std::thread thr{ f };

    return thr;
}
```

The caller now has ownership of the execution thread.

std::thread and Exceptions

Each execution thread has its own call stack. All the data within the thread is private; other threads cannot directly access the thread's internal data.

When an exception is thrown in a single-threaded program, the destructors are called for all the objects in the current scope. The program then looks for a suitable handler for the exception. If it does not find one, it jumps to the enclosing scope, destroys all its local objects and looks for a handler.

This process of “stack unwinding” continues until the program finds a handler for the exception. If it has reached `main()` without finding one, the program calls `std::terminate()`, which will terminate it by default.

When an exception is thrown in a thread, a similar stack unwinding takes place. However, because each thread has its own stack, none of the other threads can catch the exception, and that includes the main thread of execution.

If there is no suitable handler in the thread, the entire program is terminated. This happens even if `main()` could handle the exception.

Exceptions can only be handled in the thread in which they occur. This is done in the normal way, using `try/catch` blocks.

```
#include <thread>

#include <iostream>

// Thread entry point
void hello()
{
    try {
        // Throw an exception
        throw std::exception();
    }
    catch(std::exception& e) {
        // Handle the exception
        std::cout << "Caught an exception\n";
    }

    std::cout << "Hello, Thread!\n";
}

int main()
{
    // Create the thread object
```

```
std::thread t{hello};  
  
t.join();  
  
// Check that the program is still running  
  
std::cout << "Finished\n";  
  
}
```

The output when the program runs is:

Caught an exception

Hello, Thread!

Finished

Chapter Eight

Managing Threads

Joining and Detaching Threads

Detaching a Thread

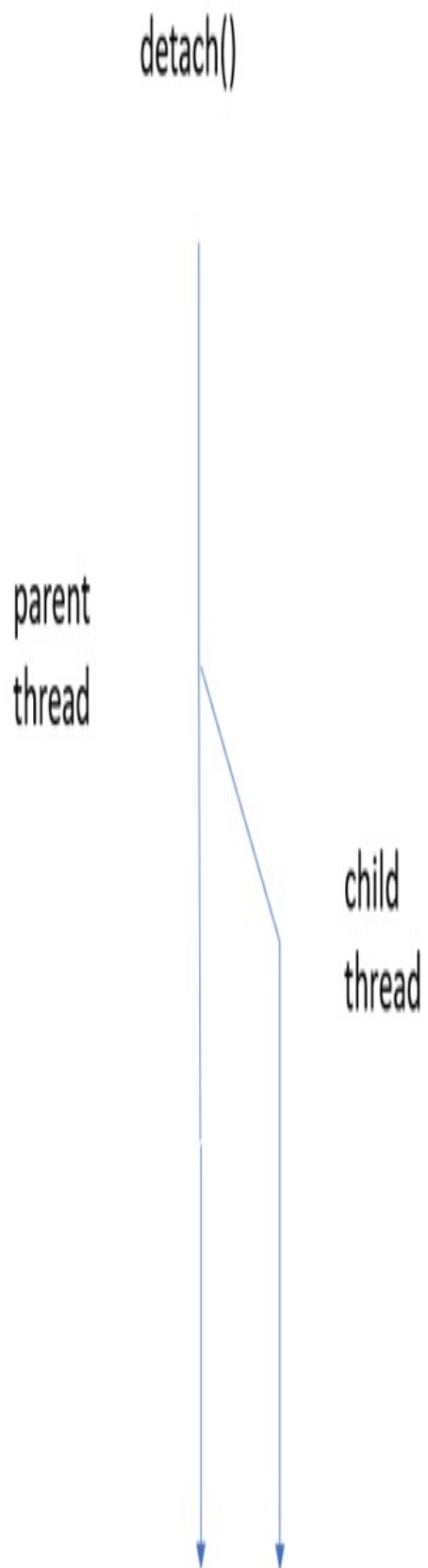
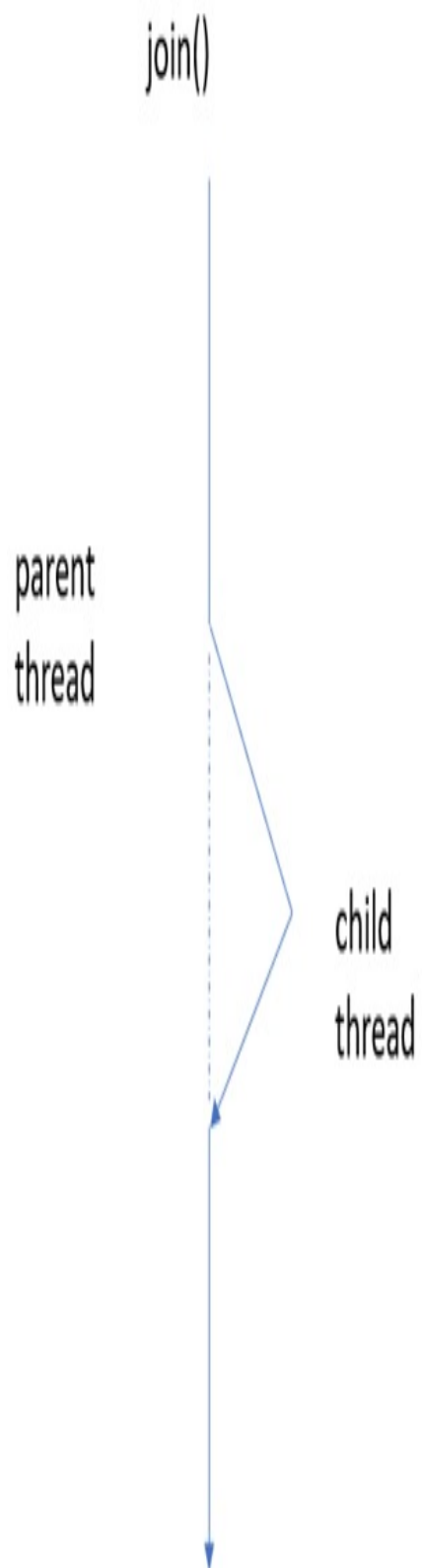
The main thread, or perhaps some other thread, creates an `std::thread` object. This starts a system thread, which calls a function and executes the code in that function.

We said earlier that the parent thread should always call `join()`, to wait for the child thread to complete. In fact, there is an alternative, which is to call `detach()`. In that case, the parent will carry on executing and ignores the child.

The detached thread will continue executing, until either it has completed or the program terminates. The parent thread will also continue executing. This is similar to a “daemon” process in Unix, or a “service” in Windows.

When an `std::thread` object is created, it contains some resources which are needed for joining to its parent thread. A detached thread no longer needs these resources: when the parent calls `detach()`, they are released immediately.

If the detached thread is still running when the program terminates, the `std::thread` destructor is called, which terminates the system thread correctly.



The difference between the parent thread calling join() and calling detach()

When the parent thread calls join(), it stops and waits for the child thread to complete before it executes any more instructions. When the parent thread calls detach(), it will continue running while the child thread is executing.

Exception Thrown By Parent

If a thread throws an exception, it must be handled in the thread where the exception occurs. What happens if this thread is the parent of another thread?

If an exception is thrown in the parent thread, the destructor is called for every object in scope. This will include any std::thread objects which were created in that scope.

The std::thread destructor checks whether join() or detach() have been called on that object. If they have, the destructor terminates the system thread which is associated with the object.

If neither join() or detach() have been called, the destructor calls std::terminate(), which will immediately terminate the entire program by default. For example:

```
// Start of a scope
```

```
{
```

```
    // Create a thread object
```

```
    std::thread t{ ... };
```

```
    // Code that might throw an exception
```

```
    ...
```

```
    // If an exception is thrown before here,
```

```
// join() is not called

t.join();

}
```

If an exception is thrown in the ... code, the program will call t's destructor and exit the scope immediately, before the call to join(). Since t has not been joined or detached at that point, the destructor will call std::terminate().

Try/Catch Solution

The obvious solution is to put a try/catch block around code that might throw an exception:

```
// The start of a scope

{

    // Create a thread object

    std::thread t{ ... };

    try {

        // Code that might throw an exception

        ...

    }

    catch(std::exception& e) {

        // Call join()

        t.join();

        // Rethrow the exception
    }
}
```

```
    // so it can be handled by the caller  
  
    throw;  
  
}
```

```
    // Normal program flow  
  
    // join() is called  
  
    t.join();  
  
}
```

This will work, but it is verbose and not very elegant. There is a lot of “boilerplate” code which does not provide any functionality. Also, it has to be repeated in every situation where a parent thread could throw an exception.

RAII Solution

A better solution is to use the “RAII” idiom. This is a common technique for writing C++ classes which manage resources. The constructor acquires ownership of the resource, and the destructor releases it. The copy and move operators are written so that ownership is transferred to the target object, but only if it can be done safely.

If an object of an RAII class is created on the stack, it will be destroyed automatically and immediately when it goes out of scope, including when an exception is thrown. This ensures that the resource is always released and gives a very strong protection against resource leaks.

In this case, we are concerned about unjoined threads, rather than resource leaks, but we can use RAII to solve this as well.

We make the class’s destructor call `join()`. Here, we need to be careful, because a thread should only be joined or detached once.

Fortunately, `std::thread` has a `joinable()` member function, which we can use to

check this. If `join()` or `detach()` have already been called on the object, `joinable()` will return false, and we will know not to call `join()` again.

`joinable()` can also return false if the `std::thread` object does not have an execution thread associated with it. This can happen if a thread object is default constructed, without being given an entry point function. It can also happen if the associated execution thread has already completed. In this case, we should not call `join()` either.

If `joinable()` returns true, then the thread has not been joined or detached, and we need to call `join()` before the object is destroyed.

RAII Solution Class

We will write an RAII class which has an `std::thread` object as a private member. The constructor will move its argument into this member. This is necessary, because `std::thread` is a move-only class.

The constructor is declared explicit, to prevent it being used as a conversion operator. This is probably not necessary here, but it is generally good practice to make single-argument constructors explicit.

Typically, resource management classes are not copyable, as it is not desirable to have two objects which both think they own the same resource. We declare the copy constructor and copy assignment operator as `=delete`. Calling either of these special member functions will give a compiler error.

Since we do not define move operators, any move operations will use the copy operator equivalent, which will again give a compiler error. It is possible to implement these functions, but care must be taken to make sure that the original execution thread does not continue to run after its `std::thread` object has been moved into. This requires similar code to the destructor.

The destructor will join the thread if necessary, otherwise it does nothing. Once this destructor returns, the destructor for the `std::thread` member will be called.

```
class thread_guard {  
  
    std::thread t;
```

public:

// explicit constructor prevents conversions

// std::thread is move-only, so the

// constructor takes an rvalue reference

explicit thread_guard(std::thread&& t) :

t(std::move(t)) {}

// Destructor

~thread_guard()

{

// Join the thread if not already joined

if (t.joinable())

t.join();

}

// Prevent copying by declaring

// the copy operators as deleted

thread_guard(const thread_guard&) = delete;

thread_guard& operator=(const thread_guard&) = delete;

};

RAII Class Usage

The `thread_guard` class would be used like this:

```
// Start of a scope

{

    // Create an std::thread object

    std::thread thr{ func };


    // Move it into an std::thread_guard object

    thread_guard tg{ std::move(thr) };


    // Do something

    ...


    // End of scope - destructors are called

}
```

In C++, destructors are always called in the reverse order to how the constructors were called.

`tg`'s destructor is called first. If necessary, it calls `join()` on its `t` member and waits for the associated execution thread to finish. The `t` member is then destroyed.

Finally, `thr`'s destructor is called. Since this no longer has an associated execution thread, it is not joinable and its destructor does not call `std::terminate()`.

This applies to both normal execution and when an exception is thrown.

Using this `thread_guard` class, we have replaced several lines of “boilerplate” code by a single statement.

Stopping Threads

In many languages, the thread implementation allows the program to interrupt or stop a thread. This is not directly supported in C++.

In general, abruptly terminating a thread in mid-execution is not a good idea. This could result in memory leaks, incomplete transactions, unreleasable locks or even heap corruption. Most language implementations use cooperative interruption, in which a thread is allowed to choose the time of its termination. This gives the thread the opportunity to tidy up before it leaves.

It is possible to call an operating system API which stops the execution thread, via `native_handle()`. Alternatively, the Williams book ¹ has an `interruptible_thread` class, which implements cooperative sharing.

Chapter Nine

Multiple Threads

How to Work With Multiple Threads

Starting Multiple Threads

To start multiple threads, we just create multiple `std::thread` objects. However, the interactions between multiple threads can be rather involved. For the moment, we will “dodge” these problems.

In the following code, we have an entry point function which displays a message. Each thread will be paused for a different interval of time after it starts.

These intervals are arranged so that a thread will not display its message until the previous thread has completed. In effect, the threads run sequentially, instead of concurrently. This avoids having to deal with interactions between the threads.

We have to convert the function argument to seconds. This is because `std::chrono` uses its own types. This avoids the ambiguity of literals of built-in types (“does that 50 mean 50 milliseconds or 50 seconds?”)

```
#include <iostream>
```

```
#include <thread>
```

```
#include <chrono>
```

```
// Thread entry point function
```

```
void hello(int num)
```

```
{
```

```
    // Convert the argument to seconds

    auto delay = std::chrono::seconds(num);

    std::this_thread::sleep_for(delay);

    std::cout << "Hello from thread " << num << "\n";

}
```

```
int main()

{

    // Start three threads

    std::thread t1{hello, 1};

    std::thread t2{hello, 2};

    std::thread t3{hello, 3};


    // Wait for them to complete

    t1.join();

    t2.join();

    t3.join();

}
```

The output from the program will look like this:

Hello from thread 1

Hello from thread 2

Hello from thread 3

main() launches the threads. t1's execution thread will sleep for one second, then display its message. t2's execution thread sleeps for one more second before displaying its output, and so on.

Thread Scheduling

The way that threads execute is controlled by the operating system, which has a "thread scheduler". This organizes the thread executions and tries to produce the optimal performance for the system as a whole.

The thread scheduler decides which core a thread will run on, and how long it is allowed to run for before it is task switched. After the task switch, another thread will run on the same core and the first thread will be suspended. It will not execute any more instructions until it is scheduled to run again. This could be on a different core from the previous one.

- The thread scheduler may start threads in any order.
- The thread scheduler is allowed to interrupt threads at any time while they are executing.
- The thread scheduler can restart an interrupted thread at any time.
- While a thread is inactive, another thread can run. This can happen if the first thread is interrupted, but also if it is sleeping or waiting for something.
- A running thread can execute the same instructions as an inactive thread. It can also access data which it shares with the inactive thread.

This means we cannot make any assumptions about the order in which threads run, or when they run. We cannot even assume that they start in the same order in which the `std::thread` objects are created.

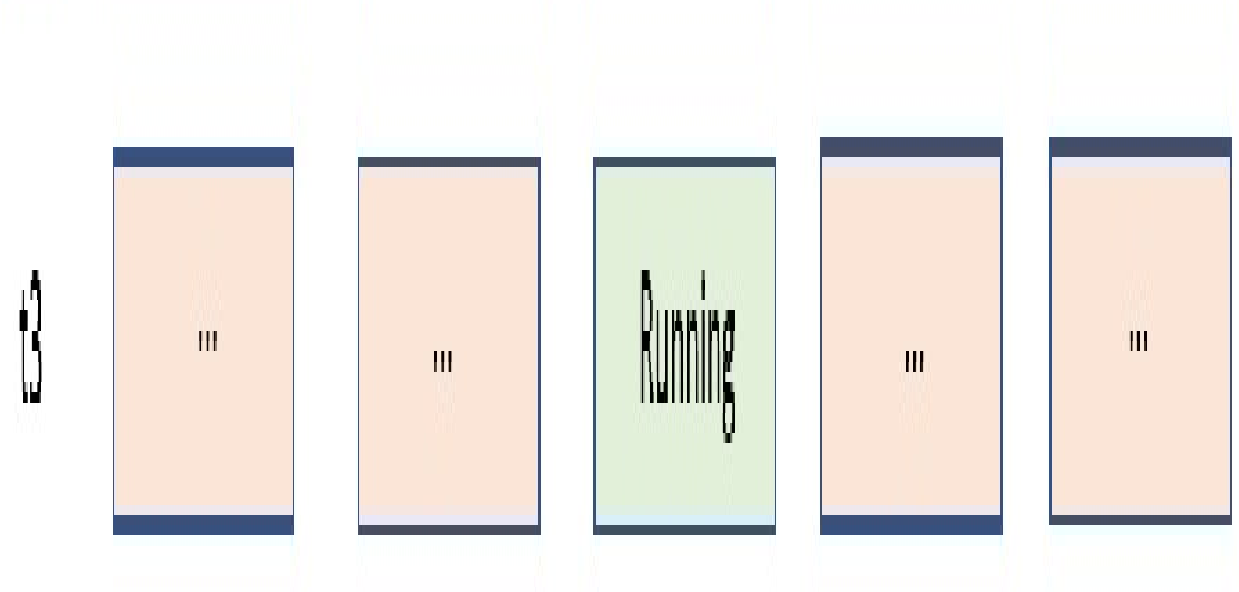
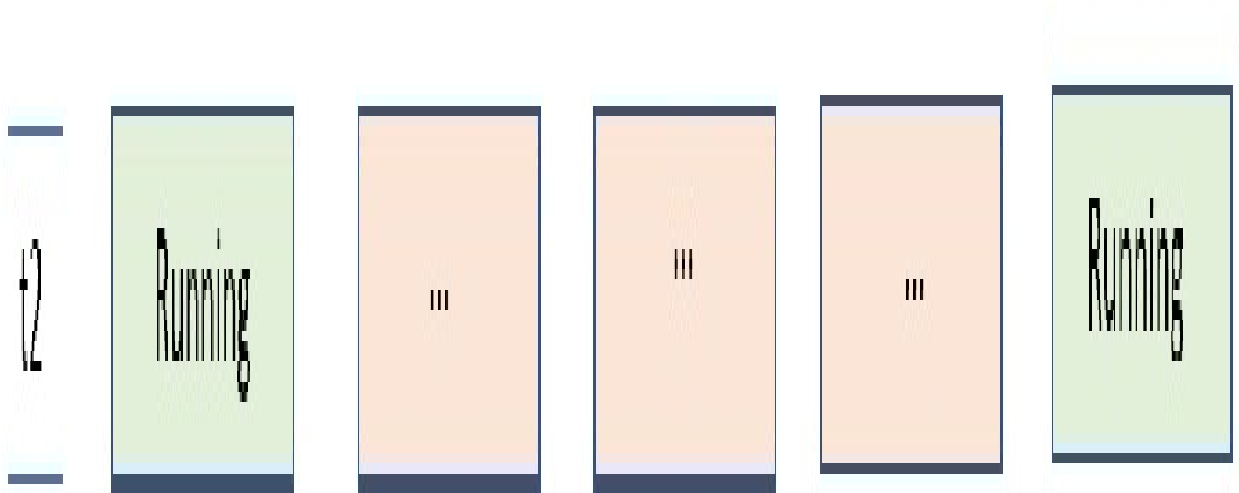
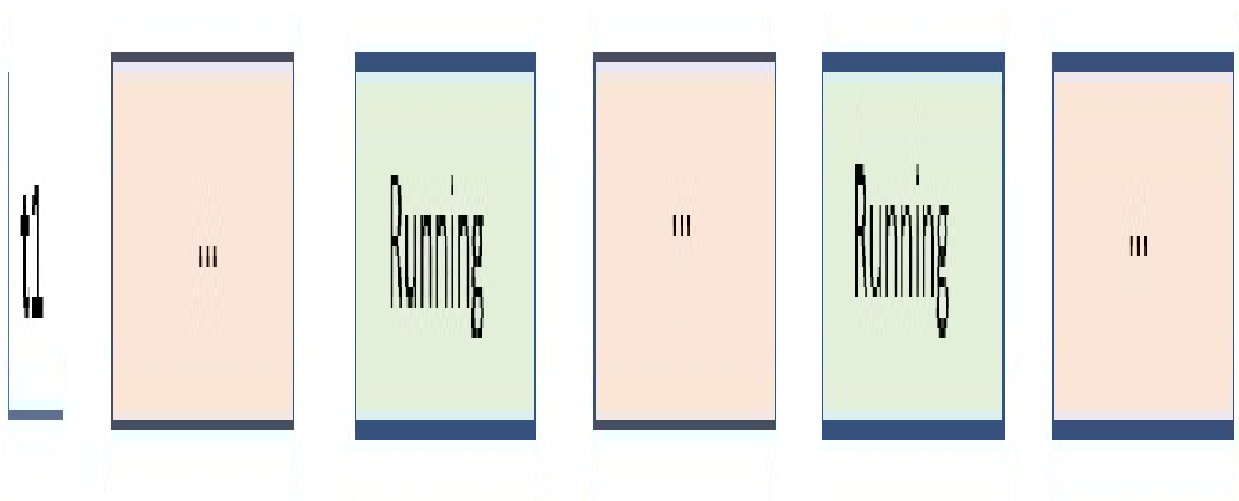
This is why the example program has sleeps in it, to make the threads appear to run in the same order as they appear in the source code. The sleep interval is

very long by processor core standards - one second - to prevent task switching from having any impact.

Thread Scheduling of the Example Program

In our program, the execution of the threads is interleaved. One possible order of execution is:

- t2 starts, then is interrupted
- t1 starts, then is interrupted
- t3 starts, then is interrupted
- t1 restarts, executes some instructions, then is interrupted
- t2 restarts, executes some instructions, then is interrupted



Threads running on a processor core are task switched. Each thread gets to run for a short interval of time. This happens in no particular order.

In effect, the threads appear to execute "inbetween" each other. This is known as "interleaving".

Data Sharing Between Threads

Sharing data between threads is very easy to arrange. The only requirement is that the shared data variable is visible in the entry point functions. This is because each thread has its own execution stack, so local variables will not be visible to other threads.

- For free functions, use a global or static variable
- For member functions, use a class member

However, interleaved execution means that threads can interfere with each other while the shared data is being modified

- A value written by one thread can be overwritten by another thread, without the first thread's knowledge.
- A value can be partially written by one thread and partially written by another thread.

Unsafe accesses to shared data is the largest source of errors in threaded programs; preventing them is the biggest source of difficulties.

Data Races

An unsafe access to shared data is known as a "data race".

C++ has very precise definition of a data race. The entire threading implementation is based on this definition. Understanding data races is essential to writing safe threaded programs in C++

A data race can occur when there are potentially conflicting accesses to the same

“memory location”. This occurs when two or more threads access the memory location, and at least one of the threads is trying to modify it.

This is safe only if the accesses are synchronized in some way. The programmer can arrange things so that each accesses will “happen before” the next, so the threads make their accesses in turn, instead of concurrently.

In our example, we did this in a very crude way, by making the threads sleep in such a way that each thread executed in turn. We will learn a better way to do this later on.

The alternative is to make the accesses “atomic”, which means a single operation which cannot be interrupted.

If the accesses are not synchronized, a data race occurs and the program’s behaviour is undefined. This means the program is not guaranteed to behave consistently.

This is so important that I will say it again.

If the accesses are not synchronized, a data race occurs and the program’s behaviour is undefined. This means the program is not guaranteed to behave consistently.

Memory Location

C++ defines a memory location as a scalar object, or sub-object. This means a simple object, as opposed to a compound object such as an array or a struct.

- Variable of built-in type
- A pointer
- A container element which is a scalar object
- A class or struct member which is a scalar object

And also, an obscure edge case, relating to bitfields:

- Bitfields within the same word, which are contiguous, and not separated by a

bitfield of zero length

Compound Objects

C++ does not place any restrictions on types that we create ourselves. We can create classes which are internally synchronized, and use them in threaded programs without needing to provide any extra code to synchronize them. However, the internal synchronization will make them less efficient in single-threaded programs, or in threaded programs where there is not a data race. Programmers may also not realize that the object is already synchronized and add their own code to do this. In addition to the obvious wasted effort, this can cause serious problems.

C++ implements the containers in the Standard Library as memory locations. This means that multiple threads accessing the same container object can conflict, and the programmer must prevent a data race by adding synchronization. However, if the threads are accessing different container objects, or containers with different types, then there is no conflict and no need for synchronization.

This means that the C++ standard containers are as efficient in multi-threaded programs as in single-threaded programs, unless there is a potential data race. It also avoids the need to define and implement a multithreaded version of the C++ library.

Usually it is good practice to make our own types behave like the ones provided by C++. This avoids the complications of having different rules depending on who wrote the code.

Race Condition

Another problem in multi-threading is the “race condition”. This is when the outcome of a program can be different, depending on how the threads are scheduled to run.

For example, if we took out the sleeps in our program, it is possible that the messages could be printed in a different order. And we could get a different order the next time we run the program (Remember, the threads could be started

in any order, not just the order they are defined in the source code!)

This is timing-dependent code, in which different runs of the same program may produce different outcomes.

A data race is a special case of a race condition. Depending on how threads are scheduled to run, data corruption may result.

In the next chapter, we will see examples of the problems that data races can cause.

Chapter Ten

Data Races

Examples of Data Races

Data Race Example

To remind ourselves, a data race can occur when there are conflicting accesses to the same memory location from multiple threads.

The example in the previous chapter had a data race. But where is the shared data?

In C++, the standard output is represented by `std::cout`. This is a global stream variable, which is shared by all the threads in a program. Every time the program displays output, it modifies `std::cout`. If we have multiple threads displaying output at the same time, without synchronization, then we have the conditions for a data race.

In the last chapter, we were careful to make sure that threads could not display output at the same time, by making them sleep. We will now find out what happens when the sleeps are removed.

```
#include <iostream>
```

```
#include <thread>
```

```
#include <chrono>
```

```
using namespace std::literals;
```

```
int main()
```

```
{  
  
    // Thread entry point function  
  
    auto print = [](const std::string& str)  
    {  
        for (int i = 0; i < 5; ++i) {  
            std::cout << str[0] << str[1] << str[2] << "\n";  
            std::this_thread::sleep_for(50ms);  
        }  
    };  
};
```

```
    // Start the threads  
  
    std::thread t1{print, "abc"};  
    std::thread t2{print, "def"};  
    std::thread t3{print, "xyz"};  
  
    t1.join();  
    t2.join();  
    t3.join();  
}
```

Here, we are trying to encourage the threads to display output at the same time. The output statement has been broken up into separate calls to operator <<; this increases the probability of the thread being interrupted, as there is more code

for the thread to execute. The sleep also gives the thread scheduler more opportunities to interrupt the thread.

For variety, we have used a lambda expression as the thread entry point, instead of a free function.

Output from Data Race Example

The output could look like this

axbydzce

f

axdbyeczf

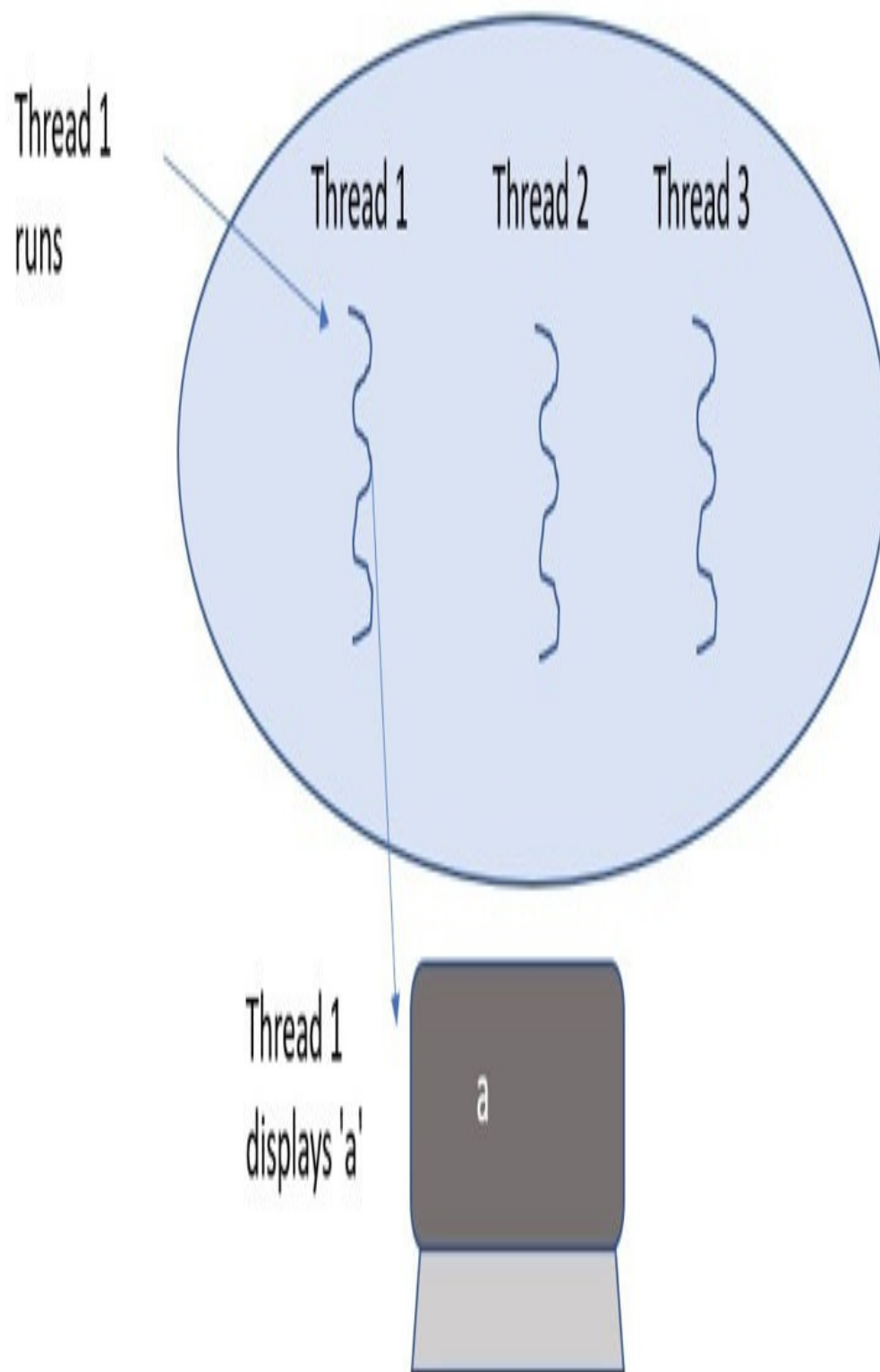
t1 displays the letter ‘a’, then gets interrupted before it can display the letter ‘b’. t3 runs and displays the letter ‘x’, then is interrupted. t1 resumes and displays the letter ‘b’, then is interrupted by t3, which displays the letter ‘y’. t2 now runs and displays the letter ‘d’ before it is interrupted. And so on.

Another program run could produce quite different output, depending on how the thread scheduler decides when the threads should run. The results are unpredictable, because the threads can interleave in different ways each time.

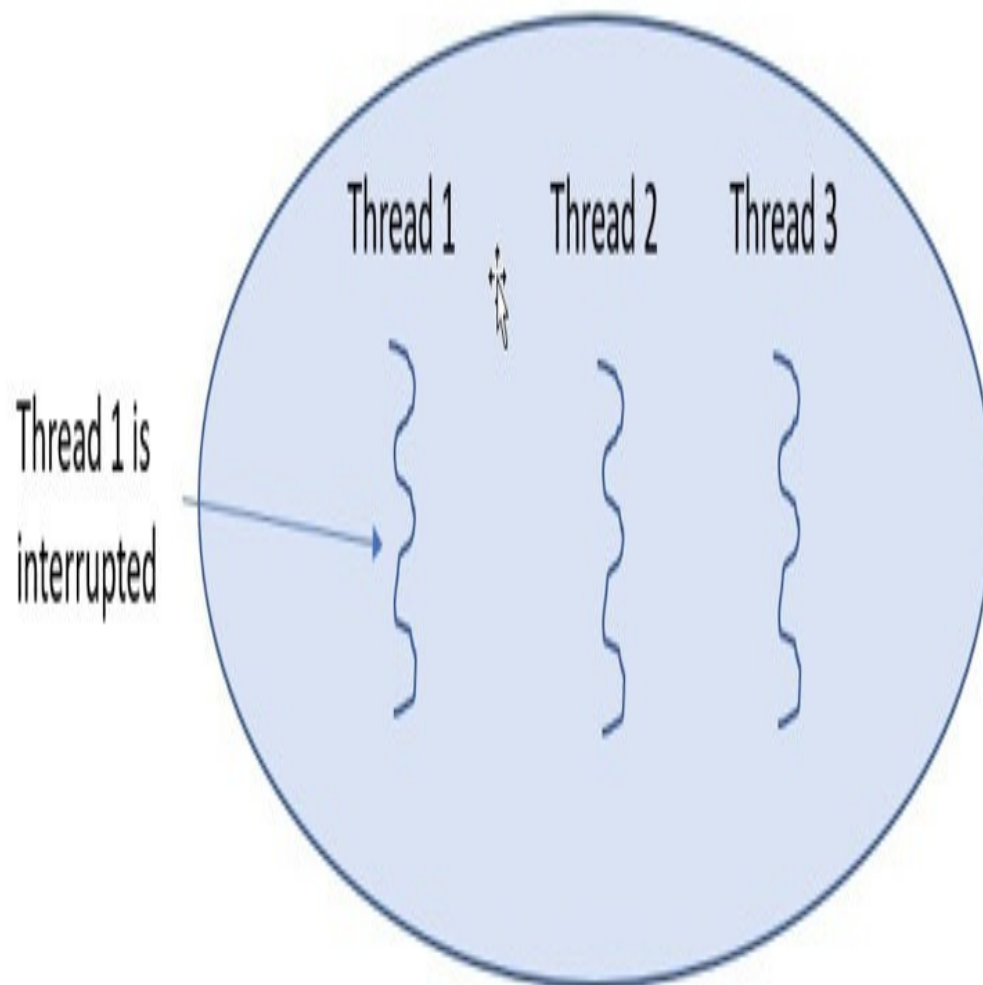
Write with Data Race

The blue oval represents the program. The squiggles represent threads of execution following their code paths.

Thread 1 runs first. It displays ‘a’, the first character of “abc”.



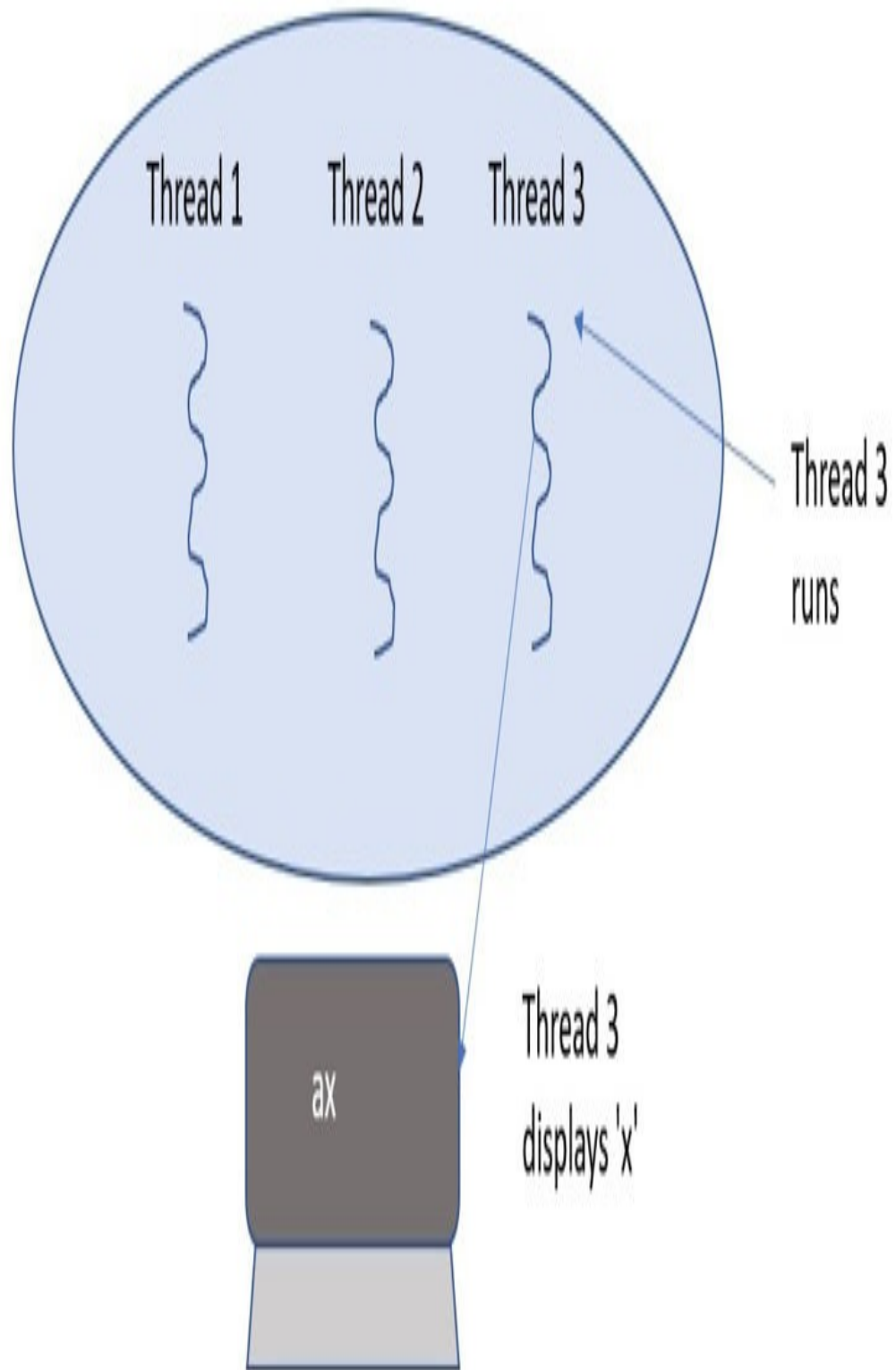
Thread 1 sends 'a' to std::cout



Thread 1 is interrupted before it can send 'b' to `std::cout`

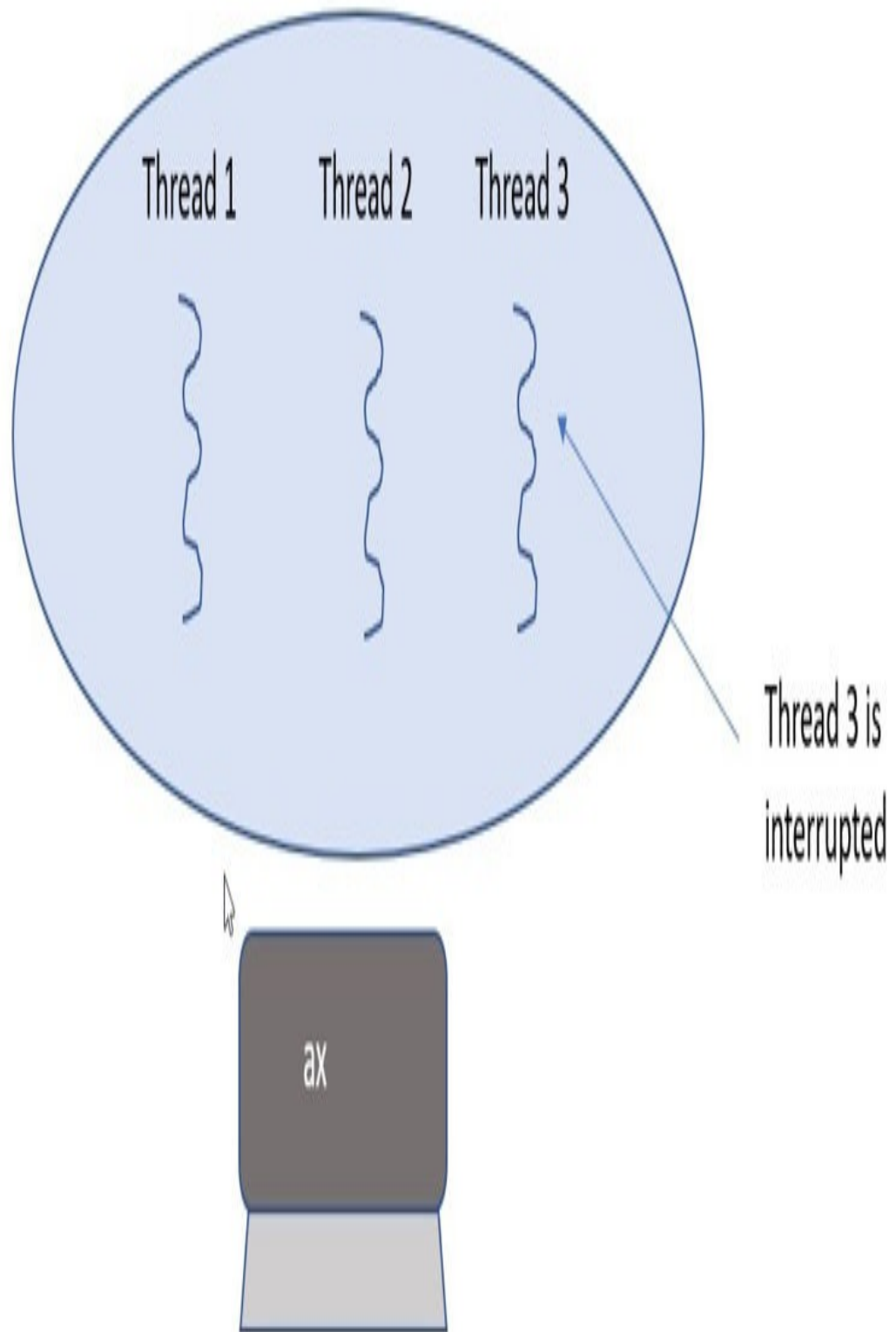
Before it can display the next character, it is interrupted.

Thread 3 then runs and displays its first character



Thread 3 sends 'x' to `std::cout`

Again, it is interrupted before it can display the next character.



Thread 3 is interrupted before it can send 'y' to `std::cout`

And so on. Each thread is only able to display part of its string before it is interrupted and another thread writes to `std::cout`.

There is no synchronization between the threads. There is nothing to make one thread's access to `std::cout` happen before another thread's. Displaying the string is not an atomic operation, because a thread can be interrupted between each character.

Consequences of a Data Race

Our example of a data race produced interleaved output, in which the output from the threads was mixed together. In fact, `std::iostream` is specially designed so that interleaved output is the only consequence.

The results of a data race can be much more serious:

- Computations have incorrect results
- Incorrect program flow
- “Torn” reads and writes of data variables
- Objects which are not properly constructed or destroyed

Please note that the following examples demonstrate how to write code which is not thread-safe. They are deliberately written using bad code and should not be used as "models".

Incorrect Results

// Shared data variable

```
int x;
```

// The entry point of a thread

```
void calc()
```

```
{
```

// Use x without synchronization

```
int a = x*2 + 3;
```

```
int b = x*3 + 2;
```

```
}
```

Imagine that a thread is executing the code in `calc()`. The thread is interrupted between calculating `a` and calculating `b`.

Another thread, executing a different function, now runs and modifies the shared variable `x`, again without synchronization.

The first thread resumes execution and calculates `b`, using the new value of `x`. This will be inconsistent with `a`, which was calculated using the old value.

This can even occur inside statements:

```
y = x*x + 2*x + 1;
```

Each multiplication and addition is a separate operation; a thread can be interrupted between any of these operations. Another thread could modify `x` before the first thread resumes execution and continues evaluating the statement.

Incorrect Program Flow

Imagine we need the square root of a number. We wish to be careful and make sure that the number is positive:

// Shared data variable

```
int x;
```

```
// The entry point of a thread
```

```
void calc()
```

```
{
```

```
    // Check whether x is positive
```

```
    if (x > 0) {
```

```
        // It's safe to use x (1)
```

```
        int z = std::sqrt(x);
```

```
    }
```

```
}
```

The thread could be interrupted at (1), between the $x > 0$ check and the `sqrt(x)` call. Another thread could run and change `x` to have a negative value. The first thread then resumes execution and passes this negative value of `x` to `std::sqrt()`. This causes a runtime error which “can’t possibly happen”.

Incorrect Program Flow Continued

Another example is bounds-checking the variable in a switch statement.

```
// Check whether x is within bounds
```

```
if (x >= 0 && x <= 9) {
```

```
    // It's safe to use x (1)
```

```
    switch(x) {
```

```
        case 0:
```

```
            func0();
```

```
        break;

    case 1:

        func1();

        break;

    ...

    case 9:

        func9();

        break;

}

}
```

Another thread could interleave at (1) and modify `x`, between the check and the switch statement. If the new value of `x` is out of bounds, none of the case statements will be executed.

In fact, there is an even nastier bug hiding here.

Optimizing compilers may well convert this to an array of function pointers known as a “branch table”. This has the case label as the index, so `table[0]` holds the address of `func0`, `table[1]` holds the address of `func1`, and so on.

When `x` is 0, the program will jump to the address in `table[0]` and execute the code there.

If another thread does modify `x` to 10, for example, then the original thread will jump to the address in `table[10]`, which is an arbitrary location, and execute the code there. The results could be disastrous!

Torn Reads and Writes

“Torn” writes can occur when data cannot be written in a single operation. This

can happen on 32-bit systems which support 64-bit integers, but the hardware can only write 32 bits at a time.

For example:

- Thread A writes the first 32 bits of its number and is interrupted
- Thread B runs and writes the first 32 bits of its number to the same memory location
- Thread B runs and writes the second 32 bits of its number to the same memory location
- Thread A resumes and writes the second 32 bits of its number

The memory location will now contain the first 32 bits of B's number, followed by the second 32 bits of A's number.

Similarly, a “torn” read can occur when a thread which is reading a variable is interrupted by a thread which writes to the variable. The first thread will see the first 32 bits of the old value, followed by the second 32 bits of the new variable.

Object not Properly Constructed

A thread is creating an object by calling its constructor. If it is interrupted by another thread, which also calls the constructor for the same memory location, we have a torn write. The object will be initialized with a mixture of values from each thread.

```
vector<Base *> vec;
```

```
Base::Base() { ...; vec.push_back(this); }
```

```
Derived::Derived() : Base(), ...
```

When the Derived constructor is called, it will call the Base constructor before initializing the Derived object. The Base constructor will push a pointer to this partially initialized object onto vec.

If another thread accesses that element in vec, before the Derived constructor has

completed, it will see a partially constructed object.

Improper Destruction

C++11 introduced `std::shared_ptr`, which is a reference-counted object. A counter is incremented every time an `std::shared_ptr` object is copied, and decremented every time one of these copies is destroyed. When the counter reaches zero, there are no more references, and the object and its management data are destroyed.

If the destructor for `std::shared_ptr` allows data races, there could be serious problems. Consider the following (unsafe) code:

```
// In the destructor of a reference-counted object
```

```
// Decrement the reference counter
```

```
*ref_count--;
```

```
// If there are no more references (1)
```

```
if (*ref_count == 0) {
```

```
    // Destroy the object and the counter
```

```
    delete p;
```

```
    delete ref_count;
```

```
}
```

The problem is that the thread could be interrupted at (1), between decrementing `ref_count` and comparing `ref_count` to 0.

If the other thread also decrements `ref_count`, there are two problematic scenarios:

- `ref_count` is initially 2. Thread A decrements `ref_count` to 1, then is interrupted.

Thread B decrements `ref_count` to 0 and deletes the pointers. Thread A resumes, sees that `ref_count` is 0, and deletes the pointers again!

- `ref_count` is initially 1. Thread A decrements `ref_count` to 0, then is interrupted. Thread B decrements `ref_count` to -1 and does not delete the pointers. Thread A resumes, sees that `ref_count` is -1, and does not delete the pointers either.

This is not a problem with C++'s `std::shared_ptr`, which specifies that all operations on the counter are thread-safe.

Managing Data Races

Data races can be very difficult to detect, as they rely on timing coincidences. This often results in mysterious, intermittent bugs. Often, the problem will only appear when the system is heavily loaded.

The best way to deal with data races is to prevent them occurring in the first place.

A data race occurs when multiple threads make conflicting accesses to shared data. To prevent a data race:

- Always access data in the same thread. Do not share data between threads unless necessary
- Make each access to shared data “happen before” the next. Impose an ordering on how the threads access the shared data
- Alternatively, only use “atomic” operations when accessing shared data. These operations cannot be interrupted by other threads.

In the next chapter, we will find out how to impose ordering on threads which access shared data.

Chapter Eleven

Working With Shared Data

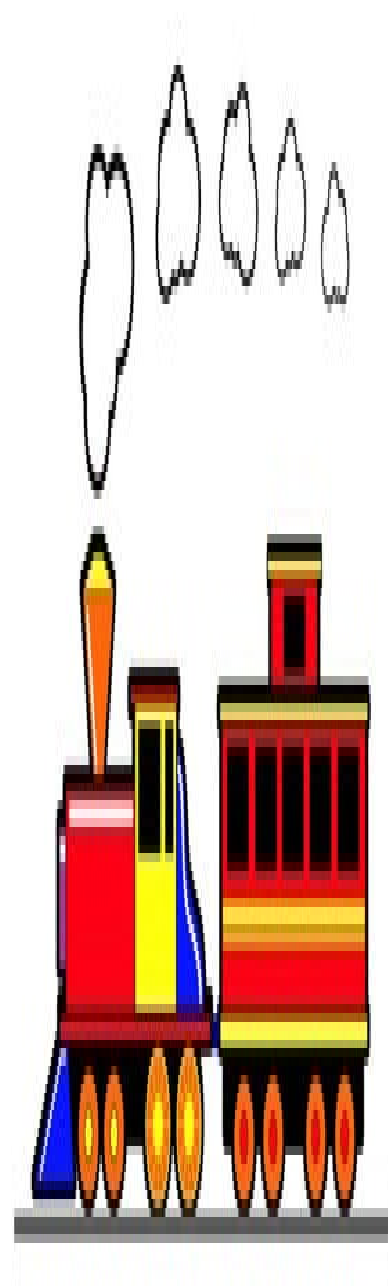
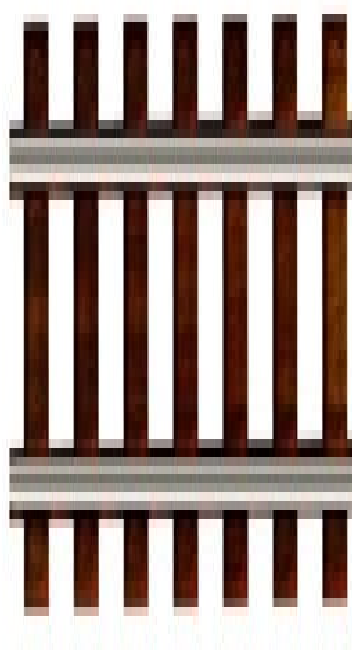
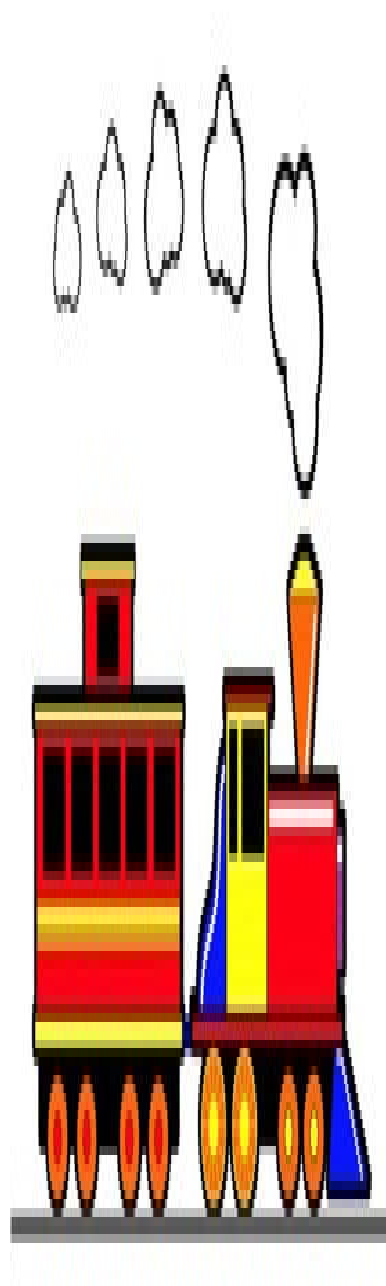
Synchronizing Threads with Mutexes

Mutex Introduction

We saw in the last chapter that data races can occur when multiple threads access the same memory location. Data races are highly undesirable, to put it mildly, but we can avoid them by imposing some sort of ordering on the threads as they access the location.

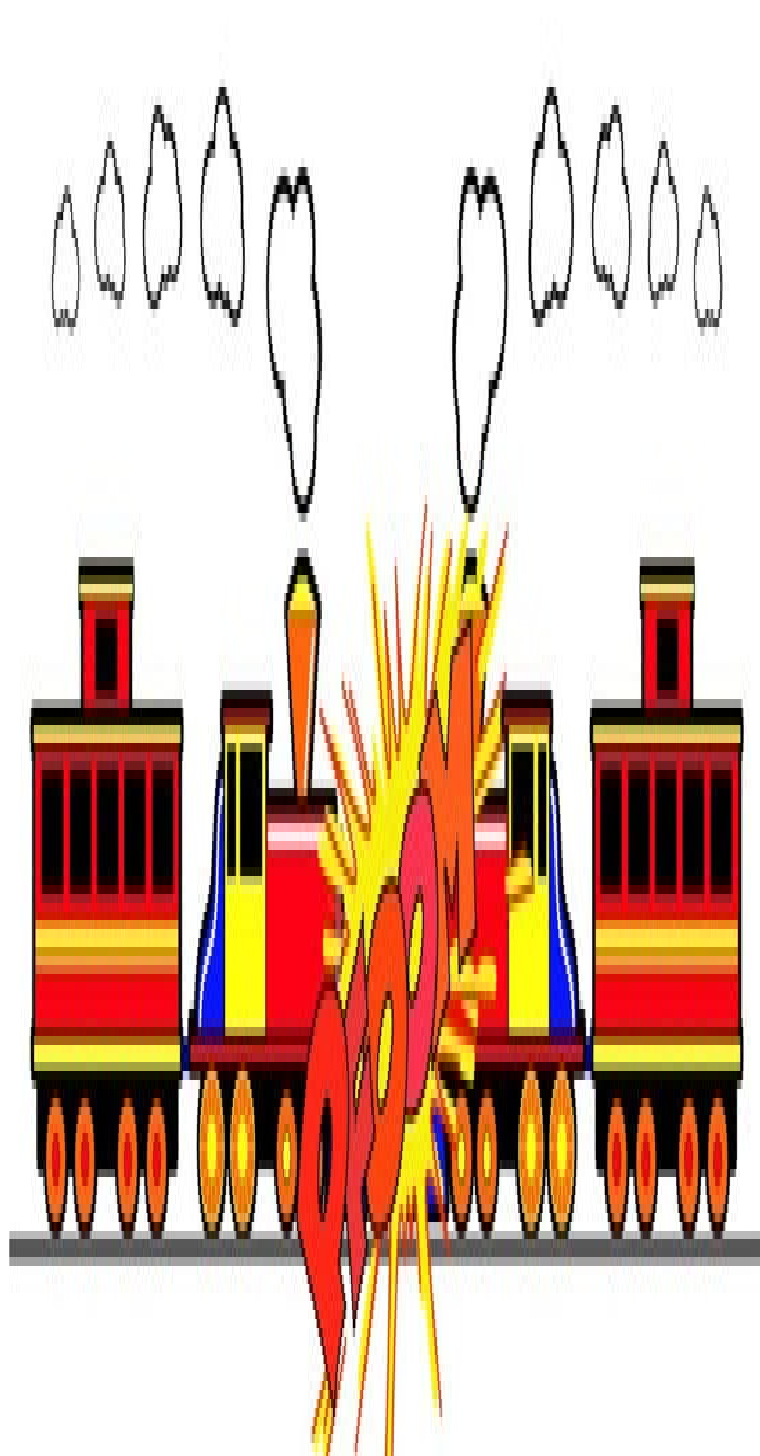
Single-Track Railway

Imagine we have a section of single railway track, and two trains which wish to run on it.



Two trains want to run on the same section of track

If they both try to enter the track section, there will be a collision.



Two trains try to run on the same track at the same time

Railways prevent this by imposing an ordering on the trains. The first train to arrive runs on the track; the train which arrives later has to stop and wait until the first train has left the section.

Critical Section

We have a similar problem in threaded programs. The trains represent the threads, the track represents code that accesses shared data, and the collision represents a data race.

There is a region in the code that must execute in isolation from the rest of the program. Threads must not be able to interfere with each other while this code executes.

The technical term for this is a “critical section”. With a potential data race, the critical section is the code that accesses the shared data.

We want to allow only one thread to execute the code in the critical section at any one time.

Mutex

One way to do this is with a “mutual exclusion object”, or “mutex” for short. A mutex is a simple data structure which has two states: “locked” and “unlocked”.

The threads which access shared data agree to use a mutex to control access to it.

A thread will only execute the critical section if the mutex is unlocked. It will then set the mutex state to “locked”, to indicate to other threads that the critical section is not currently accessible. When it finishes executing in the critical section, it sets the mutex state back to “unlocked”.

If a thread wishes to execute in the critical section, but the mutex state is “locked”, the thread must stop and wait for the mutex state to change to “unlocked” before it can continue.

This will impose an ordering on the threads which access the shared data, and this will prevent a data race. However, it relies on all the threads playing by the same rules.

If a train driver ignores a signal, there will be a collision. If a thread accesses shared data without checking the mutex state, there will be a data race.

Mutexes and Threads

A thread can only access shared data if the relevant mutex is in the “unlocked” state.

- Threads A, B, C wish to enter the critical section
- Thread A sets the mutex state to “locked”
- Threads B and C must wait until the mutex state changes to “unlocked”
- Thread A executes the code in the critical section, without interference from other threads
- Thread A finishes executing in the critical section and sets the mutex state to “unlocked”
- One of threads B and C can now set the mutex state to “locked” and enter the critical section

In the critical section, thread A can modify shared data without causing a data race, because thread A's accesses to the shared data “happen before” any access by thread B or thread C.

If thread A modifies the shared data, the new value will be stored before the other threads resume execution. Threads B and C will always see the new value.

C++ Mutex Class

The C++ library provides an `std::mutex` class, which represents a mutex implemented in the operating system. It is defined in the `<mutex>` header.

An `std::mutex` object cannot be copied or moved.

As with a shared data variable, an `std::mutex` object must be defined outside the entry point functions of the threads which use it.

- For free functions, use a global or static variable
- For member functions, use a class member

`std::mutex` has three main member functions.

`lock()` sets the mutex's state to locked. This is only possible if the state is currently unlocked. If the mutex's state is already locked, this call will “block”; the thread will stop and wait until the state changes to unlocked.

`try_lock()` is similar, but returns immediately if the mutex's state is locked. It returns a `bool` which is true if the thread was able to set the mutex's state from unlocked to locked, otherwise false.

This is useful if you do not want the thread to wait indefinitely to get the lock. The thread could do some other work, then try again later.

`unlock()` sets the mutex's state to unlocked.

For convenience, programmers usually say “lock the mutex” when they mean “set the mutex's state to locked”. They also say “enter the critical section” as shorthand for “start executing the code in the critical section” and “leave the critical section” as shorthand for “finish executing the code in the critical section”.

`std::mutex` Example

Here is the data race example from the previous chapter, rewritten to use a `mutex`.

There is a critical section in this program, where the threads display output. This modifies `std::cout`, which is a global variable. Before, we did this without any synchronization of the threads, which caused a data race.

```
#include <iostream>
```

```
#include <thread>
```

```
#include <chrono>
```

```
#include <mutex>
```

```
using namespace std::literals;
```

```
// Mutex to protect the critical region
```

```
std::mutex print_mutex;
```

```
auto print = [](const std::string& str)
```

```
{
```

```
    for (int i = 0; i < 5; ++i) {
```

```
        // Lock the mutex before the critical section
```

```
        print_mutex.lock();
```

```
        // The critical section, which modifies std::cout
```

```
        std::cout << str[0] << str[1] << str[2] << "\n"
```

```
        // Unlock the mutex after the critical section
```

```
        print_mutex.unlock();
```

```
        std::this_thread::sleep_for(50ms);
```

```

    }
};

int main()
{
    std::thread t1{print, "abc"};
    std::thread t2{print, "def"};
    std::thread t3{print, "xyz"};

    t1.join();
    t2.join();
    t3.join();
}

```

First, we create a global `std::mutex` object. This is used to control access to the critical section.

Immediately above the code for the critical section, the thread locks the mutex. If the mutex is already locked, this is because another thread is executing in the critical section. This thread must stop and wait for the other thread to leave the critical section before it can continue.

If the mutex is not locked, then no other thread is executing in the critical section, and this thread can continue. No other thread will be able to execute in the critical section until this thread leaves it.

Below the code for the critical section, the thread unlocks the mutex. This will allow other threads to lock the mutex and execute in the critical section.

The output could look like this:

abc

xyz

def

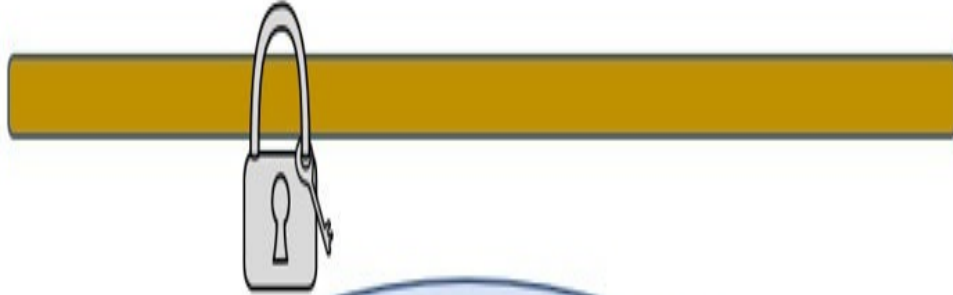
The output is no longer interleaved. Each thread is able to display all its output before another thread can interrupt it.

Notice that the output from t3 appears before the output from t2. This is because threads can start in any order, and can be interrupted at any time, and can resume execution in any order. When reasoning about multi-threaded code, it is very important not to make any assumptions which rely on sequential behaviour (“t1 runs first, displays its output, then t2 runs and displays its output, then t3”.) Unfortunately, this can be hard to do.

Write with Mutex

Thread 1 runs and locks the mutex

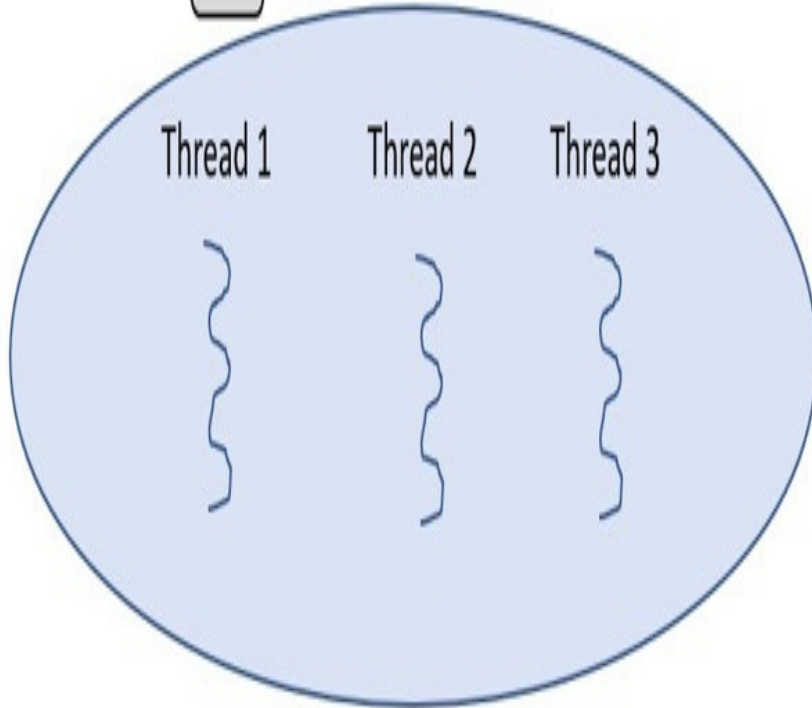
Thread 1
locks mutex



Thread 1

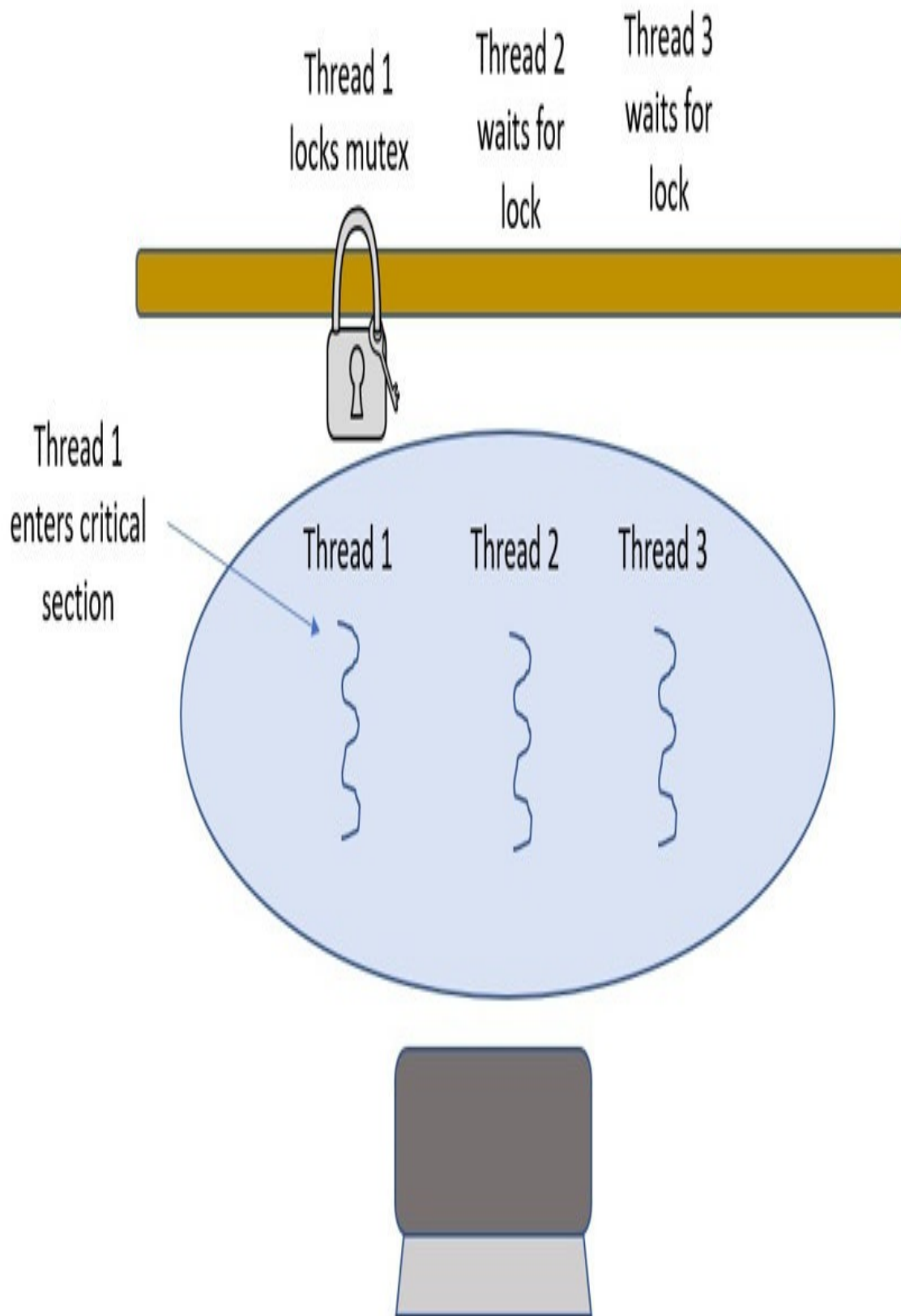
Thread 2

Thread 3



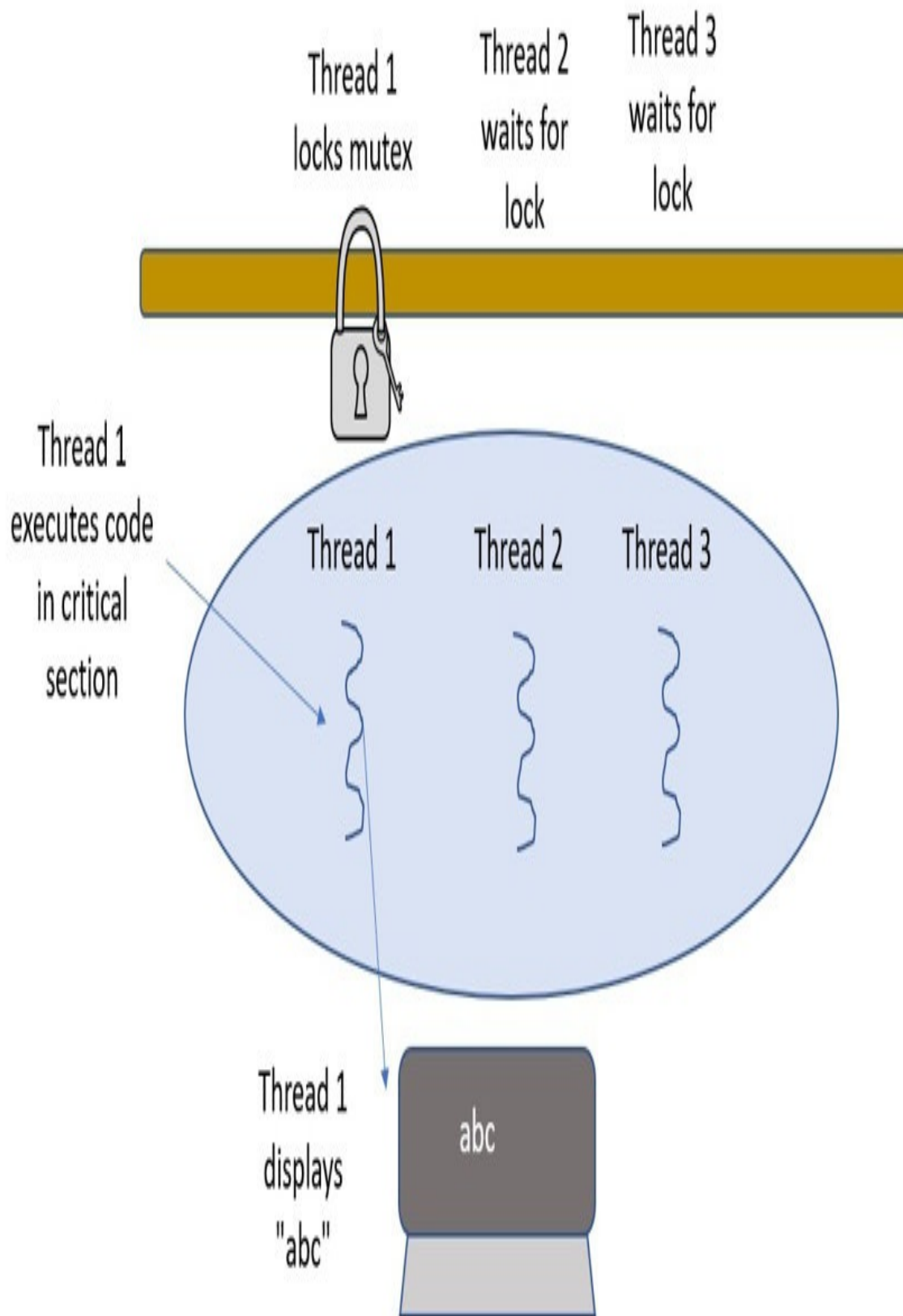
Thread 1 locks the mutex which controls access to the critical section

While thread 1 has the lock, no other thread can get a lock. It is safe for thread 1 to enter the critical section.



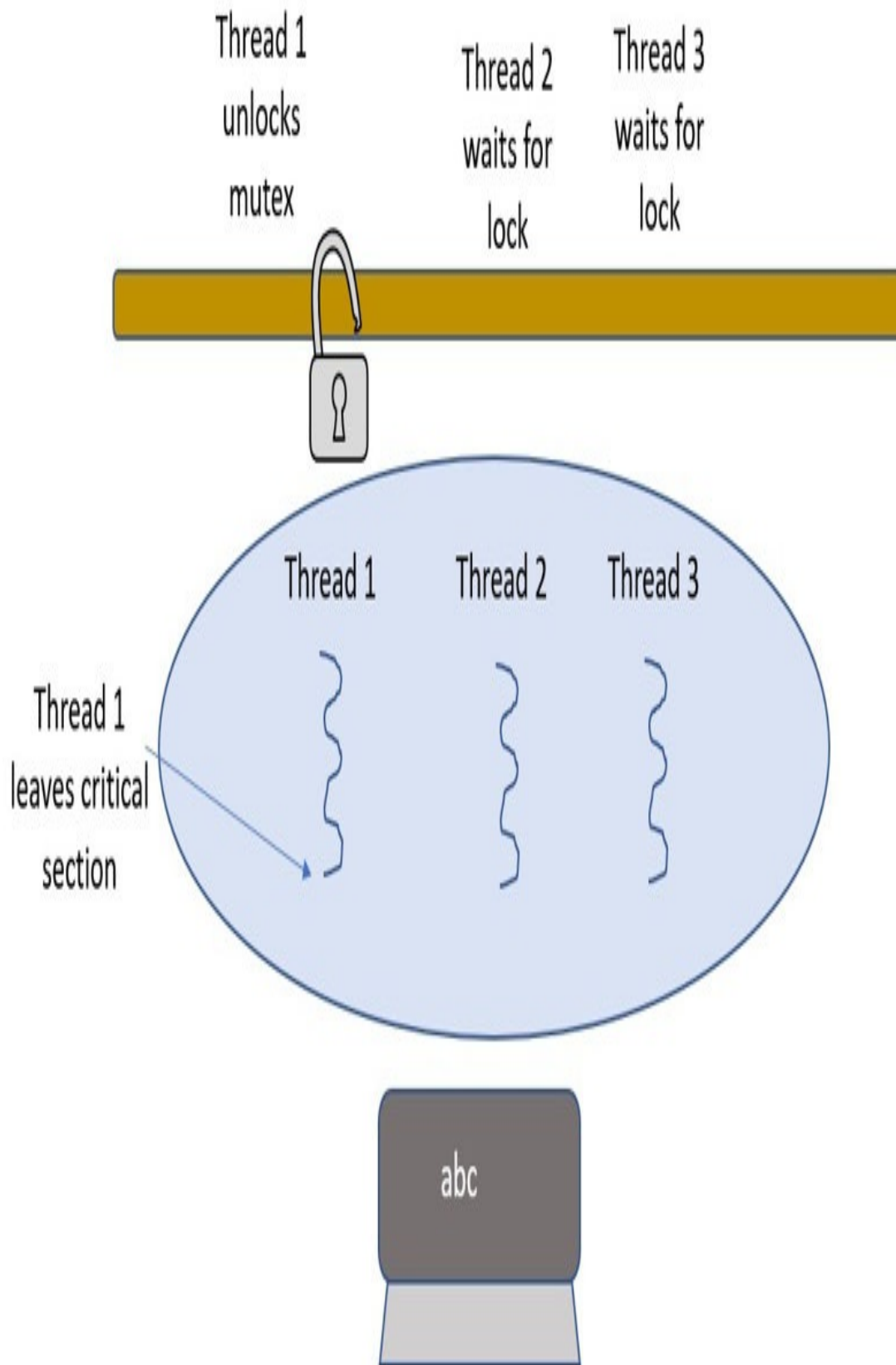
Other threads which want to access the critical section must wait for the lock

Thread 1 executes the code in the critical region without any interference.



Thread 1 writes all its output without other threads interfering

Finally, Thread 1 leaves the critical section and unlocks the mutex. This allows one of the other threads to lock the mutex and enter the critical section.



Thread 1 unlocks the mutex after it has executed the critical section

Mutexes and Data

It can often be useful to encapsulate a mutex. We do this by writing a class which has the shared data as a member, and the mutex as another data member. The public member functions of the class perform operations on the shared data, under the protection of the mutex.

As an example, we could write a thread-safe vector class, which has an `std::vector` member and an `std::mutex` member. The public member functions lock the mutex, forward the call to the corresponding `std::vector` member function, then unlock the mutex.

// Very simplistic thread-safe vector class

```
class Vector {  
  
    // Mutex as private data member  
  
    std::mutex mut;  
  
    // Shared data - the mutex controls access to it  
  
    std::vector<int> vec;  
  
public:  
  
    void push_back(const int& i)  
    {  
  
        //Lock the mutex  
  
        mut.lock();
```

```
// The critical section

vec.push_back();


// Unlock the mutex and return

mut.unlock();

}

};
```

This code is only meant as an illustration. A real implementation would use templates with perfect forwarding, and it would implement the entire `std::vector` interface.

Acquire and Release Semantics

A mutex is an example of “acquire and release semantics”.

When a thread locks a mutex, it “acquires” exclusive access to a critical section.

When the thread unlocks a mutex, it “releases” this exclusive access. It also releases the result of any modifications made while it was executing the code in the critical section.

The next thread which locks the mutex will acquire the result of those modifications.

These acquire and release semantics are how the ordering is imposed on threads which access the shared data. There is no data race, and the data is always in a consistent state.

`std::mutex::try_lock()` Example

As mentioned before, the `try_lock()` member returns immediately, returning a `bool` which indicates whether it succeeded in locking the mutex. Typically, `try_lock()` is called in a loop:

```
// Keep trying to lock the_mutex

while (!the_mutex.try_lock()) {

    // Could not lock

    // Do something else and try again later

    std::this_thread::sleep_for(100ms);

}

// Exiting the loop means success

// Now we can enter the critical section
```

Here is an example in which two threads try to lock the same mutex.

```
#include <iostream>

#include <thread>

#include <mutex>

#include <chrono>
```

```
using namespace std::literals;
```

```
std::mutex the_mutex;
```

```
void task1()
```

```
{  
  
    // Lock the mutex  
  
    std::cout << "Task1 trying to get lock\n";  
    the_mutex.lock();  
    std::cout << "Task1 has lock\n";  
  
    // Execute the critical section  
  
    // (pretend to be busy)  
    std::this_thread::sleep_for(500ms);  
  
    // Unlock the mutex  
  
    std::cout << "Task1 releasing lock\n";  
    the_mutex.unlock();  
}
```

```
void task2()  
{  
  
    std::this_thread::sleep_for(100ms);  
  
    // Try to lock the mutex  
  
    std::cout << "Task2 trying to get lock\n";
```



```

while (!the_mutex.try_lock()) {

    // Could not lock

    // Do something else and try again later

    std::cout << "Task2 could not get lock\n";

    std::this_thread::sleep_for(100ms);

}

// Exiting the loop means success

// Now we can enter the critical section

std::cout << "Task2 has lock\n";

the_mutex.unlock();

}

int main()

{

    std::thread t1{task1}, t2{task2};

    t1.join();

    t2.join();

}

```

Because t2 initially sleeps, t1 will lock the mutex first. t1 is then busy executing

its critical section, protected by the mutex. (In this book, we will often use a sleep to simulate a thread doing work.)

t2 will keep trying to lock the mutex. Eventually, t1 will unlock the mutex. The try_lock() call in t2 will succeed, and the thread will exit the loop and execute its own critical section.

The output will look like this:

Task1 trying to get lock

Task1 has lock

Task2 trying to get lock

Task2 could not get lock

Task2 could not get lock

Task2 could not get lock

Task2 could not get lock

Task1 releasing lock

Task2 has lock

Chapter Twelve

Lock Guard

An RAII Wrapper for `std::mutex`

Exception Thrown in Critical Section

`std::mutex` is a low-level object. It is little more than a wrapper around the operating system's mutex implementation. This causes some problems; in particular, when an exception is thrown.

Consider the following code fragment:

```
the_mutex.lock();
```

```
// Execute the critical section
```

```
// ... (1)
```

```
the_mutex.unlock(); // (2)
```

What happens if an exception is thrown at (1), in the critical section?

The normal C++ process of “stack unwinding” will occur. The destructors will be called for all objects in this scope, then the program will immediately exit this scope and search for a suitable handler for the exception. The `unlock()` call at (2) is never executed.

The mutex remains locked. Any thread which is waiting to lock this mutex will be blocked. Any threads which `join()` on the blocked threads will also be blocked. The likely outcome is that the entire program will be blocked.

The problem is that `std::mutex`'s destructor does not unlock the mutex.

An obvious solution is to call `unlock()` in the handler for the exception.

Drawbacks of `std::mutex`

Whenever a thread calls `lock()` on a mutex, it must make a corresponding call to `unlock()`. It must never call `lock()` on a locked mutex, or `unlock()` on an unlocked mutex, as this causes serious problems.

The programmer must ensure that `unlock()` is always called after the critical section, to allow other threads to acquire the lock. This must happen in all cases, including code which has multiple execution paths (if statements, for example) and code which could throw an exception.

This places an unnecessary burden on programmers. In practice, we do not normally lock and unlock `std::mutex` directly.

Mutex Wrapper Class

A better alternative is to use a higher level object whose destructor unlocks the mutex. This will ensure that the mutex is automatically unlocked whenever it goes out of scope. C++ provides some classes which do this for us, by wrapping `std::mutex`.

These classes use the RAII idiom for managing resources. This is widely used in C++: for example, `std::fstream` for managing files, `std::unique_ptr` for managing heap-allocated memory, and so on.

In this case, the “resource” is a lock on the mutex.

- The constructor takes a mutex object as argument and locks it
- The destructor unlocks the mutex

If we create an object of one of these classes on the stack, the mutex will always be unlocked when the object goes out of scope, even if an exception is thrown.

std::lock_guard

The first mutex wrapper class we will look at is std::lock_guard. This is defined in the <mutex> header.

std::lock_guard is a very basic wrapper, which has only two member functions: a constructor to lock the mutex, and a destructor to unlock the mutex.

It is a class template, with the mutex type as the template parameter.

```
std::lock_guard<std::mutex> lk(print_mutex);
```

In C++17, the compiler can deduce the template parameter from the constructor argument, so we can simply write this as

```
std::lock_guard lk(print_mutex);
```

The code fragment will now look like this:

```
{  
  
    // Create the lock_guard object  
  
    // This will lock the mutex  
  
    std::lock_guard<std::mutex> lk(print_mutex);  
  
  
    // Execute the critical section  
  
    // ...  
  
} // Destroy the lock_guard object
```

We have put the code for the critical section in its own scope (a pair of braces). When the thread reaches the closing brace, it will call the destructor for std::lock_guard, which unlocks the mutex. If an exception is thrown before the closing brace, the thread will also call the destructor.

The mutex is always unlocked, without any extra work by the programmer.

Instead of writing explicit `lock()` and `unlock()` calls, we just create an `std::lock_guard` object and leave the C++ implementation to do the rest.

`std::scoped_lock`

C++17 provides `std::scoped_lock`. This works exactly the same as `std::lock_guard`, except the constructor can lock more than one mutex at a time.

```
// Lock std::mutex object m1, then m2
```

```
std::scoped_lock<std::mutex> lk(m1, m2);
```

In the `std::scoped_lock` constructor, the mutex arguments are locked, in the order they are passed.

In the destructor, they are unlocked in the reverse order. As we will see later, getting these orderings wrong can cause serious problems.

`std::lock_guard` Example

Here is the data race example again, this time using `std::lock_guard` to prevent the data race. We will just show the entry point function, since the rest of the program is the same as before.

```
auto print = [](const std::string& str)
```

```
{
```

```
    for (int i = 0; i < 5; ++i) {
```

```
        // Create the lock_guard object
```

```
        // This will lock the mutex
```

```
        std::lock_guard<std::mutex> lk(print_mutex);
```

```
// The critical region, which modifies std::cout

std::cout << str[0] << str[1] << str[2] << "\n";

std::this_thread::sleep_for(50ms);

} // Destroy the lock_guard object

};
```

However, there is a drawback to this code. After the critical region, the thread does some other work, which does not need to be protected. (As usual, this is represented by the thread sleeping.) The mutex is not unlocked until `std::lock_guard`'s destructor is called, at the end of the loop.

This means that other threads will not be able to enter the critical section, even though this thread has left it. The program will take longer to run than necessary.

In the next chapter, we will look at another mutex wrapper which has a solution for this problem.

Chapter Thirteen

Unique Lock

A More Flexible Mutex Wrapper

std::unique_lock

std::unique_lock is similar to std::lock_guard

- The constructor locks the mutex
- The destructor unlocks the mutex

However, it also has an unlock() member function. A thread can call unlock() after it leaves the critical section; this avoids blocking other threads while this thread executes other code.

If the function returns, or an exception is thrown, before the thread calls unlock(), the destructor will unlock the mutex.

When we use std::unique_lock, the mutex is always unlocked automatically, but we can avoid blocking other threads after this thread leaves the critical section.

std::unique_lock Example

Here is the data race example again, this time using std::unique_lock to prevent the data race. Again, the rest of the program is the same as before.

```
auto print = [](const std::string& str)
{
    for (int i = 0; i < 5; ++i) {
        // Create the lock_guard object
```



```

// This will lock the mutex

std::unique_lock<std::mutex> lk(print_mutex);

// The critical region, which modifies std::cout

std::cout << str[0] << str[1] << str[2] << "\n";

// Unlock the mutex

lk.unlock();

std::this_thread::sleep_for(50ms);

} // Destroy the unique_lock object

};

```

If the mutex is still locked when the `std::unique_lock` object goes out of scope, the object's destructor will unlock it. Otherwise, the mutex is left unlocked.

`std::unique_lock` Constructor

`std::unique_lock` is more flexible than `std::lock_guard`. The constructor has a number of options for locking the mutex:

- Wait until the mutex is locked. This is the default.
- Try to get the lock. Do not wait.
- Do not lock the mutex. This allows it to be locked later.

There is also a move constructor, which initializes the object with the mutex from another `std::unique_lock` object.

std::unique_lock Constructor Arguments

std::unique_lock's constructor has an optional second argument.

std::defer_lock does not lock the mutex immediately. The mutex can be locked later, by calling std::unique_lock's lock() member function, or by passing the std::unique_lock object to the std::lock() function.

std::adopt_lock takes ownership of a mutex that has already been locked by this thread. This avoids deadlock caused by locking the same mutex twice.

std::try_to_lock calls the mutex's try_lock() member function instead of lock(). It returns immediately if the mutex cannot be locked. The thread can call the owns_lock() member function later, to check whether the mutex is locked.

std::unique_lock and Move Semantics

Unlike std::lock_guard and std::scoped_lock, std::unique_lock supports move semantics. This allows a locked mutex to be transferred to another std::unique_lock object in the same thread.

Locking Recommendations

std::unique_lock is more flexible than std::lock_guard and std::scoped_lock. However, it uses slightly more memory, and is slightly slower.

- Use std::lock_guard to lock a single mutex until the end of the scope
- Use std::scoped_lock to lock multiple mutexes until the end of the scope
- Use std::unique_lock if the thread needs to unlock the mutex explicitly
- Use std::unique_lock if you need the extra flexibility

Chapter Fourteen

Recursive and Timed Mutexes

Some More Mutex Types

std::recursive_mutex

Locking an std::mutex which is already locked causes undefined behaviour. Typically, the entire program becomes deadlocked.

This can be avoided by using std::recursive_mutex instead of std::mutex.

std::recursive_lock's lock() member function can be called repeatedly. However, for each lock() call, there must be a matching unlock() call at some point.

std::recursive_lock can be used with std::lock_guard, std::unique_lock and std::scoped_lock. For example:

```
std::unique_lock<std::recursive_mutex> lk(mut);
```

Usually, needing multiple locks on the same mutex is a sign of a design error. Before using std::recursive_mutex, look at the code and see if you can find a better way to write it.

std::timed_mutex

std::timed_mutex has lock(), try_lock() and unlock() member functions. These work the same as for std::mutex.

It also has two extra member functions:

try_lock_for() waits for a given duration

try_lock_until() waits until a specific time point.

These return true or false, depending on whether the mutex could be locked.

std::timed_mutex Example

Here is our two-thread example again, this time written to use std::timed_mutex and the try_lock_for() member function:

```
#include <iostream>
```

```
#include <thread>
```

```
#include <mutex>
```

```
#include <chrono>
```

```
using namespace std::literals;
```

```
// Use std::timed_mutex instead of std::mutex
```

```
std::timed_mutex the_mutex;
```

```
void task1()
```

```
{
```

```
    std::cout << "Task1 trying to get lock\n";
```

```
    the_mutex.lock();
```

```
    std::cout << "Task1 has lock\n";
```

```
    std::this_thread::sleep_for(500ms);
```

```
    std::cout << "Task1 releasing lock\n";
```

```

    the_mutex.unlock();
}

void task2()
{
    std::this_thread::sleep_for(100ms);
    std::cout << "Task2 trying to get lock\n";

    // Instead of calling try_lock() and sleeping,
    // call try_lock_for() with a timeout
    while (!the_mutex.try_lock_for(100ms)) {
        std::cout << "Task2 could not get lock\n";
    }

    std::cout << "Task2 has lock\n";
    the_mutex.unlock();
}

int main()
{
    std::thread t1{task1}, t2{task2};

```

```
t1.join();  
t2.join();  
}
```

If the thread locks the mutex, `try_lock_for()` returns true. If `try_lock_for()` returns false, the thread was not able to lock the mutex within 100 milliseconds, and we call it again.

We can use similar logic with `try_lock_until()`. If the thread was not able to lock the mutex before the given timepoint, we call it again. The current deadline has already passed. If we use the same argument again, the call will return immediately. We need to advance the argument into the future.

The code in `task2()` would look like this (the rest of the program is the same):

```
// Find a timepoint 100ms in the future
```

```
auto deadline = std::chrono::steady_clock::now() + 100ms;
```

```
// Keep trying to lock until the deadline is reached
```

```
while (!the_mutex.try_lock_until(deadline)) {
```

```
    std::cout << "Task2 could not get lock\n";
```

```
// Advance the deadline by 100ms
```

```
    deadline += 100ms;
```

```
}
```

std::unique_lock

`std::unique_lock`'s constructor has some overloads which are available only with

a timed mutex:

- Take a `std::chrono::duration` as the second argument and wait for that duration
- Take a `std::chrono::time_point` as the second argument and wait until that time.

`std::unique_lock` also has `try_lock()` and `try_lock_for()` member functions. As with the constructor overloads, these call the corresponding member function on the timed mutex.

Modifying the last example to use `std::unique_lock` gives:

```
void task1()
{
    std::cout << "Task1 trying to get lock\n";
    std::unique_lock<std::timed_mutex> lk(the_mutex);
    std::cout << "Task1 has lock\n";
    std::this_thread::sleep_for(500ms);
    std::cout << "Task1 releasing lock\n";
}
```

```
void task2()
{
    std::this_thread::sleep_for(100ms);
    std::cout << "Task2 trying to get lock\n";
```

```
// Find a timepoint 100ms in the future
```

```

auto deadline = std::chrono::steady_clock::now() + 100ms;

std::unique_lock<std::timed_mutex> lk(the_mutex, deadline);

// Keep trying to lock until the deadline is reached

while (!lk.owns_lock()) {

    std::cout << "Task2 could not get lock\n";

    // Advance the deadline by 100ms

    deadline += 100ms;

    lk.try_lock_until(deadline);

}

std::cout << "Task2 has lock\n";

the_mutex.unlock();

}

```

Chrono Library Clocks

The `std::chrono` library provides several “clocks” which can provide durations and time points. The two main ones are `std::chrono::system_clock` and `std::chrono::steady_clock`.

The system clock gets its time directly from the operating system. This represents the time as seen by users of the system. It should be used for time

points.

The steady clock is an idealized clock which increases one tick at a time, and only goes forwards. It is not affected by system clock adjustments, daylight saving changes and so on. It should be used for measuring time intervals.

Note that `try_lock_for()` and `try_lock_until()` are subject to scheduling issues. They can only return when the thread that calls them is running. If the thread is suspended when the duration ends or the time point is reached, the call will return later than requested.

In effect, the duration and time point arguments represent a minimum value. They should not be regarded as hard constraints.

`std::try_lock()`

The free function `std::try_lock()` takes multiple mutexes as argument and tries to lock them all, in the order in which they are passed. It does this by calling the `try_lock()` member function of each mutex.

If any call fails, `std::try_lock()` returns immediately, without attempting any more locks. The return value will be the index of the argument that could not be locked (0 for the first argument.)

If all the mutexes were successfully locked, it returns -1.

Chapter Fifteen

Shared Mutexes

Multiple Accesses Without Data Race

Single Writer Thread with Multiple Reading Threads

With the mutexes we have covered so far, we can arrange things so that only one thread can execute in a critical section at any one time. This is very useful, as it prevents data races, but sometimes it can be too restrictive.

Imagine a situation where data is frequently accessed for reading, but only modified very rarely. If each thread which reads the data locks a mutex before accessing it, then all other threads which wish to read the data will be unable to run until this thread releases the lock. This is necessary; if there is the possibility that the data is modified, we need to prevent a data race.

Suppose the data is only modified in 1 out of every 1000 accesses. This means that, for every access where the mutex needs to be locked, there are 999 accesses where the mutex does not need to be locked and threads are blocked unnecessarily. Clearly this will have a serious impact on performance, but we cannot avoid locking.

std::shared_timed_mutex

C++14 introduced `std::shared_timed_mutex` and `std::shared_lock` to address this. These are defined in the `<shared_mutex>` header.

An `std::shared_timed_mutex` can be locked in two different ways.

If it is locked by an `std::lock_guard`, `std::unique_lock` or `std::scoped_lock` object, the result is the same as with all the other mutexes we have seen. When a thread locks the mutex, it has exclusive access to the critical section. If another thread wishes to enter the critical section, it must wait until this thread unlocks the mutex.

However, if `std::shared_timed_mutex` is locked by an `std::shared_lock` object, the result is different. The thread which locks it has shared access to the critical section; other threads can also access it, without waiting for the mutex to be unlocked.

We can use this for our single writer, multiple reader scenario. The writing thread will have an exclusive lock on the mutex. While it holds this mutex, no other thread can enter the critical region; only one thread can access it while it is being modified. Reader threads will have a shared lock, which allows other reader threads to access the critical region.

If a reader thread tries to get a shared lock when the writer thread has an exclusive lock, it will have to wait for the writer to release its lock before it can proceed.

If the writer thread tries to get its exclusive lock, and there are reader threads which have a shared lock, the writer thread must wait until all the reader threads release their shared locks.

This ensures that only one thread can execute in the critical section when there is modification. Multiple threads can execute in the critical section, but not when there is modification occurring. This prevents a data race, because a data race requires both multiple threads and modification.

This also works if there is more than one writing thread. Only one writing thread can have an exclusive lock, so different writing threads cannot interfere with each other. As before, reading threads cannot interfere with a writing thread which has an exclusive lock on the mutex.

`std::shared_mutex`

`std::shared_timed_mutex` has member functions which support timeouts, similar to `std::timed_mutex`.

C++17 added `std::shared_mutex`. This is exactly the same as `std::shared_timed_mutex`, but without the support for timeouts. This makes it simpler to implement. On some platforms, such as Windows, it is faster.

std::shared_timed_mutex Example

```
#include <iostream>
```

```
#include <thread>
```

```
#include <mutex>
```

```
#include <shared_mutex>
```

```
#include <chrono>
```

```
#include <vector>
```

```
using namespace std::literals;
```

```
// The shared mutex object
```

```
std::shared_timed_mutex shmut;
```

```
// Mutex to protect std::cout
```

```
std::mutex print_mutex;
```

```
void write(int i)
```

```
{
```

```
{
```

```
    // Use print_mutex to protect std::cout
```

```
std::lock_guard<std::mutex> pg(print_mutex);

std::cout << "Writer thread " << i << " trying to lock\n";

}
```

// Get an exclusive lock on the shared mutex

```
std::lock_guard<std::shared_timed_mutex> lg(shmut);
```

```
{
```

// Use print_mutex to protect std::cout

```
std::lock_guard<std::mutex> pg(print_mutex);
```

```
std::cout << "Writer thread " << i << " has exclusive lock\n";
```

```
}
```

```
std::this_thread::sleep_for(2s);
```

```
}
```

```
void read(int i)
```

```
{
```

```
{
```

// Use print_mutex to protect std::cout

```
std::lock_guard<std::mutex> pg(print_mutex);
```

```
std::cout << "Reader thread " << i << " trying to lock\n";
```

```
}
```

```
// Get a shared lock on the shared mutex
```

```
std::shared_lock<std::shared_timed_mutex> sl(shmut);
```

```
{
```

```
    // Use print_mutex to protect std::cout
```

```
    std::lock_guard<std::mutex> pg(print_mutex);
```

```
    std::cout << "Reader thread " << i << " has shared lock\n";
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
    std::vector<std::thread> threads;
```

```
    for (int i = 0; i < 5; ++i)
```

```
        threads.push_back(std::thread{read, i});
```

```
    threads.push_back(std::thread{write, 5});
```

```
    threads.push_back(std::thread{write, 6});
```

```

for (int i = 7; i < 15; ++i)

    threads.push_back(std::thread{read, i});

for (auto& thr : threads)

    thr.join();

}

```

Five reader threads try to lock the mutex with a shared lock. They will succeed immediately, unless one of the writer threads already has an exclusive lock. In this case, they must wait for the writer thread to release its exclusive lock.

Two writer threads try to lock the mutex with an exclusive lock. If any of the reader threads have a shared lock, the writer threads must wait for all the shared locks to be released. Only one writer thread can have an exclusive lock at a time.

Seven reader threads then try to lock the mutex with a shared lock. They will succeed immediately, unless one of the writer threads already has an exclusive lock. In this case, they must wait for the writer thread to release its exclusive lock.

To simplify handling multiple threads, we use an `std::vector`. The `std::thread` objects are moved into the vector. Each vector element will have an attached system execution thread. The `main()` function iterates over each element and calls `join()`. We use a reference in the the loop, because the `std::thread` objects cannot be moved.

We use a second mutex to control access to `std::cout`. This prevents the output from the different threads from being interleaved. A simple `std::mutex` is good enough for this purpose. By creating an `std::lock_guard` object in its own scope, the output mutex will automatically be unlocked before the thread tries to lock the shared mutex.

Shared Mutex and “Happens Before” Ordering

The combination of shared locks and exclusive locks avoids a data race:

- The writer thread cannot get an exclusive lock until all the reader threads currently accessing the critical section release their shared locks
- These reader threads' accesses happen before the writer thread's access
- Subsequent attempts by reader threads to enter the critical section cannot happen until the writer thread releases its exclusive lock
- Subsequent attempts by other writer threads to enter the critical section cannot happen until this writer thread releases its exclusive lock
- This writer thread's access happens before the other reader threads and writer threads
- There is no case in which a reader thread can have interleaved access with a writer thread
- There is no case in which a writer thread can have interleaved access with another writer thread
- A reader thread can have interleaved access with another reader thread. This is not a problem, because neither thread modifies data. There is no conflict between them.

Chapter Sixteen

Shared Data Initialization

How to Create Shared Data

Shared Data

Shared data can have several forms:

- Global variable or static variable at namespace scope. This could potentially be shared by all the threads in the program.
- Class member which is declared static. This could potentially be shared by all threads which have a member function of this class as their entry point.
- Local variable (in a function) which is declared static. This is shared by all threads which have the function as their entry point.

A global or static variable is initialized when the program starts up. At that point, there is only one thread running; there cannot be a data race, because that requires multiple running threads.

Static Local Variable Initialization

A static local variable is initialized when the program flow passes through its declaration for the first time.

Consider this thread entry point function:

```
void func()
```

```
{
```

```
...
```

```
// str will be initialized by the first thread

// which executes this statement

static std::string str{"xyz"};

...

}
```

Is there a data race here?

Before C++11, a compiler would have implemented the statement as if it had been written like this:

```
static bool str_initialized = false;

static std::string str;

if (!str_initialized)

    str_initialized = true;

    str = "xyz";

}
```

There is clearly potential for a data race, as both `str_initialized` and `str` are modified without synchronization.

In fact, C++11 now specifies that the initialization can only happen on one thread. Any other thread that reaches the statement will be blocked until the first thread has finished initializing the variable. The initialization "happens before" any other threads access the variable, so there is no data race.

However, this only applies when the variable is being initialized. Subsequent accesses to the variable are no different from non-static variables: there will be a data race, unless we provide some form of synchronization.

The Singleton Pattern

In the Singleton Design Pattern, there can only be a single global object of a class. This is intended for situations where it would not make sense to allow duplicate objects; for example, a logger class that maintains an audit trail. It can also be useful for objects which take a long time to create, such as database connections.

Here is the classic implementation of a singleton class, exactly as given in the "Gang of Four" book². (Nowadays, we use nullptr instead of the "magic number" 0.)

```
#ifndef SINGLETON_H
```

```
#define SINGLETON_H
```

```
#include <iostream>
```

```
class Singleton {
```

```
    // Private static member
```

```
    static Singleton *single;
```

```
public:
```

```
    // Static member function which returns a Singleton object
```

```
    // This is the only way to get a Singleton
```

```
    static Singleton* get_singleton()
```

```
{
```

```
    // If the member object does not already exist
```

```
if (single == 0) {  
    // Create it  
    single = new Singleton;  
}  
return single;  
}
```

protected:

```
// The constructor is protected and  
// can only be called by a static member function  
Singleton() {std::cout << "Initializing singleton\n";}   
};
```

```
#endif //SINGLETON_H
```

The only way to obtain a Singleton object is to call `get_singleton()`. This function ensures that there is only one object in the program:

- If the Singleton object does not already exist, it is created and initialized
- Otherwise, the existing Singleton object is returned

If we start some threads which create a Singleton, we might expect that each thread gets the same singleton object. The Singleton constructor will only be called once, and each thread's Singleton object will have the same address.

```
#include "singleton.h"
```

```
#include <thread>

#include <vector>

// Define the static member variable

Singleton *Singleton::single = 0;

// Thread entry point function

void task()

{

    // Get the Singleton object

    Singleton* s = Singleton::get_singleton();

    // Display its address

    std::cout << s << std::endl;

}

int main() {

    std::vector<std::thread> threads;

    for (int i = 0; i < 10; ++i)

        threads.push_back(std::thread{task});

    for (auto& t : threads)
```

```
t.join();  
}
```

In fact, we get output like this:

```
Initializing singletonInitializing singleton
```

```
Initializing singleton0x7f9368504080
```

```
Initializing singleton0x7f9368504080
```

```
0x7f9368704090
```

```
Initializing singleton
```

```
0x7f9368704080
```

The output is interleaved, because of a data race on `std::cout`, but that is not the biggest problem. The Singleton is initialized four times, each with a different address in memory.

The static object was initialized here, when the program started up:

```
// Define the static member variable
```

```
Singleton *Singleton::single = 0;
```

The data race is in the code below. The static object is already initialized, so the normal rules apply. We have multiple threads accessing the variable, with modification occurring, which are not synchronized.

```
if (single == 0) {  
    single = new Singleton;  
}
```

For example, a thread could be interrupted between the if statement and the initialization. If another thread runs now, it will see that single is still null and initializes it. The first thread then resumes and does the initialization again! single is initialized twice.

Meyers Singleton

With C++11's guarantees about static local initialization, we can write a thread-safe singleton implementation very concisely:

```
class Singleton {  
    ...    // Provide the class's functionality  
};  
  
Singleton& get_singleton()  
{  
    static Singleton single;  
    return single;  
}
```

C++11 guarantees that the first thread that calls this function will create the Singleton object. If another thread calls get_singleton(), it will stop at the instruction that creates the Singleton object. It cannot proceed until the first thread has completed the initialization. At that point, the object has already been initialized. C++11 guarantees that the second thread will not create another object or try to initialize it again.

No other thread can initialize the Singleton or interfere with the initialization. When any thread subsequently calls the function, it will return the object which was created by the initial call.

This idiom was discovered by Scott Meyers, the well-known C++ author and

speaker.

Chapter Seventeen

Double-checked Locking

An Alternative Approach to Singleton

Singleton without Data Race

We saw in the previous chapter that the classic singleton implementation suffers from a data race.

```
static Singleton* get_singleton()
{
    // Conflicting accesses to singleton cause a data race

    if (single == nullptr) {
        single = new Singleton;
    }

    return single;
}
```

We learned that C++11 makes guarantees about static variable initialization. These give us an easy way to avoid the problem; the Meyers singleton uses a static variable instead of a pointer.

How were C++ programmers able to write thread-safe singletons before these guarantees were added to the language? The obvious way is to add a lock, to prevent multiple threads accessing single concurrently.

```
std::mutex mut;
```

```
static Singleton* get_singleton()
{
    // Protect access to single with a mutex

    std::lock_guard<std::mutex> lk(mut);

    if (single == nullptr) {
        // Create the object

        single = new Singleton;
    }

    return single;
}
```

This is safe, but not very efficient. The mutex is locked every time the function is called, even though it is only needed for the first call. Every subsequent call will take longer than necessary.

Double-checked Locking

We think about this and then we come up with a clever idea: we only lock the mutex when we initialize single. This only happens when single is nullptr. So we change that part of the code:

```
// First check if we need to initialize

if (single == nullptr) {
```

```
// Protect access to single with a mutex
```

```
std::lock_guard<std::mutex> lk(mut);
```

```
if (single == nullptr) {
```

```
    // Create the object
```

```
    single = new Singleton;
```

```
}
```

```
}
```

and now the mutex is only locked once.

This is known as "double-checked locking", because the check is done twice.

Imagine this thread is interrupted between the first check and locking the mutex, and a second thread initializes `single` after locking the mutex. The first thread cannot continue until the second thread unlocks the mutex. When the first thread locks the mutex and does the second check, it will see that `single` is not null, and it will not initialize it again.

If the first thread is interrupted after locking the mutex, the second thread cannot lock the mutex until the first thread has initialized `single` and unlocked the mutex. The second thread will see that `single` is not null, and it will not initialize it again.

This all sounds safe enough. Unfortunately, the code suffers from a subtle but very serious flaw.

```
single = new Singleton;
```

This statement involves three separate operations:

- Allocate memory on the heap to store the Singleton object
- Construct a Singleton object in the allocated memory

- Store the address of this object in single

It is possible for these operations to be performed in a different order. C++ compilers often reorder instructions to improve the performance of the code. Modern hardware can also execute instructions out of order, and perform other optimizations that have a similar effect.

For example, the program could perform the operations in this order when it runs:

- Allocate memory on the heap to store the Singleton object
- Store the address of this object in single
- Construct a Singleton object in the allocated memory

If a thread is interrupted between storing the address and constructing the object, another thread could do the first check. `single` is not null, so it does not lock the mutex and immediately returns. It may well use the object before the first thread has finished initializing it, which could be disastrous.

`std::call_once`

There are several ways to address this problem. C++11 provides `std::call_once()`, which ensures that

- The function is only called once
- The call is only done in one thread
- This thread cannot be interrupted until the function call completes

To use `std::call_once()`, we create a global object of type `std::once_flag`. We pass this object, together with the function we want to call, as argument to `std::call_once()`.

```
// Header for std::once_flag and std::call_once
```

```
#include <mutex>
```

```
// Global object to store synchronization data
```

```
std::once_flag single_flag;
```

```
static Singleton* get_singleton()
```

```
{
```

```
    std::call_once(single_flag, []() {
```

```
        // Create the object
```

```
        single = new Singleton;
```

```
    });
```

```
    return single;
```

```
}
```

This is completely thread-safe, and takes less time than using a mutex.

C++ Language Changes

We have already mentioned C++11's initialization guarantees, which give us a simple solution: the Meyers singleton idiom.

C++17 defined the order of operations within expressions. With these definitions, there is only one allowed order for the initialization.

- Allocate memory on the heap to store the Singleton object
- Store the address of this object in single

- Construct a Singleton object in the allocated memory

The alternative orders which cause a data race are no longer allowed. The compiler enforces this by generating special machine instructions; these temporarily prevent reordering at the hardware level.

Conclusion

We now know four ways to implement the thread-safe Singleton pattern:

- Use single-checked locking and a mutex with any version of C++
- Use `std::call_once()` with C++11 or later
- Use the original double-checked locking algorithm with C++17 or later
- Use a static variable with C++11 or later

We will see later in the course that atomic variables offer yet another alternative.

Chapter Eighteen

Thread-local Variables

Non-shared Data

Thread-local Variables

We can use the `thread_local` keyword to declare a variable as local to a thread.

Thread-local variables are quite similar to static variables. However, instead of having a single object which is shared between threads, each thread has its own object.

Thread-local variables can be declared at namespace scope, as data members of a class, or as local variables in a function.

When declared at namespace scope or as a class member, a `thread_local` variable will be constructed at or before the first time they are used in a translation unit. (A translation unit is a source code file in which all the included header files have been preprocessed.) This is important when creating dynamic libraries in Windows, because problems can occur when sharing variables across different translation units.

For local variables, a thread-local variable is initialized in the same way as static local variables.

Thread-local variables are destroyed when the thread completes execution, in the reverse order to which they were constructed. (However, there is no guarantee about the order in which thread-local objects are constructed.)

Thread-local variables are not subject to data races, because only one thread can access a given thread-local variable object.

Thread-local Variable Example

If we use a thread-local variable for a random number engine, each thread will have its own engine object. This ensures that each thread generates the same sequence of random numbers, which can be useful for testing.

```
// Random number engine declared as thread_local
```

```
thread_local mt19937 mt;
```

```
// Thread entry point function
```

```
void func()
```

```
{
```

```
    // Fit the random numbers into a uniform distribution in the range 0 to 1
```

```
    uniform_real_distribution<double> dist(0, 1);
```

```
    // Generate ten random numbers
```

```
    for (int i = 0; i < 10; ++i)
```

```
        std::cout << dist(mt) << ", ";
```

```
}
```


Chapter Nineteen

Deadlock

When Things Grind to a Halt

Deadlock

We have mentioned "deadlock" a few times, but what does it mean?

A thread is "deadlocked" when it cannot run at all. Sometimes a thread cannot run because it has been interrupted, but then it will wake up and run again. If a thread is deadlocked, it cannot make any progress, even when it should be able to run.

Deadlock often occurs when two or more threads are waiting for each other. Thread A is waiting for thread B to finish doing something or send it a result, but thread B cannot finish until it gets something from A. Thread B is not running, because it is waiting for thread A, and thread A cannot run because it is waiting for thread B.

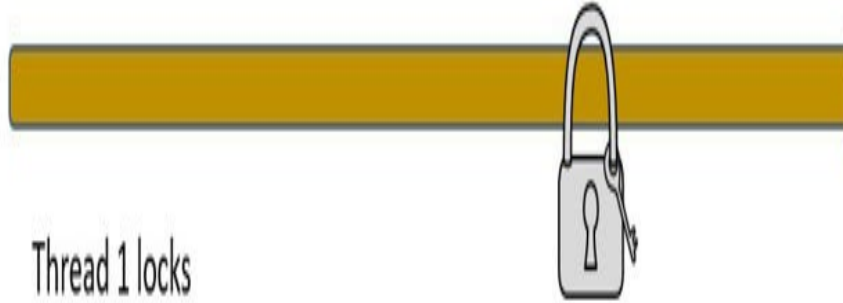
The classic example of this "mutual deadlock" involves waiting for mutexes to be unlocked, but there are other possibilities, such as

- Waiting for the result of a computation performed by another thread
- Waiting for a message sent by another thread
- Waiting for a GUI event sent by another thread

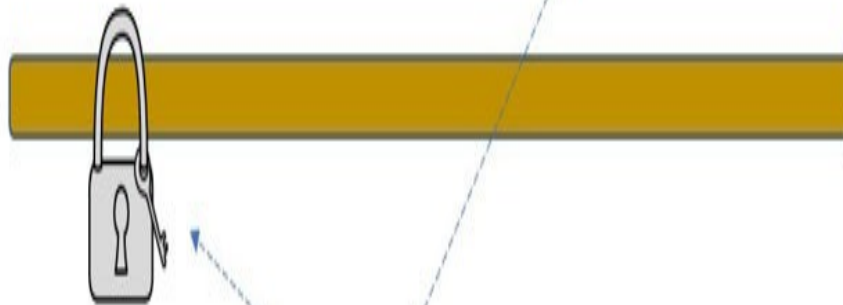
If the other thread is waiting for our thread to do something, then both threads will be deadlocked.

Deadlock Example

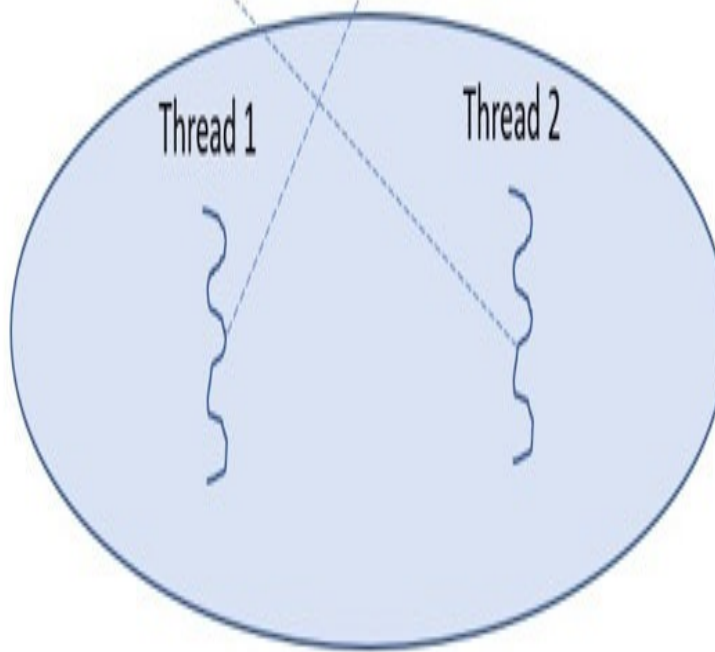
Thread 2
locks mutex2



Thread 1 locks
mutex1



Thread 1 waits for
lock on mutex2



Thread 2 waits for
lock on mutex1

Thread 1 waits for Thread 2 to unlock a mutex, while Thread 2 waits for Thread 1 to unlock a mutex

The code would look like this:

```
// Entry point for Thread 1
```

```
void func1()
```

```
{
```

```
    // Lock mutex1 first
```

```
    std::unique_lock<std::mutex> lka(mutex1);
```

```
    // Do some work here
```

```
    ...
```

```
    // Lock mutex2
```

```
    std::unique_lock<std::mutex> lkb(mutex2);
```

```
    // Do some more work here
```

```
    ...
```

```
}
```

```
// Entry point for Thread 2
```

```
// This locks the same mutexes,
```

```

// but in the opposite order

void func2()
{
    // Lock mutex2 first

    std::unique_lock<std::mutex> u1a(mutex2);

    ...    // Do some work here


    // Lock mutex1

    std::unique_lock<std::mutex> ulb(mutex1);

    ...    // Do some more work here
}

```

Thread 1 locks mutex1 and waits to lock mutex2. Thread 2 locks mutex2 and waits to lock mutex1. Once it gets this lock, it will be able to continue and it will unlock mutex2 when it completes. Thread 1 will then be able to lock mutex2 and continue. However, thread 1 has already locked mutex1, which means that thread 2 cannot lock it. Neither thread can run, because they are both waiting for the other thread to unlock a mutex that they need. And this cannot happen until the threads release the locks they already have.

Example Without Deadlock

If we change the example, so that both threads try to lock the mutexes in the same order, there is no deadlock.

```

// Entry point for Thread 1

```

```

void func1()

```

```
{  
  
    // Lock mutex1 first  
  
    std::unique_lock<std::mutex> lka(mutex1);  
  
    ...    // Do some work here  
  
    // Lock mutex2  
  
    std::unique_lock<std::mutex> lkb(mutex2);  
  
    ...    // Do some more work here  
  
}
```

```
// Entry point for Thread 2  
  
// This locks the same mutexes,  
  
// in the same order  
  
void func2()  
{  
  
    // Lock mutex1 first  
  
    std::unique_lock<std::mutex> u1a(mutex1);  
  
    // Do some work here  
  
    ...
```

```
// Lock mutex2

std::unique_lock<std::mutex> ulb(mutex2);

// Do some more work here

...

}
```

Both threads try to lock mutex1. Let's say that thread 2 gets the lock. Thread 1 will wait for the lock to become available. Meanwhile, thread 2 does some work, locks mutex2, completes its execution and unlocks both the mutexes. Thread 1 can now lock mutex1 and continue.

Deadlock Avoidance Guidelines

The basic rule for avoiding deadlock: never wait for a thread if there is a possibility it could wait for your thread.

Note that deadlock may involve more than two threads; for example, thread A waits for thread B, which waits for thread C, which waits for thread A.

Avoid nested locks if possible. If your thread already has a lock, do not acquire another lock, even if it is on a different mutex. If you do need multiple locks, acquire them in a single operation.

Acquire locks in a fixed order. If you do need multiple locks, and they cannot be acquired in a single operation, make sure that every thread acquires the locks in the same order.

When in a critical section, avoid calling code that you do not have control over:

- Library functions
- Function pointers, functors, callbacks, and so on

- Virtual functions
- Template functions, or member functions of a template class

...unless you know for certain that the code does not acquire locks. And even if it does, there is then the risk that different threads could acquire the locks in a different order.

Deadlock Avoidance

We saw that we could avoid deadlock in our example by writing the threads so that they acquire the locks in the same order. This relies on the programmer, and is not feasible in larger programs. A better option is to use language features to ensure that one thread locks all the mutexes it needs in a single operation.

Deadlock Avoidance in C++17

In C++17, we can use `std::scoped_lock`:

```
// Entry point for Thread 1
```

```
void func1()
```

```
{
```

```
    // Lock mutex1 and mutex2
```

```
    std::scoped_lock<std::mutex> lka(mutex1, mutex2);
```

```
    // Do some work here
```

```
    ...
```

```
    // Do some more work here
```

```

...
}

// Entry point for Thread 2
void func2()
{
    // Lock mutex2 and mutex 1
    std::scoped_lock<std::mutex> u1a(mutex2, mutex1);

    // Do some work here
    ...

    // Do some more work here
    ...
}

```

If thread 2 locks the mutexes first, thread 1 has to wait until thread 2 completes and releases both the mutexes. This has the same effect as locking the mutexes in the same order, even though the programmer actually wrote them in different orders.

Deadlock Avoidance in C++11

In C++11, we can call `std::lock()` to lock both the mutexes, then pass them to the constructor of `std::unique_lock`. To acquire an existing lock, we pass an extra argument, `std::adopt_lock`:


```

// Entry point for Thread 1
void func1()
{
    // Lock mutex1 and mutex2
    std::lock(mutex1, mutex2);

    // Associate each locked mutex with a unique_lock
    std::unique_lock<std::mutex> lka(mutex1, std::adopt_lock);
    std::unique_lock<std::mutex> lkb(mutex2, std::adopt_lock);

    // Do some work here
    ...

    // Do some more work here
    ...
}

```

and similarly for thread 2. Alternatively, we can pass the `std::defer_lock` option, then lock the mutexes later:

```

// Entry point for Thread 1
void func1()
{

```

```
// Associate each locked mutex with a unique_lock

std::unique_lock<std::mutex> lka(mutex1, std::defer_lock);

std::unique_lock<std::mutex> lkb(mutex2, std::defer_lock);


// Lock mutex1 and mutex2

std::lock(lka, lkb);


// Do some work here

...


// Do some more work here

...

}
```

Hierarchical Mutex

There are some cases where it is not possible to acquire multiple locks simultaneously, due to the nature of the problem. In that case, the programmer can impose an ordering, by associating each mutex with an identifier: a thread cannot lock a mutex unless it has already locked a mutex with a lower identifier.

The Williams book³ implements a `hierarchical_mutex` class which does this.

Non-locking Deadlock Avoidance Guidelines

In general, deadlock can arise whenever threads are waiting. Waiting is an important technique in multi-threaded programming. However, it should be done

carefully and in moderation, as it can lead to situations where threads are waiting for each other and are unable to proceed.

Chapter Twenty

Livelock

Going Nowhere Fast

Livelock

Livelock is another problem that can occur in multi-threaded programming. As with deadlock, threads are unable to make progress. However, in livelock, the threads are still active, as opposed to deadlock, where the thread cannot run.

Livelock is often caused by trying to avoid deadlock. Consider the example in the previous chapter, where two threads try to acquire locks in the same order. Suppose we try a different approach to avoid the deadlock. Instead of waiting indefinitely to lock the second mutex, each thread has a timeout in a loop.

```
#include <thread>
```

```
#include <mutex>
```

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std::literals;
```

```
// Use timed mutexes
```

```
std::timed_mutex mutex1, mutex2;
```

```
void func1()
{
    std::this_thread::sleep_for(10ms);

    bool locked{false};

    while (!locked) {
        // Lock mutex1

        std::unique_lock<std::timed_mutex> lk(mutex1);

        std::this_thread::sleep_for(1s);

        std::cout << "After you, Claude!\n";

        // Try to get a lock on mutex2

        locked = mutex2.try_lock_for(5ms);
    }

    if (locked)
        std::cout << "Thread1 has locked both mutexes";
}
```

```
void func2()
{
    bool locked{false};

    while (!locked) {
```

```

// Lock mutex2

std::unique_lock<std::timed_mutex> lk(mutex2);

std::this_thread::sleep_for(1s);

std::cout << "No, after you, Cecil!\n";


// Try to get a lock on mutex1

locked = mutex1.try_lock_for(5ms);
}

if (locked)

    std::cout << "Thread2 has locked both mutexes\n";

}


int main()

{

    std::thread t1(func1);

    std::this_thread::sleep_for(10ms);

    std::thread t2(func2);

    t1.join();

    t2.join();

}

```

Thread 1 locks mutex1, then it tries to lock mutex2. If it cannot lock it, it

releases its lock on mutex1, sleeps for 1 second and then tries again. Thread 2 has similar logic, but with the mutexes the other way round.

The output looks like this:

After you, Claude!

No, after you, Cecil!

After you, Claude!

No, after you, Cecil!

After you, Claude!

No, after you, Cecil!

After you, Claude!

No, after you, Cecil!

No, after you, Cecil!

After you, Claude!

This continues indefinitely.

("Cecil" and "Claude" were two characters in an ancient comedy program on BBC radio. Imagine two Civil Servants walking down a corridor together, then trying to go through a door which is only wide enough for one person. In typically British fashion, they both apologize, step back and wait for the other person to go through the door. Then they realize that the other person is waiting for them, and both try to go through the door at the same time again. Then they apologize, step back and wait again...)

The problem is that both threads wake up at the same time. This could be fixed by making the threads sleep for different lengths of time. A better solution is to use the deadlock avoidance techniques from the last chapter, in which one thread acquires all the locks in a single operation.

Thread Priority

Another way to avoid livelock is to assign different priorities to threads. This is not directly supported by C++, but it is available using the operating system's thread interface via `std::thread`'s `native_handle` member function.

When choosing which threads to run, the thread scheduler will prefer a thread with high priority over a thread which has lower priority. A high priority thread will run more often, while a low priority thread will sleep or be interrupted more often.

In effect, the thread scheduler makes the threads sleep for different intervals.

However, there are some issues which can arise when threads have different priorities.

Priority Inversion

Imagine a thread needs some data from another thread, and the thread which supplies the data has a lower priority. The high priority thread will stop and wait for the low priority thread. The thread which has a low priority will keep getting interrupted by other threads which have a higher priority but are not relevant to this activity.

The result is that the "high priority" thread runs at the speed of a low priority thread, hence the name.

Convoying

Suppose that we have a high priority thread and a low priority thread. Both threads need to lock the same mutexes. The low priority thread acquires a lock on mutex A first, forcing the high priority thread to wait until the low priority thread unlocks it. The low priority thread then locks mutex B first. Again, the high priority thread is forced to wait for the low priority thread.

The high priority thread is forced to run at the same speed as a low priority thread. This is similar to a sports car being stuck behind a slow-moving vehicle on a narrow road.

Resource Starvation

Deadlock and livelock are both examples of resource starvation. Resource starvation occurs when a thread cannot get the resources it needs to run. In deadlock and livelock, the resource is a lock that the thread cannot acquire.

There can be situations where threads are affected by the operating system suffering from resource starvation; for example, there is not enough memory to start a new thread, or the operating system has reached the maximum number of threads it can support.

Low priority threads may get starved of processor time. Because they have low priority, they do not run very often. Good schedulers will try to introduce some element of "fairness", so that low priority threads get to run occasionally. However, they may not be able to run fast enough to do their work within the required time.

Chapter Twenty-one

Mutex Conclusion

How to Use Mutexes Effectively

Locking Guidelines

Do not hold a lock on a mutex for any longer than is necessary. Otherwise, threads which are trying to execute the critical section will take longer to run.

For a thread which reads data, a good usage pattern is:

- Lock the mutex
- Copy the shared data into a local variable
- Unlock the mutex
- Process the data, using the local variable

For writing data:

- Do the work
- Lock the mutex
- Copy the result into the shared data
- Unlock the mutex

This gives other threads more opportunities to run, which speeds the program up.

Input/output operations are very slow and should be avoided in real programs. (They can be used in "toy" programs for experimenting, or by authors who are demonstrating things!)

Do not lock more data than is necessary. For example, if you are writing a linked list implementation, it is not good to have a single lock for the entire list. If a thread accesses the first element, for example, then any other thread which wishes to access an element in the list will be blocked, even if it would not conflict with the first thread.

Do not make locking too fine-grained. If the linked list has one lock per element, then the operating system may run out of mutex resources. There is also the possibility of different threads locking mutexes in different orders, resulting in deadlock.

The best solution is to have a lock which covers a node and its neighbours. In fact, writing a thread-safe linked list which is also correct and efficient is surprisingly difficult, and beyond the scope of this book.

Pros and Cons of Mutexes

Mutexes are a fairly straightforward way to prevent race conditions with shared data, provided the programmer takes a little care.

Locking and unlocking mutexes are usually slow operations, because they require a call into the operating system.

Mutexes are a rather low-level concept which place a lot of reliance on the programmer

- The programmer must remember to use a mutex when necessary
- The programmer has to use the right mutex in the right place
- The programmer must understand how different threads can modify the data

In large programs, or if there are several developers working on the code, this is not realistic. Most real-world programs use higher-level structures. We will start looking at these in the next chapter.

Some Remedies

Some of the problems with using mutexes can be mitigated by encapsulating the

mutex and the shared data in a class

- Make the shared data a private member of the class
- Make the mutex a private member of the class
- Make the thread entry point a public member function
- Provide a public "getter" member function which returns the shared data

Chapter Twenty-Two

Condition Variables

Waiting...

Communication between Workers

A common problem in many workplaces, including software development: you are ready to start work, but you cannot do anything yet, because someone else has to do something or give you something. Often, there is someone - a foreman, a supervisor or a team leader - who coordinates the different people.



Manager



Worker A



Worker B

Worker A needs an image that worker B is producing. The Manager will tell worker A when the image is ready.

- Worker A is working on a document
- The document needs an image that worker B is producing
- Worker A cannot continue with the document until worker B has produced the image
- Worker A tells the manager they are waiting
- Worker B is working on the image
- Worker B finishes the image
- Worker B tells the manager they have finished
- The manager tells worker A the image is ready
- Worker A resumes work on the document.

Communication between Threads

We often have similar situations between threads, where a thread has to wait for some event, or to get some result, before it can proceed.

For example, a program has a thread which fetches data over the network. There is another thread which displays a progress bar, showing how much data has been downloaded so far, and a thread to process the data.

When all the data has been received, the thread that is fetching the data will terminate. The thread with the progress bar will also terminate. The thread which processes the data will then run.

How can a programmer control all these threads?

One solution is to use a shared Boolean variable. The fetching thread will set this variable to true when it completes. The progress bar thread will check this

variable in a loop, and exit when it is true . The data processing thread will check this variable in a loop and start processing the data when it is true.

As we have multiple threads accessing this variable, with the possibility of modification, there is a data race. All the threads must use a mutex lock when they access it.

This will work, but it is not a very efficient solution. The loops to check the Boolean will consume a lot of processing power and will constantly lock and unlock the mutex, which slows the program down. We could improve this by making the loops sleep between checking the Boolean, but then we have to decide on the waiting time. If we make this too long, the program will run slowly; too short and it will use too much power.

A much better solution is to use a condition variable.

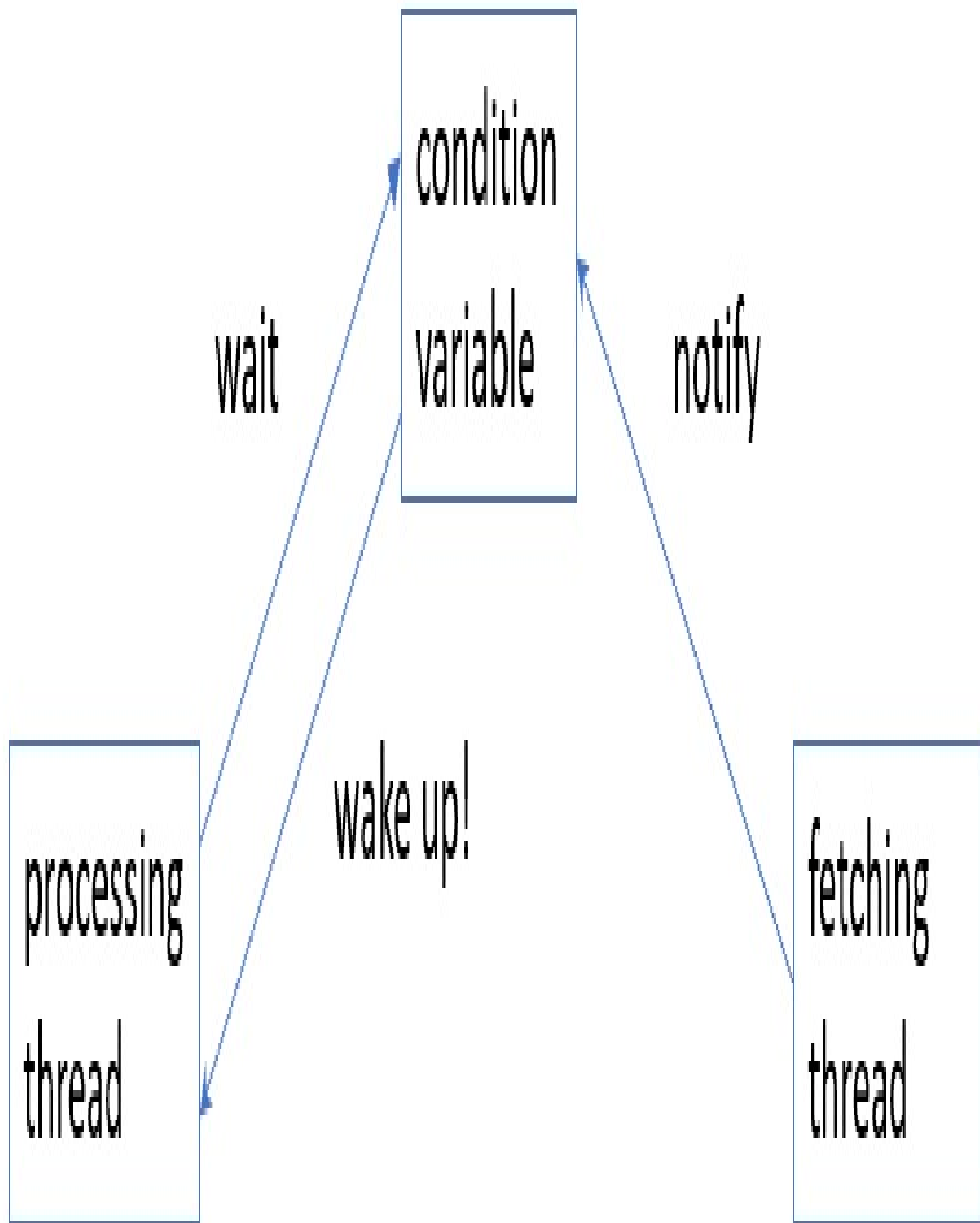
Condition Variable Overview

A condition variable can be used when we have two or more threads with critical sections which need to be protected by a mutex. The condition variable uses the mutex to coordinate the threads.

In our example, the processing thread would tell the condition variable that it is waiting, similar to worker A telling the manager that they need the image.

The fetching thread notifies the condition variable when it finishes, similar to worker B telling the manager they have finished the image.

The condition variable wakes up the processing thread, which can then run and do its work, similar to the manager telling worker A that the image is ready.



The condition variable will wake the processing thread up when all the data has been fetched

The progress bar thread also tells the condition variable that it is waiting, and receive a notification when the fetch is complete. The progress bar thread can then exit.

Condition Variable Scenario

The processing thread creates an `std::unique_lock` object, which locks the mutex. It then calls the condition variable's `wait()` member function. This call releases the lock and makes the processing thread sleep.

The fetching thread creates an `std::lock_guard` object to lock the mutex. When it has fetched the data, it unlocks the mutex and notifies the condition variable. The condition variable will then wake up the processing thread and lock the mutex again, so that the processing thread can safely access the shared data.

The condition variable ensures that the critical section in the processing thread is always protected when the thread is running. When the processing thread is not running, the mutex is unlocked, allowing other threads to lock the mutex and access the shared data.

`std::condition_variable`

`std::condition_variable` is defined in `<condition_variable>`.

Its `wait()` member function takes an argument of type `std::unique_lock<std::mutex>`. It unlocks the argument mutex, then blocks the thread until the thread receives a notification.

There are versions with timeouts: `wait_for()` and `wait_until()`. If the thread does not receive a notification within the given duration, or before the given timepoint, the function call will lock the mutex again.

`std::condition_variable` has two member functions for sending notifications to waiting threads. `notify_one()` will send a notification to a single waiting thread. The implementation decides which thread is woken up; the programmer has no

say in the matter. `notify_all()` sends a notification to all waiting threads.

In our example above, the processing thread and the progress bar thread would call `wait()` and the fetching thread would call `notify_all()`, to wake up both the waiting threads.

Condition Variable Example

We will have a slightly simpler example with two threads, "reader" and "writer". The writer thread will modify a shared `std::string` object; the reader object will wait until the string is modified, then wake up and display it.

We will use a condition variable to coordinate the two threads; we will also need a mutex, to protect the shared data, and for the condition variable to manage the threads.

```
#include <iostream>
```

```
#include <thread>
```

```
#include <condition_variable>
```

```
#include <string>
```

```
#include <chrono>
```

```
using namespace std::literals;
```

```
// Global variables
```

```
std::mutex mut;
```

```
std::condition_variable cv;
```

```
std::string sdata{"Empty"};
```

```
// Waiting thread
```

```
void reader()
```

```
{
```

```
// Acquire lock
```

```
    std::unique_lock<std::mutex> guard(mut);
```

```
// Unlock the mutex and
```

```
// wait for condition variable to be notified
```

```
    cv.wait(guard);
```

```
// Wake up and use the new value
```

```
// The mutex is now locked again
```

```
    std::cout << "Data is " << sdata << "\n";
```

```
}
```

```
// Modifying thread
```

```
void writer()
```

```
{
```

```
    std::cout << "Writing data...\n";
```

```
{
```

```
    // Lock the mutex
```

```
std::lock_guard<std::mutex> lg(mut);

// Pretend to be busy...

std::this_thread::sleep_for(2s);

// Modify the data

sdata = "Populated";
}

// Notify the condition variable

// The mutex is now unlocked

cv.notify_one();
}

int main()
{
    std::cout << "Data is " << sdata << "\n";

    std::thread read{reader};

    std::thread write{writer};

    write.join();

    read.join();
}
```

```
}
```

We define the `std::condition_variable` as a global variable, along with the `std::mutex` which will protect the shared data, and the shared `std::string` object.

In the reader thread, which waits for the data to be modified, we create an `std::unique_lock` object which locks the mutex, then we pass this lock object as the argument to `wait()`. This call will cause the thread to sleep, after it unlocks the mutex.

In the modifying thread, we lock the mutex ourselves. There is no requirement to use `std::unique_lock` here, so we use the more efficient `std::lock_guard` instead. We create it inside an inner scope, so its destructor will automatically unlock the mutex when the thread execution leaves the scope. We then call `notify_one()`. The condition variable will wake up the reader thread and lock the mutex again.

When the program is run, the output is

Data is Empty

Writing data...

Data is Populated

The shared data is always protected by a mutex, so there is no data race. In `write()`, the shared data is modified while it is protected by the `std::lock_guard`. In `read()`, when the call to `wait()` returns, it will re-lock the mutex before the thread reads the shared data.

The condition variable uses the mutex to coordinate the threads. In `read()`, the call to `wait()` unlocks the mutex, allowing the `write()` thread to lock the mutex and modify the shared data. `write()` unlocks the mutex before notifying the condition variable. In `read()`, the condition variable locks the mutex before accessing the shared data.

Chapter Twenty-Three

Condition Variables and Predicates

Lost Wakeups

Lost Wakeup

The example code in the last chapter has a serious problem. To see this, change the `main()` function so that the writer thread starts first:

```
std::thread write{writer};  
  
std::this_thread::sleep_for(500ms);  
  
std::thread read{reader};
```

The sleep is to make sure that the writer thread completes before the reader thread starts.

The program hangs after displaying "Writing Data...". The "Data is Populated" output never appears.

The problem is that the reader thread is waiting for the condition variable to receive a notification, but the notification has already been sent. The condition variable does not queue notifications; if there is no thread which is waiting for the notification, it is lost.

As an analogy, if someone is waiting for you to call them on a landline telephone, and they are not there to answer it when you call them, the call is lost. It does not stay in the telephone system, waiting for someone to arrive and take the call!

The problem is that `wait()` will block until the notification is received. If the writing thread sends the notification before the reading thread calls `wait()`, the call will never return.

This situation is known as a "lost wakeup". It is an example of a race condition, in which the behaviour of a program changes when threads are scheduled to run in a different order.

Spurious Wakeup

In fact there is another, very obscure, problem with the code. It is possible for the `wait()` call to return and wake up the thread, even though the writing thread has not notified the condition variable. This is called a "spurious wakeup".

This is in fact the result of a race condition in the condition variable itself. The race condition is actually intentional, and not the result of bad design or a programming error in the C++ library. Implementing condition variables without the race condition would affect performance too much.

Continuing the landline analogy, this is similar to an intermittent line fault which occasionally makes the telephone ring when no-one is calling.

Fortunately, there is a solution which addresses both lost and spurious wakeups.

`wait()` with Predicate

`wait()` takes an optional second argument, which is a callable object: a predicate function. `wait()` will not return until this predicate function returns true.

Typically, this predicate function returns the value of a shared Boolean variable. The variable is initially set to false; when the waited-for event occurs, it is set to true.

This covers both cases. For lost wakeups, the writing thread will set the flag to true before it completes. When the reading thread starts, the predicate will return true and the `wait()` call will return.

For spurious wakeups, the flag will be false, since the data has not been written and the flag has not been updated. `wait()` will not return until the writing thread modifies the string and sets the flag.

Example with Predicate

To use a predicate in our example, first we add a shared Boolean flag.

```
// Flag for predicate
```

```
bool condition{false};
```

In the wait() call, we provide a callable object which returns the value of this flag. A lambda expression is ideal for this.

```
// Waiting thread
```

```
void reader()
```

```
{
```

```
    // Acquire lock
```

```
    std::unique_lock<std::mutex> guard(mut);
```

```
    // Unlock the mutex and
```

```
    // wait for condition variable to be notified
```

```
    cv.wait(guard, []() { return condition; });
```

```
    // Wake up and use the new value
```

```
    // The mutex is now locked again
```

```
    std::cout << "Data is " << sdata << "\n";
```

```
}
```

In the writing thread, we set the flag to true before unlocking the mutex and calling notify.

```
// Modifying thread
```

```
void writer()
{
    std::cout << "Writing data...\n";
    {
        // Acquire lock
        std::lock_guard<std::mutex> lg(mut);

        // Pretend to be busy...
        std::this_thread::sleep_for(2s);

        // Modify the data
        sdata = "Populated";
        condition = true;
    }

    // Notify the condition variable
    // The mutex is now unlocked
    cv.notify_one();
}
```

Lost Wakeup Avoidance

When `wait()` is called, it calls the predicate. At this point, the mutex is still locked.

If the predicate returns true, it means the writing thread has called `notify`. The reading thread will resume execution, with the mutex locked, even if the notification occurred before this thread started.

If the predicate returns false, it means the writing thread has not called `notify`. The reading thread will unlock the mutex and wait for a notification.

This is analogous to a landline with an answering machine. If the person arrives to take the call and finds it has already been made, they can listen to the call on the answering machine.

Spurious Wakeup Avoidance

The predicate only returns true if the writing thread actually called `notify`. Otherwise, it was a spurious wakeup, and the reading thread waits for a genuine notification.

Continuing the landline analogy, this is analogous to a "silent" message on the answering machine. The person taking the call will wait until a genuine call is made.

As we are using a shared variable which gets overwritten on each notification, the waiting thread will only see the result of the most recent notification. This is analogous to an answering machine which can only store one message at a time; each new message is recorded over the previous message. To get the full "answering machine" functionality, we need to provide our own queueing facility. We will cover this in a later chapter.

Condition Variables and Multiple Threads

Condition variables are particularly useful when several threads are waiting for the same event. There are two options for notifying the threads that the event has occurred:

`notify_all()`. All the threads which are waiting on this condition variable resume

execution, with the mutex locked. The threads can wake up in any order, depending on when they lock the mutex.

`notify_one()`. Only one waiting thread is woken up. The other threads will remain blocked, waiting for the next notification.

Once the thread has been woken up by `notify_one()`, it cannot be notified again by the same condition variable. This means that a different thread will respond to the event on each notification.

Chapter Twenty-Four

Futures and Promises

Sending Data between Threads

Returning Data from a Thread

`std::thread` does not provide any way to return a value from the entry point function. We have used shared variables to transfer data between threads. However, this raised the issue of data races, and we had to use mutex locks to protect the shared variables.

In the remainder of this book, we will move away from `std::thread` and start using higher-level objects. We have already seen `std::condition_variable`, which helps us coordinate different threads.

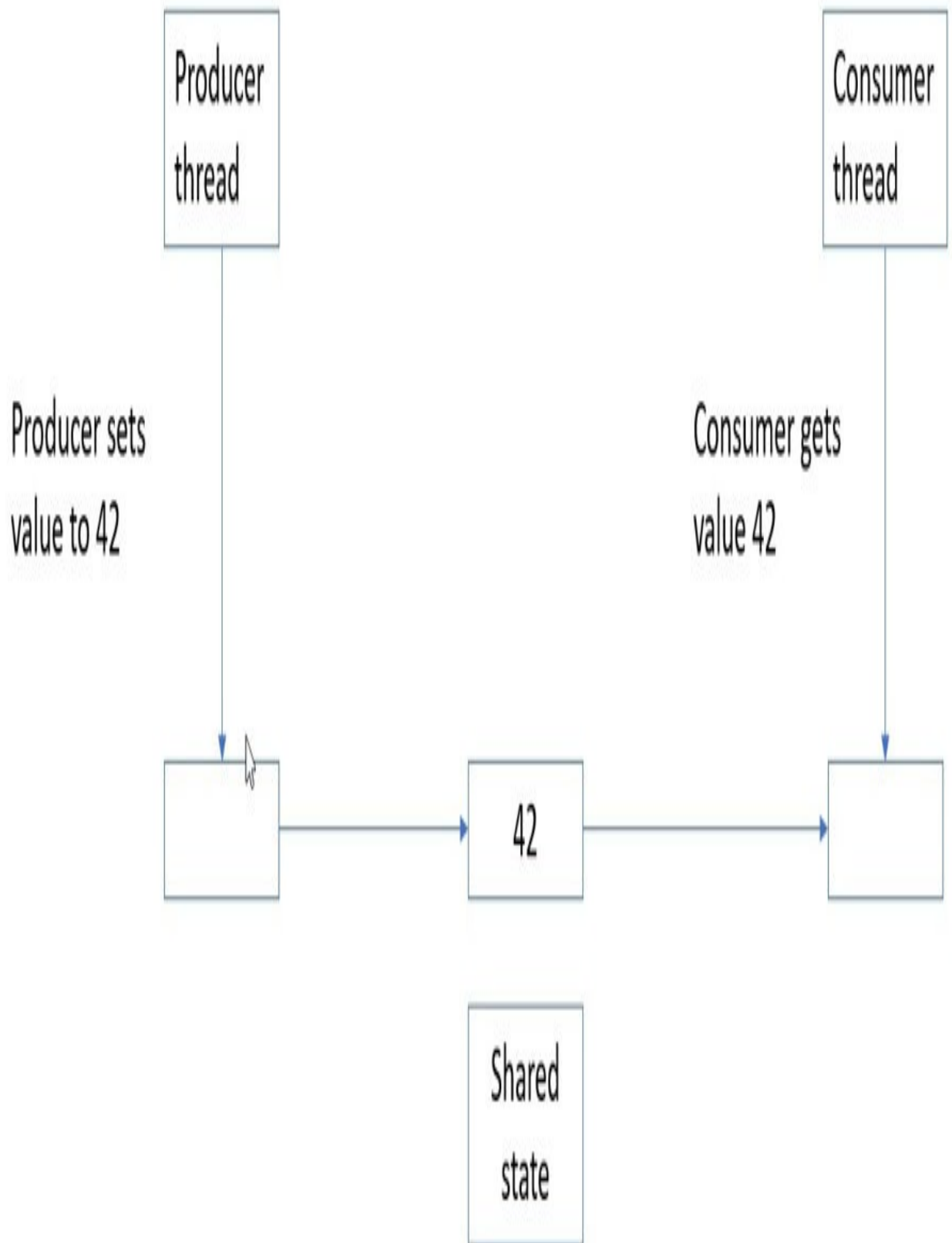
The C++ library provides "future" and "promise" objects. These can be used to transfer data between threads through a "shared state"; this avoids the need to add shared variables and mutexes to our code.

Producer-Consumer Model

In the last two chapters, we had programs with reader and writer threads. These are an example of the "producer-consumer" model. A "producer" thread generates some data or calculates a result. A "consumer" thread uses this result.

The producer thread has a promise object, where it stores its result. The promise object sets this value in the shared state. This is similar to the write thread calling `notify`.

The consumer thread has a future object, which waits for the value to become available in the shared state. The consumer thread then gets the value, via the future object. This is similar to the read thread calling `wait`.



The producer thread sends the value 42 to the consumer thread through their "shared state"

Future

In computing, a "future" represents a value which has not been calculated yet.

The future is one of the most important data structures in C++ concurrency. It is used with many different asynchronous objects and operations, as well as with a promise.

Normally, we do not create a future object directly. When we use a promise, we can obtain a future which is associated with that promise and can receive data through their shared state. With asynchronous operations, the return value is a future; when the operation completes, the future object will hold the result.

std::future

`std::future` is defined in `<future>`. It is a template class; the parameter is the type of the result that the future will receive.

To obtain the result, we call the `get()` member function. The call will block until the operation is complete. It will then fetch the result and return it.

`std::future` also has the `wait()` member function, which stops and waits for the result but does not return it. There are `wait_for()` and `wait_until()` versions which block with a timeout.

If we call `get()` and the operation has already completed, it will return the result immediately. This allows programmers to call one of the wait member functions to see whether the result is ready, then `get()` to retrieve it when it becomes available.

Using **std::future** with **std::promise**

When using an `std::future` object with a promise, we start by creating an `std::promise` object. We then call its `get_future()` member function:


```
std::promise<int> prom;
```

```
auto fut{ prom.get_future(); }
```

This creates an `std::promise` object which can store an int in the shared state. `get_future()` returns an `std::future` object which can read an int from the shared state. `std::future` is a move-only class, so `fut` will be move-initialized.

These two statements set up the shared state between the promise object and the future object. We can use this to send a result from the promise to the future in a thread-safe way.

Once we have created the promise and future objects, we can start up the producer and consumer threads. We pass a reference to the `std::promise` object as an argument to the producer thread's constructor. For the consumer thread, we pass a reference to the `std::future` object.

When the consumer thread is ready to receive the result, it calls the future's `get()` member function. The thread will stop and wait until the result is ready. The `get()` call will then return the result.

In some ways, this resembles a cross-thread function call: the consumer thread "calls" the producer thread, which returns a value to the consumer thread.

std::promise

`std::promise` is also a template class, where the parameter is the type of the result.

The `get_future()` member function returns the `std::future` object associated with this promise.

`set_value()` stores its argument in the shared state. There is also a `set_exception()` member function, which can be used with exceptions.

Example with std::future and std::promise

```
#include <future>
```

```
#include <iostream>
```

```
#include <thread>
```

```
#include <chrono>
```

```
using namespace std::literals;
```

```
// Producer function with std::promise argument
```

```
void produce(std::promise<int>& px)
```

```
{
```

```
    int x{42};
```

```
    std::this_thread::sleep_for(1s);
```

```
    std::cout << "The promise sets the shared state to " << x << "\n";
```

```
// Set the result
```

```
    px.set_value(x);
```

```
}
```

```
// Consumer function with std::future argument
```

```
void consume(std::future<int>& fx)
```

```
{
```

```
    std::cout << "The future calls get()..." << "\n";
```

```
// Get the result
```

```
int x = fx.get();
```

```
std::cout << "The future returns from get()\n";
```

```
std::cout << "The answer is " << x << "\n";
```

```
}
```

```
int main()
```

```
{
```

```
// Create the promise object
```

```
std::promise<int> p;
```

```
// Obtain the associated future
```

```
std::future<int> f = p.get_future();
```

```
// Start the consumer thread
```

```
// Pass the future to it
```

```
std::thread fut{consume, std::ref(f)};
```

```
// Start the producer thread
```

```
// Pass the promise to it
```

```
std::thread prom{produce, std::ref(p)};

fut.join();

prom.join();

}
```

The producer function takes the promise as argument. It calls `set_value()`, which stores the value of a local variable `x` in the shared state.

The consumer function takes the future as argument. It calls `get()` on the future. The thread stops and waits until the shared state is updated. The function call then returns, and the value from the shared state will be shared in the local variable `x`.

In the `main()` function, we create the `std::promise` object and get its associated `std::future` object. We start a consumer thread with the `std::future` object as argument and a producer thread with the `std::promise` object as argument.

When run, the program gives this output:

The future calls `get()`...

The promise sets the shared state to 42

The future returns from `get()`

The answer is 42

Exception Handling

As well as storing return values in the shared state, a promise can also store thrown exceptions, if the producer thread does not wish to handle the exception itself.

This is done by calling the `set_exception()` member function.

The exception will be thrown again in the caller thread, when it calls `get()` or one of the wait functions.

Going back to the function call analogy, this resembles calling a function which throws an exception, then catches it and rethrows the exception for the caller to handle.

Producer-Consumer with Exception Handling

In the producer, put a try block around any code that might throw an exception.

The saved exception object must be usable in other threads. This means we cannot work with a copy of the exception object in the catch handler, as we usually do. Instead, we call `std::current_exception()`. This returns a pointer which is guaranteed to point to a valid object.

We then call the `set_exception()` member function, to store the exception pointer in the shared state.

```
try {  
    // Code that may throw  
  
    ...  
  
    // Set the result  
  
    px.set_value(x);  
}  
  
catch (...) {  
    // Exception caught - save it in the shared state  
  
    px.set_exception(std::current_exception());  
}
```

```
}
```

In the consumer thread, `get()` will throw, so we need to call it inside a try block.

```
try {
```

```
    // Get the result - may throw an exception
```

```
    int x = fx.get();
```

```
}
```

```
catch (...) {
```

```
    // Handle the exception
```

```
}
```

Here is the previous example, modified to throw an exception in the producer thread. This exception is handled by the consumer thread.

```
include <future>
```

```
#include <iostream>
```

```
#include <thread>
```

```
#include <exception>
```

```
#include <chrono>
```

```
using namespace std::literals;
```

```
// Producer function with std::promise argument
```

```
void produce(std::promise<int>& px)
```

```

{
    try {
        int x{42};

        std::this_thread::sleep_for(1s);

        throw std::out_of_range("Oops");

        std::cout << "The promise sets the shared state" << " to " << x << "\n";

        // Set the result

        px.set_value(x);
    }
    catch(...) {
        px.set_exception(std::current_exception());
    }
}

```

// Consumer function with std::future argument

```

void consume(std::future<int>& fx)
{
    std::cout << "The future calls get()..." << "\n";

    try {

        // Get the result

```

```
int x = fx.get();

std::cout << "The future returns from get()\n";

std::cout << "The answer is " << x << "\n";

}

catch(std::exception& e) {

    std::cout << "Exception caught: " << e.what() << "\n";

}

}
```

```
int main()

{

    // Create the promise object

    std::promise<int> p;

    // Obtain the associated future

    std::future<int> f = p.get_future();

    // Start the consumer thread

    // Pass the future to it

    std::thread fut{consume, std::ref(f)};
```



```
// Start the producer thread

// Pass the promise to it

std::thread prom{produce, std::ref(p)};

fut.join();

prom.join();

}
```

The output now looks like this:

The future calls get()...

Exception caught: Oops

Chapter Twenty-Five

Shared Futures

Promises with Multiple Waiting Threads

Single Producer and Multiple Consumers

We have learned how to write applications where there is one producer thread and one consumer thread. However, many applications require multiple consumer threads. For example, the producer thread could be reading stock prices, and there could be consumer threads which perform automated trades, send alerts to brokers, update customer account balances, and so on.

`std::future` is designed to have exclusive access to the shared state. Calling `get()` destroys the shared state. Once the value has been read by a thread, the shared state no longer exists.

`std::future` objects cannot be copied. If multiple threads try to share the same `std::future` object, there will be a data race when they call any of its member functions. This is because there are multiple accesses to the shared state, and `std::future` does not perform any synchronization.

This is intentional: `std::future` is designed for use by a single consumer thread which fetches a value, once.

`std::shared_future`

`std::shared_future` allows multiple consumer threads to wait for a result from the producer thread.

`std::shared_future` can be copied, so each consumer thread can have its own object instance. It performs synchronization when accessing the shared state, so calling member functions from different objects is thread-safe. Calling `get()` does not destroy the shared state, so it is safe to call `get()` repeatedly, even from the same object. Since the promise can only send the result once, this will always

give the same value.

The shared state will only be destroyed when the last `std::shared_future` object which uses it is destroyed.

`std::shared_future` is analogous to `std::shared_ptr`, which is designed to allow multiple objects to share the same allocated memory.

Obtaining an `std::shared_future` object

As with `std::future`, we do not normally create an `std::shared_future` object directly. Usually we obtain an `std::future` object and convert it to an `std::shared_future`.

We can move an existing `std::future` object into an `std::shared_future`:

```
std::promise<int> p;  
  
std::future<int> f = prom.get_future();  
  
std::shared_future<int> sf1{std::move(f)};
```

Or we can call the `std::future`'s `share()` member function:

```
std::shared_future<int> sf1{f.share()};
```

We can also obtain an `std::shared_future` object directly from the promise:

```
std::shared_future<int> sf1{p.get_future()};
```

In all these cases, the `std::future` object is moved into the `std::shared_future` object. The moved-from object is no longer bound to any shared state. It is not safe to use the `fut` variable again, unless it is re-initialized.

Example with `std::shared_future`

We will write a program which has a single producer thread and multiple consumer threads.

The producer entry function is the same as before. The consumer entry function is almost the same: we just need to change the argument to `std::shared_future`.

```
void consume(std::shared_future<int>& fx) { ... }
```

In the `main()` function, we create the `std::promise` and obtain its associated `std::future`. We then convert this to an `std::shared_future` and make a copy of it, because each consumer thread needs its own copy. Finally, we start the threads, passing a different copy to each consumer thread.

```
int main()
```

```
{
```

```
    std::promise<int> p;
```

```
    // Move the promise's future into a shared future
```

```
    std::shared_future<int> sf1{p.get_future()};
```

```
    // Make a copy for the second consumer thread
```

```
    std::shared_future<int> sf2 = sf1;
```

```
    // Start two consumer threads
```

```
    // Each has its own copy of the shared future
```

```
    std::thread fut1{consume, std::ref(sf1)};
```

```
    std::thread fut2{consume, std::ref(sf2)};
```

```
    // Start the producer thread
```

```
// Pass the promise to it
```

```
std::thread prom{produce, std::ref(p)};
```

```
fut1.join();
```

```
fut2.join();
```

```
prom.join();
```

```
}
```

The output will look like this:

The future calls get()...

The future calls get()...

The promise sets the shared state to 42

The future returns from get()

The answer is 42

The future returns from get()

The answer is 42

When to Use Futures and Promises

We now know two different ways of communicating between threads (apart from using shared data protected by a mutex): condition variables and a promise with a future.

These are intended for different purposes. A condition variable is used to indicate that an event has occurred, while promise and future send a result from

one thread to another.

A promise can only send a result once. A condition variable can send as many results as desired.

The promise with a future is easier to set up, and can deal with exceptions. A condition variable requires a mutex, and extra code to lock the mutex, and perhaps code to avoid lost and spurious wakeups.

For sending a result from one thread to another, once, use a promise with `std::future`. For sending the same result to multiple threads, once for each thread, use a promise with `std::shared_future`.

If you want to signal multiple events to other threads, `std::condition_variable` may be a better choice.

Chapter Twenty-Six

Atomic Operations Introduction

Thread-Safety Without Mutexes

Integer Operations and Threads

On the x86 architecture, integer operations are always performed as a single instruction (assuming the data fits into a single machine word and is correctly aligned.)

This means that a thread cannot be interrupted while it is performing an integer operation. If we are using a shared integer, do we still need to protect it with a mutex?

Here is some code to test this:

```
#include <thread>
```

```
#include <iostream>
```

```
#include <vector>
```

```
// Shared integer
```

```
int counter {0};
```

```
// Thread entry point
```

```
void task()
```

```
{
```

```
// Do we need a lock here?
```

```
for (int i = 0; i < 100'000; ++i)
    ++counter;
}

int main()
{
    std::vector<std::thread> tasks;

    for (int i = 0; i < 10; ++i)
        tasks.push_back(std::thread{task});

    for (auto& t: tasks)
        t.join();

    std::cout << counter << "\n";
}
```

Each thread increments counter 100,000 times, and there are 10 threads. We expect the final value of counter to be exactly 1,000,000. However, when we run the program, it displays a much smaller number.

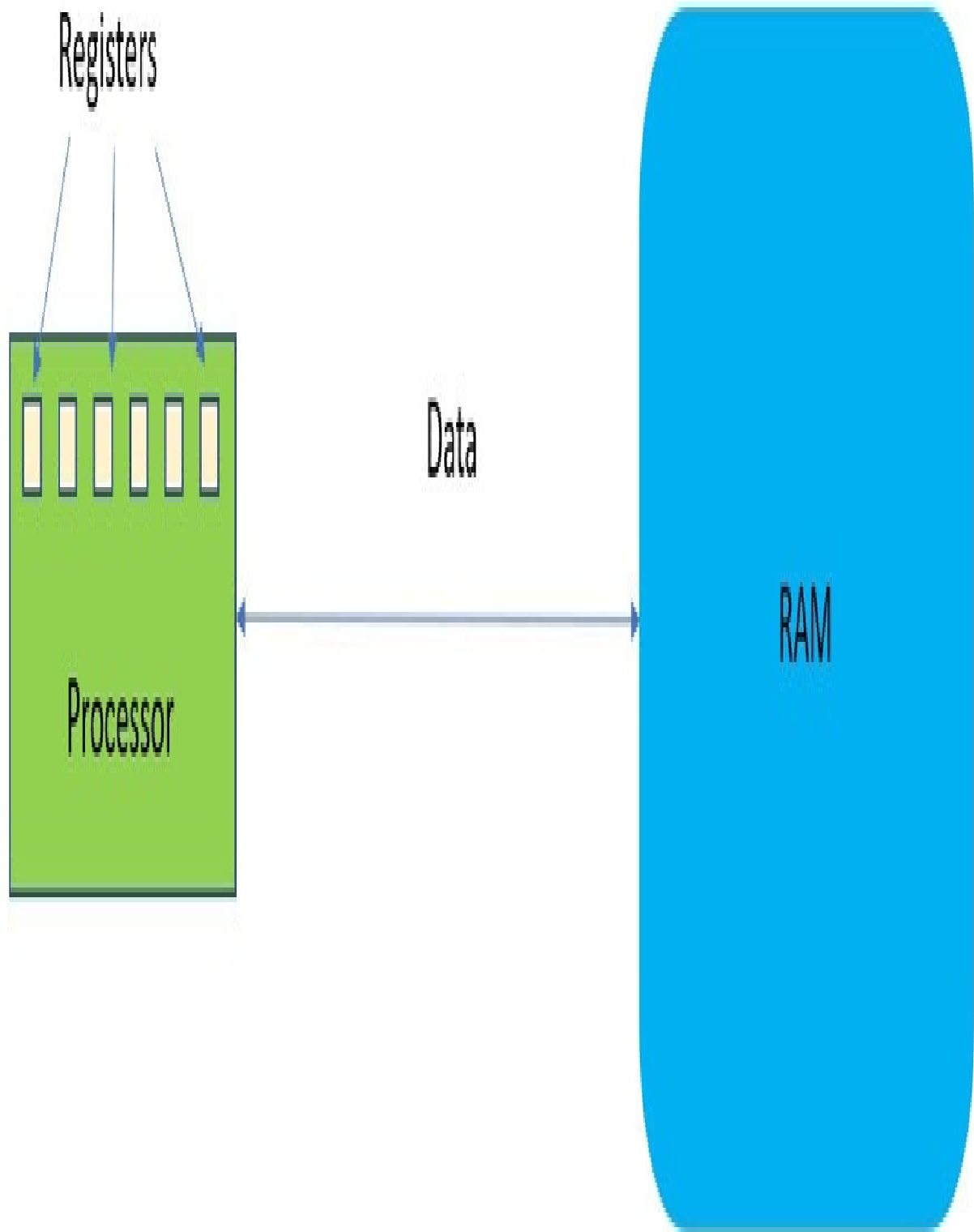
A "language lawyer" would observe that there is a data race, because of the conflicting accesses to counter. The program's behaviour is undefined. The program could display any number, or format the computer's hard drive, and it would still comply with the C++ standard.

This is true, but it does not help us understand why our program has gone wrong. To do this, we need to learn a little about how computers work.

Simplified Computer Architecture

Modern computer hardware is very complicated. We will keep things simple and assume that we are using a PC from around 1985, which has a single processor and some RAM. (Obviously it will also have a screen and a keyboard, and so on, but we are not concerned about those.) The processor has some internal memory cells, called "registers"; these are used for storing data which the processor is currently working on, among other things.

Every time a thread is interrupted on this single processor system, a task switch occurs. The register values for the current thread are saved and the values needed by the new thread are loaded up into the registers.



Very simple computer architecture. The program's data is stored in RAM and fetched into processor registers when required.

The processor executes the instructions from the program's binary executable file. The variables are stored in RAM.

When the processor needs a variable for some operation, it fetches its value from RAM and copies it into one of its registers, then it does the operation. If this operation modifies the value of the variable, the processor stores the new value in RAM.

So, in fact, our `++count` statement actually consists of three distinct operations:

- Copy the current value of count from RAM into one of the processor's registers
- Increment the value in the processor register
- Copy the new value of count to RAM

The actual increment done by the processor is a single instruction and cannot be interrupted. However, the thread could be interrupted either before or after the increment, and this allows data corruption to occur.

Data Corruption Scenario

A thread is about to increment count. It fetches the current value from RAM, which is 5, say, and stores that in a register. At that point, the thread is interrupted, and its register values are saved.

The register values for the second thread are loaded up. This thread is also about to increment count, so it fetches the value from RAM, which is 5. The second thread does its increment and saves the value 6 in RAM.

The first thread now resumes execution and its own register values are restored. The processor increments the value in the register to 6 and writes it to RAM. It has no way of knowing that another thread has already changed the value of count, so it overwrites it with its own value, which is also 6.

The result of these executions is that count initially had the value 5. It was

incremented twice, so it should now have the value 7, but in fact it has the value 6. Due to the timing of when the threads ran, one of the increments was lost.

There are other scenarios, for example if a thread is interrupted after doing the increment but before saving the result in RAM.

This leads us to a very important conclusion about data races: preventing a thread from being interrupted during a modification is not enough. We also need to make sure that other threads will see the modified value when they need to.

Modern hardware is much more complicated. Transfers to and from RAM go through several levels of "cache" memory. Systems have multiple processors, which have internal buffers, and so on. There are many more situations where different threads can have different views of the current value of shared data. In fact, it is not really meaningful to say that the shared data has a "current value".

However, the basic problem is the same. We need to make sure that

- A thread cannot be interrupted while modifying shared data
- The next thread to access the shared data will use the modified value.

With a mutex, this is done automatically when the mutex is unlocked.

Atomic Variable

An alternative to using a mutex is to declare count as an "atomic" variable. This causes the compiler to issue special processor instructions which make the hardware "lock" the variable. Other threads cannot interrupt while count is being modified; when they do access it, they will see the modified value.

This is different from a mutex, which locks a critical section of code and uses the operating system to control access to it. However, an atomic variable and a mutex are both ways of preventing data races.

Chapter Twenty-Seven

Atomic Types

How to Make an Atomic Variable

Atomic Type

C++11 defines a template for atomic types in `<atomic>`. The parameter is the type of the variable, which must be "trivially copyable". Loosely speaking, this means a built-in type, or a class where the copy and move operators do not do anything special.

Usually only variables of type `bool` and `int` are made atomic. For more complex types, the implementation may use a mutex instead. This rather defeats the point of trying to avoid mutex locks! A better alternative is to use a pointer instead.

In C++17, `atomic<std::shared_ptr>` is guaranteed not to use a mutex.

Using an Atomic Variable

Atomic variables must be initialized.

```
// Define x as an atomic int, with initial value 0
```

```
std::atomic<int> x{0};
```

We can assign to and assign from atomic variables.

```
// Assignment to x is an atomic operation
```

```
int y;
```

```
x = 2;
```

```
// Assignment from x is an atomic operation, even if the argument is not atomic
```

```
y = x;
```

Note that these assignments are two distinct atomic operations. Each operation on its own is atomic, but the combination of the two is not. Another thread could interleave between these two statements and change the value of `x` before the first thread can assign to `y`. It is not safe to assume that the second assignment will result in `y` having the value 2.

Atomic Variable Example

Going back to the counter example in the last chapter, we only need to make one change to make it work as expected:

```
#include <atomic>
```

```
...
```

```
std::atomic<int> counter{0};
```

Now a thread cannot be interrupted while modifying `counter`; each thread will use the result of the previous thread's modification. There are no lost increments, and the final value of `counter` is 1,000,000.

The Volatile Keyword

In Java and C#, the `volatile` keyword has a particular meaning in multi-threaded programs. Any modification to a volatile variable within a synchronized method or statement will immediately be visible in every other thread. This is similar to the way that an atomic variable in C++ releases its value after the operation, although the locking is done differently.

This is not the case in C++, where `volatile` has exactly the same meaning as in single-threaded code: The value of a volatile variable may change, even if there is no code which explicitly modifies it. It is an instruction to the compiler that it must not optimize away reads and writes to the variable.

This is mainly used in programs which interact with hardware devices. Typically,

a hardware device appears to be located at an address in memory: reading from that address will fetch data from the device, and writing to the address will send data to the device.

A program could for example read five times in succession from a device's address, to get the next five packets of data. An optimizing compiler will remove the "duplicate" statements, leaving only a single read; this is not what the programmer intended. However, if the data is read into a variable which was declared volatile, the compiler will leave all five read statements in the program.

Before C++11 added support for multi-threading, some programmers mistakenly used volatile where we would now use atomic. To add to the confusion, Visual C++ had a non-standard interpretation of the volatile keyword, which gave the same behaviour as in Java and C#. This was removed in later versions of the compiler which supported C++11.

```
volatile int counter{0};
```

If we make the counter variable in the program volatile instead of atomic, we still get incorrect results. Clearly, this is not the correct solution.

Chapter Twenty-Eight

Atomic Operations

How to Use an Atomic Variable

Member Functions of `std::atomic`

The supported operations depend on the type parameter.

These operations are supported by all instantiations:

`store()` atomically replaces the object's value by its argument.

`load()` atomically returns the object's value.

`operator =()` is a synonym for `store()`.

`operator T()` is a synonym for `load()`.

`exchange()` atomically replaces the object's value by its argument, and returns the object's previous value.

The operator synonyms allows programmers to write normal-looking code:

```
// Code using operator synonyms
```

```
// Calls x.operator =()
```

```
x = 2;
```

```
// Calls x.operator int()
```

```
y = x;
```


// Equivalent code using store() and load()

```
x.store(2);
```

```
y = x.load();
```

It is probably better to use store() and load(), as this makes it obvious that the variable is atomic. Also, if the variable's type is accidentally changed to non-atomic, there will be a compiler error, which indicates that something is wrong. This will not happen with the operator synonyms.

Pointer Specializations

Pointer specializations also support pointer arithmetic operations, increment and decrement operators, and the += and -= operators.

Integer Specializations

Specializations for integer types support some additional operators:

fetch_add() atomically adds its argument to the object and returns the object's previous value (similar to postfix ++)

fetch_sub() does the same, but for subtraction

fetch_and(), fetch_or() and fetch_xor() do the same for bitwise logical operations

These also have operator synonyms: operator +=, operator -=, operator &=, operator |= and operator ^=.

Chapter Twenty-Nine

Spin Locks

Locking without a Mutex

std::atomic_flag

std::atomic_flag is an atomic Boolean type. It is guaranteed to be lock-free and has less overhead than std::atomic<bool>.

std::atomic_flag supports only three operations:

clear() sets its value to false

test_and_set() sets its value to true and returns the previous value

operator=() sets its value to the argument and returns the previous value

An std::atomic_flag object must be initialized using the ATOMIC_FLAG_INIT symbol. This sets its value to false.

```
std::atomic_flag flg = ATOMIC_FLAG_INIT;
```

Spin Lock

In a spin lock, a thread runs in a loop which checks whether the lock is available.

We can use a shared std::atomic_flag object to implement a basic spin lock. The thread which has the lock sets the flag to true. When it releases the lock, it sets the flag to false.

A thread which is waiting for the lock checks the flag in a loop. If its value is true, then another thread has the lock, and the thread will check the flag again on the next loop iteration. If the flag's value is false, then the thread acquires the lock by setting the flag to true.

We cannot use a non-atomic bool variable for the flag because there would be a data race. The thread must check the flag and set it in a single operation, which cannot be interrupted.

The `test_and_set()` member function of `std::atomic_flag` is ideal for this. It sets the flag's value to true. If a thread already holds the lock, the previous value was true, so `test_and_set()` will return true. This indicates that our thread does not hold the lock.

However, if the previous value is false, then no thread holds the lock, because the last thread that held the lock set the flag to false when it released the lock. `test_and_set()` will return false, and this indicates that our thread now holds the lock.

The flag's value is now true, so `test_and_set()` will return true in other threads. This indicates that those other threads do not have the lock.

Here is the loop for the spin loop, in its entirety:

```
while (lock_cout.test_and_set()) {}
```

An alternative way of looking at this is that all the threads are competing to change the flag's value from false to true. The "winner" is the first thread which succeeds; the reward is that it can execute the critical section.

There are no prizes for coming second; a thread which sets the flag to true after another thread has already set it to true cannot execute the critical section.

Spin Lock Example

Here is a program with an example of a spin lock. It is similar to an earlier example, when we used a mutex to control access to `std::cout`.

We start by creating a global `std::atomic_flag` object and initializing it. This gives it the value false.

The first thread to call `test_and_set()` will set the flag's value to true. The return value will be the initial value of the flag, which was false. This thread exits its wait loop and executes the critical section.

If another thread calls `test_and_set()`, the return value will now be true. The thread will continue its loop.

When the first thread exits the critical section, it sets the flag's value to false. If another thread calls `test_and_set()`, the return value will be false and that thread executes the critical section. Other threads will get a return value of true and continue looping.

```
#include <atomic>
```

```
#include <thread>
```

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std::literals;
```

```
// Use std::atomic_flag as a simple lock
```

```
// Initialize it to false
```

```
std::atomic_flag lock_cout = ATOMIC_FLAG_INIT;
```

```
void task(int n)
```

```
{
```

```
    // "Lock" the flag
```

```
    while (lock_cout.test_and_set()) {}
```

```
    // Pretend to do some work...
```

```
std::this_thread::sleep_for(50ms);

std::cout << "I'm a task with argument " << n << "\n";

// "Unlock" the flag

lock_cout.clear();

}

int main()

{

    std::vector<std::thread> threads;

    for (int i = 1; i <= 10; ++i)

        threads.push_back(std::thread{task, i});

    for (auto& th : threads)

        th.join();

}
```

The output looks like this:

I'm a task with argument 1

I'm a task with argument 8

I'm a task with argument 3

I'm a task with argument 9

I'm a task with argument 4

I'm a task with argument 5

I'm a task with argument 7

I'm a task with argument 2

I'm a task with argument 10

I'm a task with argument 6

Pros and Cons of Spin Locks

A spinning thread remains active, so it can continue immediately when it gets the lock. With a mutex, the thread may need to be reloaded or woken up.

Waiting for a spin lock is very processor intensive, because the thread is running in a tight loop. This is something that should only be done for a very short time, preferably when there are not many threads contending for the lock.

If spinning threads interrupt each other, or are rescheduled, then this can have a very serious impact on performance.

For these reasons, spin locks are mainly used in operating systems or in libraries, by programmers who understand how to work with the thread scheduler at a low level.

Mutex Implementation

One of the main applications of spin locks is for implementing mutexes. Many operating systems implement a "hybrid" mutex. The thread briefly tries to get a spin lock; if it cannot do this immediately, it then tries to get a mutex lock. This is faster than a mutex if the critical section is not already locked, but it avoids the problems with spin locks if there is contention for the lock.

Acquire and Release Semantics

The spin lock is another example of acquire-release semantics.

If `test_and_set()` returns false, the thread acquires exclusive access to the critical section.

When the thread exits the critical section and clears the flag, it releases exclusive access to the critical section. It also releases the result of any modifications that were made in the critical section. The next thread for which `test_and_set()` returns false will acquire the modified values.

These acquire and release semantics impose an ordering on threads which access the variable and the critical section. There is no data race, and the data is always in a consistent state.

Chapter Thirty

Lock-free Programming

Look, No Locks!

What is Lock-free Programming?

In lock-free programming, threads can safely execute critical sections without using the operating system's locking facilities.

Lock-free programs can avoid, or at least reduce, some of the drawbacks to using locks:

- Data races caused by forgetting to lock, or locking the wrong mutex
- Risk of deadlock when using multiple mutexes
- The overhead of making a call into the operating system
- Lack of scalability from coarse-grained locking (e.g. if the entire linked list is locked, only one thread can access the list at any one time)
- Code complexity and increased overhead from fine-grained locking (e.g. if each element in a linked list is locked separately)
- Lack of composability. A function which acquires a lock should not call any function which takes a lock, even if it is on a different mutex.

Locking vs Lock-free

Locking and lock-free programming are two different ways of managing shared data. Managing shared data is rather like designing a traffic intersection. The critical section corresponds to a traffic junction and the threads in the program correspond to vehicles going through the junction.

Obviously, the designer wants the vehicles to get through the junction as quickly as possible. However, if all the vehicles try to go through at the same time, there will be chaos!



Threads executing a critical region without synchronization

One solution is to put some sort of controller on the intersection. This could be a policeman, or a set of traffic lights (stop lights.) Each vehicle approaching the junction has to stop and wait until it is allowed to proceed.



Controller which only allows one thread to execute a critical region at a time

This is analogous to using a lock to protect the critical section.

Another option is to have a motorway intersection (freeway interchange.) This allows traffic from different levels to go over the same section at the same time. Traffic from one level can merge with traffic from a different level without having to stop.

The intersection needs to be designed very carefully, to avoid situations where vehicles can collide.



Threads using lock-free programming to execute a critical section

This is analogous to lock-free programming.

Advantages of Lock-free Programming

If lock-free programming is done correctly, threads can never block each other. Deadlock and livelock cannot happen, because there are no locks.

This is useful in real-time systems, where a task must be completed within a time limit. It is also useful in low-latency systems, where data must be processed with minimal delays.

Disadvantages of Lock-free Programming

Writing lock-free programs, which are correct and efficient, is very difficult, even for experts. It requires thinking about code in a way which many programmers find unnatural.

For example, when we write some code, we assume that it will be executed in the same order that we wrote it. Without even realizing it, we expect the values of the variables will be modified in a given order. In lock-free programming, this may not necessarily be true.

Writing high-quality lock-free code requires a very good understanding of processor architectures and cache systems, which is beyond the scope of this book.

The extra complexity of lock-free programming makes it unsuitable for many applications. For example, if a GUI application uses separation of concerns to keep the various functional features in different threads, there is no advantage to making it lock-free.

The main advantage from lock-free programming is when there is some data structure in the program which is subject to high contention, and performance is critical. For example, a financial application where a buffer or queue receives all the incoming orders and using mutexes would add too much delay.

Double-checked Locking

[Earlier in this book, we looked at the double-checked locking algorithm for implementing a singleton. We saw that C++11, and later C++17, added features to make this thread-safe.](#)

In older versions of C++, and in other languages, the only way to implement this safely is to make the class member atomic, or the equivalent. (Some thread libraries, such as Boost and ACE, supported atomic variables before C++11 added them.)

Written using the equivalent classes from C++11, the code would look like this:

```
#ifndef SINGLETON_ATOMIC_H

#define SINGLETON_ATOMIC_H

#include <iostream>

#include <mutex>

#include <atomic>

std::mutex mut;

class Singleton {

    // Private static member

    static std::atomic<Singleton *> single;

public:

    // Static member function which
```



```

// returns a Singleton object

// This is the only way to get a Singleton

static Singleton* get_singleton()
{
    // If the member object does not already exist

    if (single == nullptr) {
        std::lock_guard<std::mutex> lg(mut);

        // If the member object does not already exist

        if (single == nullptr) {

            // Create it

            single = new Singleton;

        }

    }

    return single;
}

```

protected:

```

// The constructor is protected and

// can only be called by a static member function

Singleton()
{

```

```
        std::cout << "Initializing singleton\n";
    }
};
```

```
#endif //SINGLETON_ATOMIC_H
```

Test program for this Singleton class:

```
#include "singleton_atomic.h"
```

```
#include <thread>
```

```
#include <vector>
```

```
// Define the static member variable
```

```
std::atomic<Singleton *> Singleton::single{nullptr};
```

```
// Thread entry point function
```

```
void task()
```

```
{
```

```
    // Get the Singleton object
```

```
    Singleton* s = Singleton::get_singleton();
```

```
    // Display its address
```

```
    std::cout << s << "\n";
```

```
}
```

```
int main()
```

```
{
```

```
    std::vector<std::thread> threads;
```

```
    for (int i = 0; i < 10; ++i)
```

```
        threads.push_back(std::thread{task});
```

```
    for (auto& t : threads)
```

```
        t.join();
```

```
}
```

Chapter Thirty-One

Asynchronous Programming

Let Me Get Back to You

Synchronous and Asynchronous Operations

Operations can be either synchronous or asynchronous. When we start a synchronous operation, we have to wait until it finishes before we can start the next one.

This is like buying something in an old-fashioned shop. You ask for the item, then the assistant fetches it, then you agree to buy it, then you pay for it, then you take it home and then you can use it. Each step has to be completed before the next one can start.

With an asynchronous operation, we can start another operation straight away, without waiting for the first one to complete. The first operation executes "in the background".

An asynchronous operation is like buying something over the Internet. Once you send off your order, you can do something else while you wait for the item to arrive. You do not have to stop and wait for the item to arrive, unless of course you need to do something that requires the new item.

Synchronous Task

Imagine we are writing some code which needs some data that is stored in a database. The workflow could look like this:

- Do some work
- Request the data from the database
- Wait for the data

- Receive the data from the database
- Continue working

This is an example of a synchronous task.

In C++, function calls are normally synchronous. For example, if a class has a member function that stores data in a file:

```
data.save(filename);
```

```
// Stop and wait for the function call to return
```

```
...
```

```
// Now we can continue with the next operation
```

```
...
```

While this thread is waiting for the function call to return, it cannot do any work of its own. If the next operation does not depend on the data being saved, then that waiting period is "dead time" which increases the program's execution time.

Asynchronous Task

We could also write the database code to use an asynchronous task. The workflow would now look like this:

- Do some work
- Start a task which requests the data from the database
- Do some other work
- Receive the data from the database when the task completes
- Continue working

The original task can do other work, provided this work does not require the data which is being fetched.

When the original task gets to a point where it needs the data, only then does it have to stop and wait. It is possible that the fetching task could already have finished by then, in which case the original task can continue without stopping.

If the class has a member function that stores data in a file asynchronously, the pseudo-code would look like this:

```
data.async_save(filename);
```

```
// Continue working while the async task runs
```

```
...
```

```
// Check if the async task has completed
```

```
...
```

```
// Get the result of the async call
```

```
...
```

Blocking and Multi-threaded Programs

Blocking is undesirable in multi-threaded programs. It reduces the throughput and responsiveness of the blocked thread. Also, any threads which are waiting for this thread will also be blocked.

Blocking in a critical section is particularly bad, because any other thread which wants to execute that section will also be blocked. If the critical section is protected by locks, there is also a risk of deadlock.

Using asynchronous programming techniques reduces the need for threads to block, although it cannot be removed completely. If the asynchronous operation has not completed when it is needed, the caller will still need to wait for it; however, the caller may at least have been able to achieve some useful work, instead of being completely idle.

Non-blocking Synchronization

Operations which block are synchronized by mutexes or atomic operations. A thread will remain blocked until another thread unlocks the mutex or completes the atomic operation.

Non-blocking operations are synchronized by message queues:

- Producer threads push a message onto a queue. This is a concurrent queue, so adding and removing messages is thread-safe.
- Consumer threads remove messages from the queue and process them.
- The other threads continue running without waiting.

For example, an Internet store could have threads which communicate with customers and take their orders. These threads push the order details onto a queue. Some more threads remove the order details from the queue and process them, before sending them off to the store's warehouse which will dispatch the physical goods.

This is often implemented by using messages which consist of callable objects; processing the message involves starting a new thread which uses the callable object as its entry point.

Unfortunately, writing a safe and efficient queue with multiple consumers and producers is very difficult, and the C++ standard library does not provide one. There are implementations in third-party libraries, including Boost, Microsoft's Parallel Processing Library (PPL) and Intel's Thread Building Blocks (TBB).

We will implement a simplified concurrent queue later in this book.

Asynchronous Programming and Parallelism

Asynchronous programming techniques can also be used in parallel processing, for example computations involving large arrays or matrixes.

To do this, we split the data into several parts, and start a separate thread to process each part. As each thread completes, we collect its result; when all the threads have completed, we combine all the partial results to get the final answer. As we will see later, when we look at parallelism, the C++17 standard

library has some support for this.

Finally, asynchronous programming does not necessarily imply threads. Single-threaded programs and operating systems can support asynchronous operations (even MS-DOS could print a document in the background!)

Chapter Thirty-Two

Packaged Tasks

Vacuum-packed Tasks

std::packaged_task

The `std::packaged_task` class is defined in `<future>`. It associates a task function with an `std::promise`.

An `std::packaged_task` is callable. When it is invoked, the task function is run in a thread. The return value from the function is stored in the associated promise. To access this result, we use the future which is associated with the packaged task's promise.

`std::packaged_task` bundles together everything that is needed to execute a thread and obtain a result from it: the code to be executed, a promise to store the result and a future to access the result.

`std::packaged_task` therefore provides a higher level of abstraction than `std::thread`.

Creating an std::packaged_task Object

`std::packaged_task` is a class template. The type parameter is the type of the task function:

```
// Task function takes const std::string& and returns int
```

```
std::packaged_task<int(const std::string&)> pkg;
```

The packaged task will create an internal `std::promise` object which is associated with `pkg`. The parameter type of this promise will be the return type of the task function, which is `int`.

The constructor for `std::packaged_task` takes a callable object, which is used as the task function. The `std::promise` object will be passed as an argument to this function, similar to the producer thread in the “Futures and Promises” chapter.

`std::packaged_task` is a move-only class which manages a resource: the `std::promise`.

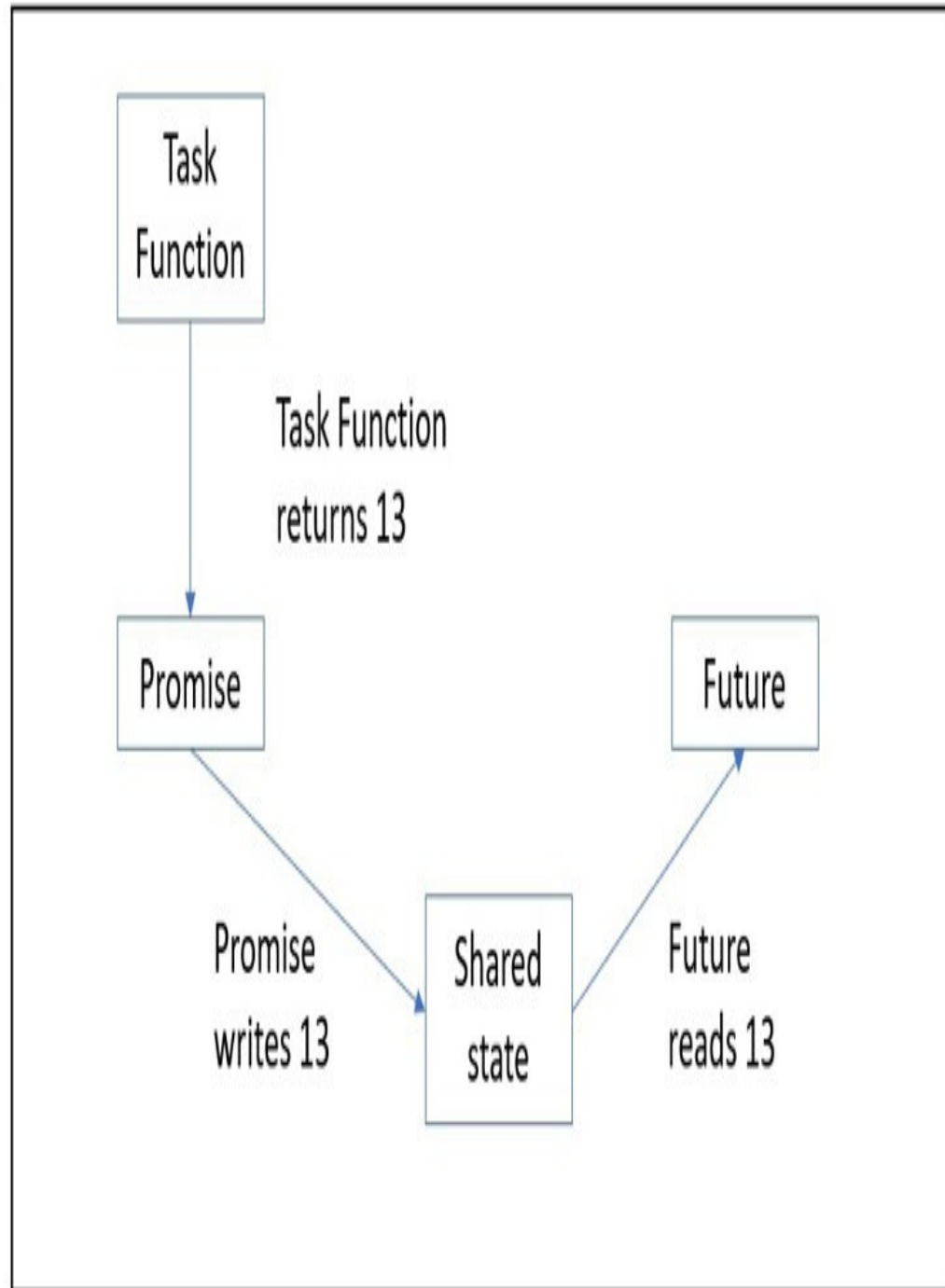
Using an `std::packaged_task` Object

Unlike `std::thread`, an `std::packaged_task` object does not immediately start executing the thread. Nothing will happen until the packaged task is started.

The packaged task can be started directly, but normally we would create an `std::thread` object and move the packaged task object into `std::thread`'s constructor. This will cause the packaged task's function to run in its own thread.

When the task function returns, the packaged task's internal promise object stores the return value in its shared state. To obtain an `std::future` object which can read from this shared state, we call the packaged task's `get_future()` member function.

Packaged
Task



A packaged task executes a task function in a thread. The packaged task has an internal promise object; this promise stores the returned value from the task function in its shared state. We can get this returned value from the future object which is associated with the shared state.

If the task function throws an unhandled exception, it will store it in the shared state. The exception will be re-thrown when the future calls `get()`.

std::packaged_task Example

```
#include <iostream>
```

```
#include <future>
```

```
int main()
```

```
{
```

```
    // Create a packaged task.
```

```
    // The task function is a lambda expression
```

```
    // It takes two ints and returns an int
```

```
    std::packaged_task<int(int, int)> pkg{[](int a, int b) { return a + b; }};
```

```
    // The packaged task has an associated promise
```

```
    // Get the corresponding future for the promise
```

```
    // Use the task function's return type for the type parameter
```

```
    std::future<int> fut = pkg.get_future();
```

```
// Create an std::thread to run the task

// Move the packaged_task into the constructor

std::thread t{std::move(pkg), 6, 7};


// Get the result of the task

std::cout << "The result is: " << fut.get() << "\n";

t.join();

}
```

We create an `std::packaged_task` object, using a lambda expression for the task function. We then call the packaged task's `get_future()` member function, which returns the `std::future` object corresponding to the `std::promise` which is associated with the packaged task.

We move the packaged task into `std::thread`'s constructor; this starts a new thread, which uses the lambda expression as its entry point. The `std::packaged_task` object is passed as the first argument, where the entry point function would normally go. The remaining arguments are forwarded as arguments to the lambda expression.

To obtain the result of the lambda expression, we call `get()` on the future object. This will read the value that the packaged task's promise object wrote into the shared state.

Advantages of `std::packaged_task`

Using `std::packaged_task` avoids having to use an `std::promise` object directly. On the other hand, the packaged task's promise can only store the return value when the function returns, while an explicit promise object can store any value at any time during the thread's execution.

We can create a container of `std::packaged_task` objects, then start up the threads when we are ready. `std::thread` is not convenient for this, because the threads

would start running as soon as the objects are constructed.

Packaged tasks are useful for managing threads:

- If we want each task to be run on a given thread
- If we want threads to execute in a particular order
- For creating a " thread pool " of threads waiting for work to arrive. We will look at this later in the course

However, if you just want to run a task in a thread and get its result, `std::async()` is easier to use.

Chapter Thirty-Three

The `std::async()` function

Another Way to Start a Thread

`std::async()`

`std::async()` also represents a higher-level abstraction than `std::thread`.

Calling `std::async()` starts a task in the background. The task may run on a new thread, allowing the current thread to continue with its work while the new task executes. Alternatively, the new task can run on the same thread.

`std::async()` is defined in `<future>`. The syntax is similar to `std::thread`'s constructor: the task function is passed as the first argument, and the remaining arguments are forwarded as arguments to the task function.

The `std::async()` version of "Hello, World!" looks like this:

```
include <iostream>
```

```
#include <future>
```

```
// The task function
```

```
void hello()
```

```
{
```

```
    std::cout << "Hello, Async!\n";
```

```
}
```

```
int main()

{

    // Call std::async()

    // Pass the task function as argument

    std::async(hello);

}
```

std::async() and std::future

std::async() returns an std::future object. This will contain the result of the background task, as soon as it is available.

When we are ready, we can call get() on this future object. This will return the result immediately, if it is available. If the result is not yet available, the get() call will stop and wait for it.

To find out if the result is available, but without fetching it, call wait(), wait_for() or wait_until().

For convenience, we will refer collectively to get(), wait(), wait_for() and wait_until() as "get() et al".

Launch Options

std::async() takes an optional extra argument. This decides whether the task will be run in a new thread.

The argument std::launch::async will start a new thread which uses the task function as its entry point. The task starts executing when the thread is launched.

The alternative is std::launch::deferred. The task will not be executed until one of get() et al is called. It will be executed in the thread which calls the member function.

This is an example of "lazy evaluation", in which an operation is not done until its result is required. (The opposite of lazy evaluation is "eager evaluation", when the operation is done immediately.)

`get()` et al can be called by a different thread from the one which called `std::async()`. For example, this initiating thread could start a new thread which is passed the `std::future` object in its constructor, and this new thread calls the future's member function.

If the optional first argument is not given, the implementation will decide whether to start a new thread or run the task in the current thread. This can also be achieved by doing a bitwise OR of the two options.

`std::async()` Example

```
#include <iostream>
```

```
#include <future>
```

```
#include <chrono>
```

```
using namespace std::literals;
```

```
// The task function
```

```
int func()
```

```
{
```

```
    std::cout << "Running in thread with ID " << std::this_thread::get_id() <<
    "\n";
```

```
    std::this_thread::sleep_for(2s);
```

```
    return 42;
```

```
}
```

```
int main()
```

```
{
```

```
    std::cout << "Main thread with ID " << std::this_thread::get_id() << "\n";
```

```
    std::cout << "Starting task\n";
```

```
    // The task will be run in a separate thread
```

```
    //auto result = std::async(std::launch::async, func);
```

```
    // The task will be run on this thread
```

```
    //auto result = std::async(std::launch::deferred, func);
```

```
    // The implementation chooses how to run the task
```

```
    //auto result = std::async(func);
```

```
    // The implementation chooses how to run the task
```

```
    //auto result = std::async(std::launch::async|std::launch::deferred, func);
```

```
    std::this_thread::sleep_for(1s);
```

```
    std::cout << "Calling get()\n";
```

```
std::cout << result.get() << "\n";  
}
```

The task function `func()` is run by calling `std::async()`. We display the ID of the thread which runs the task, and also the ID of the main thread which called `std::async()`. Comparing these will tell us whether the task is running in the same thread or a different one.

For example, with the `std::launch::async` policy, the output could look like this:

Main thread with ID 0x116943dc0

Starting task

Running in thread with ID 0x700004d94000

Calling `get()`

42

The task is running in a thread which has a different ID from the main thread. This policy caused `std::async()` to launch a new thread to run the task.

Default Launch Policy

With the default launch policy, we do not know whether a new thread will be started.

- The task may run concurrently with the thread that called `std::async()` .
- If `get()` et al are called in a different thread, it could run concurrently with that thread.
- The task may not run at all, if there is some path in the code which does not call any of `get()` et al
- If a thread uses thread-local storage, we do not know which thread's data will

be used

Launch Policy Recommendations

When calling `std::async()`, it is best to use an explicit launch policy.

Use `std::launch::async` if any of the following are true:

- The thread must run in a separate thread
- The task uses thread-local storage
- The task must be executed, even if `get()` et al are not called
- The returned future object is passed to a different thread which calls `wait_for()` or `wait_until()`

Use `std::launch::deferred` if the task must run in the same thread that called `std::async()`.

Otherwise, use the default policy. This allows the thread scheduler to choose the most efficient way to perform the task.

`std::async()` and Exceptions

As with `std::promise` and `std::packaged_task`, `std::async()` allows us to catch exceptions thrown while the task is executing. We call the returned future's `get()` member function inside a try block:

```
#include <iostream>
```

```
#include <future>
```

```
using namespace std::literals;
```

```
int func()
{
    int x{42};

    std::this_thread::sleep_for(2s);

    throw std::out_of_range("Oops");

    return x;
}

int main()
{
    auto f = std::async(func);

    try {
        auto result = f.get();

        std::cout << "Result: " << result << "\n";
    }

    catch (std::exception& e) {
        std::cout << "Caught exception: " << e.what() << "\n";
    }
}
```

Chapter Thirty-Four

Choosing a Thread Object

Which One to Use?

Choice of Thread Object

We now have three different ways to create an execution thread in standard C++:

- Call `std::async()`
- Create an `std::packaged_task` object
- Create an `std::thread` object

What are the relative strengths and weaknesses of these?

`std::async()`

`std::async()` is the simplest way to start a task. We can use it to get the return value from the task function; if the function throws an exception, we can get that as well.

We can choose whether to run the task synchronously or asynchronously. If we want to do some work which might block, but we want this thread to remain responsive, we can call `std::async()` to do the work in the background.

`std::async()` provides a higher level of abstraction than `std::thread`. The implementation manages the threads, so we do not need to create `std::thread` objects and call `join` on them. Threads can communicate by sending messages, instead of using shared data.

The main disadvantage of `std::async()` is that threads launched with the `std::launch::async` policy can only be joined. There is no way to detach the thread and leave it running indefinitely in the background.

The reason for this lies in the design of `std::future`'s destructor. The destructor will block until the thread completes, if the future was created by a call to `std::async()` with the `std::launch::async` policy. The reason for this is to keep the shared state available until the last future which uses it has completed.

`std::packaged_task`

`std::packaged_task` is the best choice if we want to use an object to represent an execution thread. For example, to create a container of execution threads.

A packaged task can be made to execute on a particular object, which is not possible with `std::async()`.

`std::packaged_task` has less overhead than `std::async()`, and is more flexible.

`std::thread`

`std::thread` is the most flexible of all. It is also the lowest level option.

We can use `std::thread` to access the underlying software thread; this allows us to use features provided by the operating system, which are not available in standard C++. For example, running a thread on a particular core, or changing the priority of a thread.

Finally, `std::thread` can be detached.

Recommendations

Use `std::async()` as the normal way of starting a thread.

Bjarne Stroustrup, the creator of C++, has compared `std::async()` to the range-for loop. For iterating over all the elements of a container, accessing each element once, in order, without adding or removing elements, you should use a range-for loop. If you want to do something more specific, such as only iterating over some of the elements, then you would use a lower-level loop and write some extra code.

Similarly, if you want to do something which is not supported by `std::async()`,

then you would use `std::thread` and write more code.

Use `std::packaged_task` if you do not want the thread to start executing immediately; for example, if you are creating a container of thread objects.

Chapter Thirty-Five

C++ Limitations

Unsupported Features

Features which are not Supported

The C++ library's support for asynchronous programming lacks a number of features found in other languages.

There is no support for continuations. With continuations, a program can start up several tasks which run in a sequence, then leave them to run in the background until all the tasks have completed. Instead, the program has to start each task separately and get its result before it can start the next one. This is less efficient and requires writing more code.

The C++ implementation can only wait on one future returned by `async()` at a time. In many languages, it is possible to start up multiple asynchronous operations and wait until they have all completed, or for the next future to become ready.

There is no way to cancel or suspend a running task.

As mentioned before, there is no concurrent queue implementation in the standard C++ library.

There have been plans to add these features since C++11, but they keep being postponed. They are not in C++20, and unfortunately it does not appear they will be in C++23 either.

However, there are third-party libraries which support some of these features.

Chapter Thirty-Six

Parallelism Overview

A Quick Introduction

Concurrency

"Concurrency" and "Parallelism" are often used interchangeably. However, it can be useful to distinguish between them.

Concurrency is when the tasks are conceptually distinct: different threads are doing different things.

Concurrency is often used for separation of concerns. For example, a Windows program could have one thread which responds to GUI events and other threads which implement different parts of the program's functionality.

Concurrency does not necessarily involve multiple processors. A concurrent program could run on a single processor core.

Concurrent tasks usually interact with each other. They wait for events, or they wait for other tasks to complete, and they can also exchange data.

Concurrency is a feature of the program structure; a consequence of the way the program is designed.

Parallelism

In parallelism, many instances of the same task run at the same time on multiple cores, to improve scalability. The tasks usually do not interact with each other.

Parallelism is a feature of the algorithm which is used to process the data. There are some algorithms where it naturally makes sense to divide the data into subsets; each subset is processed by a separate task instance.

Sporting Analogy

In sporting terms, concurrency is like a team sport. Each player has a distinct role, and the players interact with each other. Players wait to get the ball, or for another player to get into position, or for some event to occur, such as a penalty.

Parallelism is like an individual track sport, such as sprinting or swimming. Each player is doing the same thing as the others, and they are all trying to do it as fast as possible. There is no interaction between the players, or at least there is not supposed to be any!

Explicit Parallelism

There are two forms of parallelism, "explicit" and "implicit". With explicit parallelism, the programmer specifies exactly how the work should be divided. For example, you could divide the data into four parts and then start four threads; each thread processes one part of the data.

This involves more work for the the programmer, but it is not scalable. For example, if this program runs on a computer which has four available cores, the program will run at the maximum possible speed. If there are eight cores available, the program will still only use four of them and it will run at the same speed, even though it could run twice as fast in theory.

This approach is mainly useful when targetting specific hardware. For example, each generation of a game console has a fixed number of processors. The programmer can "tune" the program to make optimal use of the hardware.

It is also useful if the problem naturally divides into a fixed number of tasks, which cannot sensibly be divided further.

Implicit Parallelism

In implicit parallelism, the programmer allows the implementation to decide how to do the work in parallel. This should give the best use of the available resources, since the thread scheduler will know how many threads are available and how to allocate work among them.

In the situation where the program has eight cores available, it should run twice as fast as in the situation where only four cores available.

For most purposes, this is the best option.

Chapter Thirty-Seven

Execution Policies

What are Execution Policies?

Code Execution and Parallelism

Modern hardware supports four different ways of executing code. As an example, we will show how the sum of the integers from 1 to 8 could be computed for each option.

In "sequential" execution, a single instruction performs a single operation. This is the traditional C++ behaviour. For addition, each operation takes two arguments.

$1 + 2$	$\longrightarrow$	3
$3 + 3$	$\longrightarrow$	6
$6 + 4$	$\longrightarrow$	10
$10 + 5$	$\longrightarrow$	15
$15 + 6$	$\longrightarrow$	21
$21 + 7$	$\longrightarrow$	28
$28 + 8$	$\longrightarrow$	36

Sequential execution of $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8$

Each operation is performed in sequence, using the result of the previous addition as one of the arguments to the next addition. The calculation requires seven cycles on one processor.

In "vectorized" execution, an instruction can take more than the usual number of arguments. For example, a vectorized addition operation could add four integers in a single instruction.

$$\boxed{1+2+3+4} \longrightarrow \boxed{10}$$

$$\boxed{5+6+7+8} \longrightarrow \boxed{26}$$

$$\boxed{10+26} \longrightarrow \boxed{36}$$

Vectorized execution of $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8$

The first four numbers are added in a single operation, followed by the second four numbers. Finally, the intermediate results are added to give the final result. The calculation requires three cycles on one processor.

This requires support from the hardware, and also the data has to be arranged in contiguous memory. This is true for built-in arrays, `std::array` and `std::vector`, but not for other containers such as `std::list` or `std::map`.

In "parallelized" execution, two or more processors perform the same instruction, but with different parts of the data.

Processor 1

$$\boxed{1+2} \longrightarrow \boxed{3}$$

$$\boxed{5+6} \longrightarrow \boxed{11}$$

$$\boxed{3+11} \longrightarrow \boxed{14}$$

$$\boxed{14+22} \longrightarrow \boxed{36}$$

Processor 2

$$\boxed{3+4} \longrightarrow \boxed{7}$$

$$\boxed{7+8} \longrightarrow \boxed{15}$$

$$\boxed{7+15} \longrightarrow \boxed{22}$$

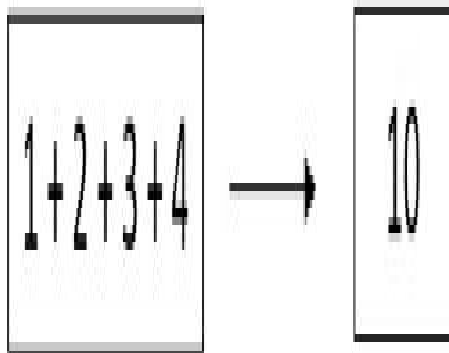
Parallelized execution of $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8$

The data is split into two parts. Processor 1 works on adding 1, 2, 5 and 6 while Processor 2 works on adding 3, 4, 7 and 8. Each processor then adds up the results of its own calculations. Finally, the result from each processor is added to give the final result. The calculation requires four cycles on two processors.

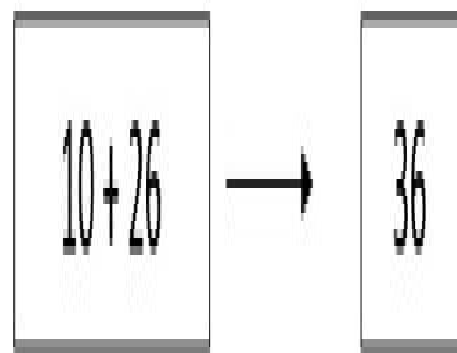
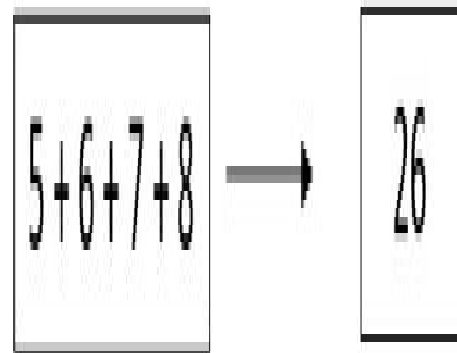
This requires multiple processor cores and a suitable algorithm. For example, sorting algorithms often split the data into half. Then they sort each half separately, by splitting that half again and sorting each sub-part separately, and so on. These sub-sorts could be performed on different processors at the same time.

Finally, it is possible to have "parallelized and vectorized" execution. Two or more processors perform the same instruction, which takes more than the usual number of arguments. The calculation requires two cycles on two processors.

Processor 1



Processor 2



Parallelized and vectorized execution of $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8$

This requires multiple processor cores, hardware support for vectorization and data which is in contiguous memory.

Standard Algorithms

C++98 added the "Special Template Library". This introduced a number of data structures and functions to the C++ standard library. These make extensive use of templates, so they can work with any data type.

The functions operate on containers and sequences of data. These implement classic Computer Science algorithms such as sorting and searching, and are often referred to as "standard algorithms". There are many other functions, for populating containers, copying elements, and so on.

The standard algorithms are mostly in `<algorithm>`, but a few are in `<numeric>`. Typically, they take an iterator range as argument, which corresponds to a data sequence or elements within a container. The function will iterate through the elements in the range and perform operations on them: it may modify the elements, swap them around, compare them, and so on. Usually, there is an optional argument, which is a callable object; this is used to customize the function's behaviour.

Execution Policies

C++17 adds "execution policies". With execution policies, the programmer can say how a standard algorithm function call should be executed. There are options for sequential execution, parallelized execution and parallelized and vectorized execution. If the programmer does not specify an execution policy, the algorithm will execute the same way it did in C++14.

An execution policy is only a request. The algorithm call may ignore it:

- If parallel and/or vectorized execution is not supported by the hardware, the compiler will not generate those processor instructions
- If there are not enough system resources to execute it efficiently; for example

the system cannot create any more threads

- If the implementation of the C++ library does not provide a parallel and/or vectorized version of this algorithm. Some algorithms are difficult to implement efficiently with execution policies, and there are still quite a few gaps.

In these cases, the algorithm will use the non-policy default, as in C++14.

Compiler Support for Execution Policies

At the time of writing, GNU C++ and Visual C++ are the only compilers which support execution policies. Other compilers, including clang, will not be able to compile the code examples in this and the following chapters.

Execution Policy Objects

C++17 defines three global execution policy objects in `<execution>`. These are in the `std::execution` namespace.

- `std::execution::seq` . Do not use parallelized or vectorized execution
- `std::execution::par` . Use parallelized execution only
- `std::execution::par_unseq` . Use both parallelized and vectorized execution

To specify an execution policy, pass the appropriate execution object as the first argument to the algorithm call

```
// Sort the elements of the container "vec"
```

```
// Using parallelized execution
```

```
std::sort(std::execution::par, begin(vec), end(vec));
```

If we do not pass an execution policy argument, we get the default behaviour:

```
// Using the default execution policy
```

```
std::sort(begin(vec), end(vec));
```

Non-policy Execution

With no execution policy argument, all the operations in the algorithm execution are performed on the same thread. This will be the thread which calls the algorithm function.

Operations will be performed in order, and will not be interleaved.

```
#include <iostream>
```

```
#include <algorithm>
```

```
#include <vector>
```

```
#include <execution>
```

```
using namespace std::execution;
```

```
int main()
```

```
{
```

```
    std::vector<int> vec(2000);
```

```
    int count {0};
```

```
    std::for_each(begin(vec), end(vec), [&] (int& x) { x = ++count; });
```

```
    for (auto v : vec)
```

```
        std::cout << v << ", ";
```

```
}
```

This code calls a standard algorithm without passing an execution policy argument. The call to `std::for_each()` iterates over each element of the vector,

and passes the element as argument to a lambda expression. This lambda expression captures the vector and the count variable by reference, so it can modify them. The vector element is assigned the value of count, which is incremented on each iteration.

After this call, the vector elements will have the values 1, 2, ..., 2000 in that order. The lambda expression calls have been performed in order and have not been interleaved.

Sequenced Execution

With the `std::execution::seq` policy argument, all the operations in the algorithm execution are performed on the same thread. This will be the thread which calls the algorithm function.

Operations will be performed in order, but may be interleaved.

```
std::for_each(seq, begin(vec), end(vec), [&] (int& x) { x = ++count; });
```

After this call, the vector elements will have the values 1, 2, ..., 2000, although they may not necessarily be in that order. The lambda expression calls have been performed in order and may have been interleaved.

Parallel Execution

With the `std::execution::par` policy argument, the operations in the algorithm execution are performed in parallel across a number of threads. This may include the thread which calls the algorithm function. Any single operation within the function call will run on the same thread for its entire duration.

Operations performed on the same thread will not be interleaved, but may not necessarily be executed in a specific order. Operations on different threads may interleave, so the programmer must prevent data races.

```
std::for_each(par, begin(vec), end(vec), [&](int& x){ x = ++count; });
```

This code has a data race: the variable count is shared between threads without any protection. The result is unpredictable; the most likely outcome is that some elements will have duplicate values.

Parallel Unsequenced Execution

With the `std::execution::par_unseq` policy argument, the operations in the algorithm execution are performed in parallel across a number of threads. This may include the thread which calls the algorithm function. This time, an operation within the function call may be migrated from one thread to another.

Operations performed on the same thread may be interleaved and may not necessarily be executed in a specific order. Operations on different threads may interleave, so the programmer must prevent data races.

The programmer must also avoid any modification of state between elements, or between threads:

- Memory allocation and deallocation
- Mutexes, locks and other forms of synchronization

```
std::for_each(par_unseq, begin(vec), end(vec), [&](int& x){ x = ++count; });
```

Again, this code has a data race and gives an unpredictable result.

Chapter Thirty-Eight

Algorithms and Execution Policies

How do Execution Policies Affect the Standard Library?

Algorithms and Execution Policies

In C++17, most of the functions in `<algorithm>` were re-specified to support execution policies. However, a few algorithms which are naturally sequential, such as `std::equal_range()`, were not changed.

Some functions in `<numeric>` were originally specified as sequential-only, even though the actual functionality did not require this. New functions were provided for these, which support execution policies:

- `std::accumulate()` has a new version called `std::reduce()`
- `std::partial_sum()` has two new versions, `std::inclusive_scan()` and `std::exclusive_scan()`

There is also a new "fused" algorithm, `std::transform_reduce()`, which is a re-implementation of `std::inner_product()`.

Algorithms and Exceptions

An exception can be thrown during a call to an algorithm function. In most cases, it will be a function provided by the programmer which throws, not the algorithm itself.

When an exception is thrown inside a non-policy algorithm call, we have the usual sequence of destructor calls and stack unwinding. The caller can handle the exception by calling the algorithm function inside a try/catch block, or leave it to be handled further up the call stack.

This is not viable with algorithms that have an execution policy. The algorithm call may be executing on multiple threads, each of which has its own stack. Transferring the exception to the calling thread and cleaning up the other threads would be complicated and add too much overhead. Instead, the `std::terminate()` function is called. By default, this immediately terminates the program.

Suppose we have an algorithm call which applies a function to every element in an iterator range. If this function throws an exception, but handles it internally, so the exception does not escape outside the function, the algorithm call will continue to execute. However, if the applied function does not handle the exception, or if it rethrows it, then the algorithm will call `std::terminate()`.

It is also possible for the algorithm call itself to throw an exception, even if it does not call any programmer-supplied function. For example, it could require a temporary memory resource and be unable to allocate the memory. In this case, the algorithm call would throw `std::bad_alloc`.

Exception Example

```
#include <algorithm>
```

```
#include <vector>
```

```
#include <exception>
```

```
#include <iostream>
```

```
#include <execution>
```

```
using namespace std::execution;
```

```
int main()
```

```
{
```

```
    std::vector<int> vec{3,1,4,1,5,9};
```

```

try {
    // Call the std::sort() algorithm
    // with a predicate function that throws

    std::sort(begin(vec), end(vec),
        [](int a, int b) {
            throw std::out_of_range("Oops");

            return true;
        });
}

catch (std::exception& e) {
    std::cout << "Caught exception: " << e.what() << "\n";
}
}

```

This code orders the elements of an `std::vector`. It calls the `std::sort()` algorithm function inside a try/catch block and passes a predicate function that throws an exception. The exception is not handled by the predicate function or `std::sort()`, but it will be caught and handled in the catch block.

The program will display the output "Caught exception".

If we now change the call to `std::sort()` to pass an execution policy:

```
std::sort(seq, begin(vec), end(vec), /* ... */);
```

The program will be terminated when the exception is thrown, due to `std::sort()` calling `std::terminate()`.

We might expect `std::execution::seq` to give the same behaviour as the non-

policy version, but in fact this is not the case. The non-policy version does not terminate the program when an exception is thrown. Also, as mentioned before, the sequential policy version is allowed to perform operations out of order.

Disadvantages of Execution Policies

Passing an execution policy argument may not have any effect. Currently only two compilers support execution policies, and those two have not yet implemented all the algorithms which are specified to take a policy argument.

If the library has not implemented a policy argument version, it is allowed to call the non-policy version instead. The program will behave the same way as if the non-policy version had been called.

Using a supported execution policy adds overhead. The algorithm will have to start threads and manage them, and this adds to the execution time and memory usage.

C++ often specifies "complexity requirements" for the algorithm functions. For example, in `std::sort()`, the number of element swaps must be equal to or less than the number of elements being sorted.

Often, good parallel implement require more operations than the sequential equivalent. The overall time taken is less, because more than one operation is performed at the same time. This can mean that policy versions have weaker complexity requirements than the non-policy version, or even none at all. It also means that compiler implementers do not have the same incentive to provide an efficient implementation.

When to Use an Execution Policy?

The best advice is not to use an execution policy unless you really need it. As with many forms of optimization, using an execution policy makes the code harder to understand, and it may not produce the expected benefits.

Only use an execution policy if the performance of the program is not acceptable without one.

"Premature optimization is the root of all evil" - Donald Knuth

Do not use an execution policy if any of these are true:

- The task is essentially sequential
- The order of operations is important
- The algorithm function call may throw an exception, and immediate termination is not acceptable
- The code involves a data race, and the overhead from synchronization outweighs the benefits of using an execution policy
- The code needs to be portable to compilers which do not support execution policies

On the other hand, if measurements show that an execution policy results in a definite and worthwhile improvement in performance, and it can be done safely and correctly, then you can consider using one.

Choosing an Execution Policy

The sequenced execution policy uses a single thread, which makes it much easier to use when testing and debugging. However, it allows out-of-order execution and calls `std::terminate()` on exceptions; this helps the programmer understand how these factors affect the program's execution.

Parallel unsequenced execution offers the most potential to improve performance, but has the strictest requirements. Use this when there are no data races and the code does not modify shared state.

Parallel execution can be used when vectorization is not supported, or is unsafe due to memory operations or using mutexes. Do not use parallel execution if there is a data race.

Chapter Thirty-Nine

New Parallel Algorithms

What's New?

Review of `std::accumulate()`

`std::accumulate()` adds each element in an iterator range to an initial value. It returns the sum of the elements plus the initial value.

```
std::vector<int> vec {1, 2, 3, 4};
```

```
// Sum the elements of vec, using the initial value 0
```

```
auto sum = std::accumulate(begin(vec), end(vec), 0);
```

After calling `std::accumulate()`, `sum` will have the value 10.

By default, the `+` operator of the elements is used to perform the addition. This can be customized by passing a callable object as an optional fourth argument.

Execution of `std::accumulate()`

The specification of `std::accumulate()` requires that it executes sequentially, processing each element in order. The above call will be executed like this:

```
// Performed in left-to-right order, one addition at a time
```

```
((((0 + 1) + 2) + 3) + 4)
```

This cannot be parallelized, because each operation must complete before the next one can start. Also, it cannot be vectorized, because each operation takes exactly two arguments.

Due to the way it is specified to behave, `std::accumulate()` cannot support execution policies.

`std::reduce()`

`std::reduce()` is a re-implementation of `std::accumulate()`, without these restrictions. The operations do not necessarily have to be sequential, or performed in any particular order.

```
std::vector<int> vec {1, 2, 3, 4, 5, 6, 7, 8};
```

```
// Sum the elements of vec, using the initial value 0
```

```
// Performed as seven sequential additions
```

```
// (1 + 2 + 3 + 4 + 5 + 6 + 7 + 8)
```

```
// or (1 + 3 + 8 + 5 + 2 + 4 + 7 + 6), etc
```

```
auto sum = std::reduce(seq, begin(vec), end(vec), 0);
```

```
// Performed as four parallel additions
```

```
// ((1 + 2) + (3 + 4) + (5 + 6) + (7 + 8))
```

```
auto sum = std::reduce(par, begin(vec), end(vec), 0);
```

```
// Performed as two parallel vectorized additions
```

```
// ((1 + 2 + 3 + 4) + (5 + 6 + 7 + 8))
```

```
auto sum = std::reduce(par_unseq, begin(vec), end(vec), 0);
```

Parallel Execution

When `std::reduce` is called with the `std::execution::par` policy, it breaks down the operations into something that looks like a tree.

$$1+2+3+4+5+6+7+8$$

10

26

$$1+2+3+4$$

$$5+6+7+8$$

3

7

11

15

$$1+2$$

$$3+4$$

$$5+6$$

$$7+8$$

The addition of eight numbers is broken down into two additions of four numbers. These are then broken down into four additions of two numbers. The addition cannot be broken down any further, because an addition requires at least two arguments.

The four additions will be performed in parallel, assuming enough threads are available. The results of these four additions will be combined in two parallel operations. Finally, these two results will be combined to give the final result of the addition.

This implements a typical pattern in parallel programming: a computation is split up into a number of tasks, which run in parallel. A "reduce" step then combines all the partial results from these tasks into the result of the computation.

std::reduce() Limitations

Operations may not necessarily be performed in sequence. If re-ordering the operations or re-grouping them alters the result, `std::reduce()` will not give the correct answer.

The operator used must be "associative" and "commutative". Commutative means that the ordering is not important - for example, $x + y$ gives the same result as $y + x$. Associative means that grouping operations does not affect the result: $(x + y) + z$ gives the same result as $x + (y + z)$.

Obviously, this is not true for operations such as subtraction and division. Less obviously, it may not be true for floating-point arithmetic, due to rounding errors, overflow and underflow.

Review of std::partial_sum()

`std::partial_sum()` is another library algorithm that had to be re-written to support execution policies.

`std::partial_sum()` iterates over the elements in an iterator range and calculates the rolling total of the elements so far. It writes the current value of this sum as an element in a destination.

```
std::vector<int> vec {1, 2, 3, 4};  
std::vector<int> vec2(vec.size());
```

```
// Write the partial sum of the elements of vec into vec2
```

```
// The elements of vec2 will be
```

```
// 1, 1 + 2, 1 + 2 + 3, 1 + 2 + 3 + 4
```

```
std::partial_sum(begin(vec), end(vec), begin(vec2));
```

The elements in vec2 will be 1, 3, 6 and 10.

As with `std::accumulate()`, the calculation must be done in a fixed order.

Note that vec2 must have at least as many elements as vec. This is because `std::partial_sum()` does not perform any bounds checking.

An alternative is to use `std::back_inserter()`, which automatically expands the destination by calling `push_back()`. However, it can invalid all the iterators into the vector; this creates a data race in threaded code, which makes it unsuitable for parallelized execution. Also, it may cause memory allocation and deallocation, which makes it unsuitable for vectorized execution.

Using a fixed-size vector with `begin()` as the destination is safe. Concurrent operations on different elements of `std::vector` do not cause a data race, provided no elements are inserted or removed. No memory operations will be performed, which makes it safe for vectorized execution.

`std::inclusive_scan()` and `std::exclusive_scan()`

`std::partial_sum()` has been reimplemented to allow out-of-order operations and execution policies. In fact, C++17 provides two separate functions, `std::inclusive_scan()` and `std::exclusive_scan()`.

`std::inclusive_scan()` gives the same result as `std::partial_sum()`:

```
// Write the partial sum of the elements of vec into vec2.
```

```
// The elements of vec2 will be
```

```
// 1, 1 + 2, 1 + 2 + 3, 1 + 2 + 3 + 4
```

```
std::inclusive_scan(begin(vec), end(vec), begin(vec2));
```

`std::exclusive_scan()` does not include the current element in the partial sum. Instead, it takes an extra argument, which is used as if it was the first element. The last element is not used.

```
// Write the partial sum of the elements of vec into vec2.
```

```
// Exclude the last element and add an extra element -1 at the front.
```

```
// Equivalent to std::inclusive_scan() with elements {-1, 1, 2, 3}.
```

```
//
```

```
// The elements of vec2 will be
```

```
// -1, -1 + 1, -1 + 1 + 2, -1 + 1 + 2 + 3
```

```
std::exclusive_scan(begin(vec), end(vec), begin(vec2), -1);
```

Here, the extra argument of -1 is used as the first element. The first element in the destination will therefore be -1, because the actual first element of `vec` is not used. For the second element of the destination, the first element of `vec`, which is 1, is added to the sum so far, which is -1, giving the result 0. When the call returns, `vec2` will have the elements {-1, 0, 2, 5}.

In effect, the elements of `vec` are shifted to the right, the last element is ignored, and an extra element is added at the front. The result of the `std::exclusive_scan()` call with elements {1, 2, 3, 4} is the same as calling `std::inclusive_scan()` with elements {-1, 1, 2, 3}.

This is often useful in numeric computation.

Review of `std::transform()`

`std::transform()` iterates through the elements of an iterator range, calls a given function object with each element as argument, then stores the result in a destination. Some functional languages provide a similar feature called "map".

// Double each element of vec and store the result in vec2

```
std::transform(begin(vec), end(vec), vec2.begin(),
```

```
    [](int n) { return 2*n; });
```

`std::transform()` has an overload which takes an extra argument; this represents a second source. It iterates over the elements from both sources and calls the function with an element from each source as argument, then stores the result in the destination.

// Add each element in vec to the corresponding element of vec2.

// Store the result in vec3

```
std::transform(begin(vec), end(vec), begin(vec2), begin(vec3),
```

```
    [](int n1, int n2) { return n1 + n2; });
```

When the call returns, the elements of `vec3` will be the sum of the corresponding elements of `vec` and `vec2`.

Transform and Reduce

A very common pattern in parallel programming is to "transform and reduce" the data:

- Divide the data into subsets
- Start a number of threads
- Each thread performs some operation on a subset of the data
- Combine the results from each thread into the final answer

This could be done by making each thread call `transform()`, then calling `reduce()`

to combine the results. However, using separate function calls adds extra overhead:

- Each thread which calls `transform()` has to store its output somewhere
- `reduce()` cannot start until all the threads have completed
- `reduce()` has to read in the output from the threads
- `reduce()` may need to start up threads itself

In C++17, we can do this with a single function call, `std::transform_reduce()`. It is based on `std::inner_product()`, but allows out-of-order execution and supports execution policies.

Review of `std::inner_product()`

`std::inner_product()` iterates through the elements in two iterator ranges. It multiplies each pair of corresponding elements together, then calculates the sum of all the products. This is equivalent to a scalar product in mathematics.

```
std::vector<int> x{1, 2, 3, 4, 5};
```

```
std::vector<int> y{5, 4, 3, 2, 1};
```

```
// Calculates x[0]*y[0] + x[1]*y[1] + ...
```

```
// 1*5 + 2*4 + 3*3 + 4*2 + 5*1 gives 35
```

```
auto result = std::inner_product(begin(x), end(x), begin(y), 0);
```

`std::transform_reduce()`

We can do the same calculation with `std::transform_reduce()`, optionally with an execution policy:

```
// Calculates 0 + x[0]*y[0] + x[1]*y[1] + ...
```

// $0 + 1*5 + 2*4 + 3*3 + 4*2 + 5*1$ gives 35

```
auto result = std::transform_reduce(par_unseq,
                                     begin(x), end(x),
                                     begin(y), 0);
```

`std::transform_reduce()` has a number of advantages when compared to `std::inner_product()`:

- It manages the creation of threads for the transform stage
- The reduce stage can run concurrently with the transform stage
- The reduce stage will receive the result of each transform thread as soon as it completes
- The threads from the transform stage can be re-used by the reduce stage
- No intermediate storage is needed to hold the transform results until the resume stage starts

Overloads of `std::transform_reduce()`

We can customize `std::inner_product()` and `std::transform_reduce()` by providing our own functions for the `+` and `*` operators, instead of using the default `std::plus` and `std::multiplies`.

We can replace the `*` operator by a "transform" function, which takes two arguments of the element type and returns a value of some intermediate result type.

We can replace the `+` operator by a "reduce" function, which takes two arguments of that intermediate result type and returns a value of the final result type.

Example of Overloading `std::transform_reduce()`

This example is based on a posting on Marco Arena's blog.

The results of a scientific experiment are stored in an `std::vector<double>`. Another `std::vector<double>` stores the values predicted by theory.

We want to find the largest "error" in the results. This will be the maximum discrepancy between an expected result and the actual corresponding result.

We can use `std::transform_reduce()` to do this, by overloading the operators. The transform stage will find the differences between corresponding elements in the two vectors. The reduce stage will go through these differences and find the largest one.

We replace the `*` operator by a function that returns the difference between corresponding elements in the two vectors.

```
[](auto l, auto r) { return std::fabs(l - r); }
```

This will be called with an element from each vector as argument. We need to use the absolute value, because the difference could be negative and we are only interested in its size. We use `std::fabs()` from `<cmath>` for this, because the elements are doubles.

We replace the `+` operator by a function that returns the larger of its arguments

```
[](auto a, auto b) { return std::max(a, b); }
```

The program could look like this:

```
#include <iostream>
```

```
#include <numeric>
```

```
#include <vector>
```

```
#include <execution>
```

```
#include <cmath>
```

```
using namespace std::execution;
```

```
int main()
```

```
{
```

```
    // The predicted results
```

```
    std::vector<double> expected{0.1, 0.2, 0.3, 0.4, 0.5};
```

```
    // The results of the experiment
```

```
    std::vector<double> actual{0.09, 0.22, 0.27, 0.41, 0.52};
```

```
    auto max_err = std::transform_reduce(par_unseq,
```

```
        begin(expected), end(expected),
```

```
        begin(actual),
```

```
        // Initial maximum error
```

```
        0.0,
```

```
        // The reduce operation finds the biggest difference
```

```
        [](auto a, auto b) { return std::max(a, b); }
```

```
        // The transform operation finds each difference
```

```
[](auto l, auto r) { return std::fabs(l - r); });
```

```
std::cout << "The largest error is " << max_err << "\n";
```

```
}
```

`par_unseq` is the execution policy for parallelized and vectorized execution. `expected` is the vector of theoretical predictions and `actual` is the vector of results. `0.0` is the initial value of the maximum error found so far; each time the transform operation returns a larger value, it will overwrite the current maximum.

`std::transform_reduce()` will iterate through the elements of `expected` and `actual`. It will call the transform operation and pass an element from each vector as argument to it. The transform operation will return the difference between the elements. The reduce operation will check whether this difference is greater than the maximum so far. If it is, it will update the maximum.

The output from the program is 0.03.

Chapter Forty

Concurrent Queue Implementation

A Simple Concurrent Queue

Concurrent Data Structures

In the final chapters of this book, we will look at implementing some simple thread-safe data structures. We will start with a concurrent queue.

The C++ library provides a `std::queue` class, so let's remind ourselves how that works.

std::queue

`std::queue` is a FIFO structure ("First In, First Out").

- Elements are stored in the order they were inserted
- The first element to be inserted is at the front of the queue
- The most recent element is at the back of the queue
- Elements are inserted by "pushing" them onto the back of the queue
- Elements are removed by "popping" them from the front of the queue

Removing an element involves two separate operations:

- Call the `front()` member function to get a reference to the element at the front
- Call the `pop()` member function to erase the element at the front

This is different from the implementation in most languages, where there is a single operation that returns the front element and removes it.

When `std::queue` was added in C++98, the language only supported two ways to do this in a single operation:

- Return the element by reference after removing it. The caller will get a dangling reference (a reference to an element that no longer exists)
- Make a copy of the element, remove the element, then return the copy by value. If the copy constructor throws an exception when returning, the element is lost and there is no way to recover it.

Clearly, neither of these alternatives was acceptable. C++98 therefore specified that removing the element requires two separate operations. If move operators had been available, which was not the case until C++11, the element could have been moved into the return value. That would be exception-safe, because move operations on STL container elements are not allowed to throw exceptions.

Another drawback of `std::queue` is that calling `pop()` on an empty container has undefined behaviour.

`std::queue` and Threading

Having to use two separate operations for removing an element creates a race condition. For example, a thread calls `front()` to get the value of the element, and stores it, but it is interrupted before it can call `pop()`. Another thread could then run and call `pop()`. When the first thread resumes, it calls `pop()`, but the element it removes is not the same element whose value it stored.

The program is in an inconsistent state. If the second thread removed the last element, then the first thread will call `pop()` on an empty queue, which is undefined behaviour.

More generally, `std::queue` does not protect against data races. There will be a data race if there are conflicting accesses to the same `std::queue` object from different threads.

Concurrent Queue

The simplest way to write a concurrent queue is to write a "wrapper" class. This

will have an `std::queue` object as a member, and an `std::mutex`. Each public member function will lock the mutex, then call the appropriate member function(s) of the queue.

This will be thread-safe, because only one thread can call a member function of the `std::queue` at any time. There is no possibility of a data race, because only one thread can access the queue at any one time.

Concurrent Queue Member Functions

For the special member functions, nothing unusual is required. The defaults will call the corresponding member functions of the `std::queue` member and the mutex, and this is sufficient.

The `push()` member function will lock the mutex member, then forward its argument to the `push()` member function of the queue member. It then unlocks the mutex and returns.

The `pop()` member function takes the element value as an output parameter (by non-const reference.) It will lock the mutex, call the queue member's `front()` member function, and store the returned value in its argument. It then calls the queue member's `pop()` member function and unlocks the mutex.

If `pop()` is called when the queue member is empty, it should behave in a well-defined way, for example by throwing an exception.

Concurrent Queue Class Definition

The code could look like this:

```
#include <queue>
```

```
#include <mutex>
```

```
// Define a namespace for the concurrent queue
```

```
namespace cq {
```

// Exception to throw when popping an empty queue

```
class concurrent_queue_exception : public std::runtime_error {  
    public:  
    cq_exception() : std::runtime_error("Queue is empty") {}  
    cq_exception(const char *s) : std::runtime_error(s) {}  
};
```

// The concurrent queue class

```
template <class T>  
class concurrent_queue {  
    // The mutex to protect the queue  
    std::mutex m;  
    // The wrapped queue object  
    std::queue<T> q;  
    public:  
    // Use defaults for the special member functions  
    concurrent_queue() = default;  
  
    // Thread-safe call to std::queue::push()  
    void push(T value)
```

```
{  
    std::lock_guard<std::mutex> lg(m);  
    q.push(value);  
}
```

```
void pop(T& value)
```

```
{  
    // Lock the mutex  
    std::lock_guard<std::mutex> lg(m);  
  
    // Do not pop() an empty queue!  
    if (q.empty()) {  
        throw cq_exception("Queue is empty");  
    }  
  
    // Save the front element's value  
    value = q.front();  
  
    // Remove the front element  
    q.pop();  
}
```



```
};
```

```
}
```

```
// End of namespace cq
```

We define a class for the exception object which is thrown when popping an empty queue. This class is derived from `std::exception`, so it can be caught by a generic exception handler. This is thrown before calling the queue member's `pop()`, so the queue member will not have been modified.

The only other place where an exception can be thrown is when copying the return value from calling the queue member's `front()`, because the element's copy constructor could throw an exception. Again, the queue member has not been modified.

In both cases, the mutex will be unlocked when the local destructors are called. This implementation provides the strong exception guarantee, in which everything is left as it was before the member function call.

Example usage:

```
#include "concurrent_queue.h"
```

```
#include <iostream>
```

```
#include <future>
```

```
#include <string>
```

```
#include <chrono>
```

```
using namespace std::literals;
```

```
// Create a concurrent queue object
```

```
cq::concurrent_queue<std::string> conc_queue;
```

```
// Waiting thread
```

```
void reader()
```

```
{
```

```
    std::string sdata;
```

```
// Pretend to be busy...
```

```
    std::this_thread::sleep_for(2s);
```

```
    try {
```

```
        std::cout << "Reader calling pop...\n";
```

```
// Pop the data off the queue
```

```
        conc_queue.pop(sdata);
```

```
        std::cout << "Reader received data: " << sdata << "\n";
```

```
    }
```

```
    catch (std::exception& e) {
```

```
        std::cout << "Exception caught: " << e.what() << "\n";
```

```
    }
```

```
}
```

```
// Modifying thread
```

```
void writer()
```

```
{
```

```
    std::cout << "Writer calling push...\n";
```

```
    // Push the data onto the queue
```

```
    conc_queue.push("Populated");
```

```
    std::cout << "Writer returned from push...\n";
```

```
}
```

```
int main()
```

```
{
```

```
    auto w = std::async(std::launch::async, writer);
```

```
    auto r = std::async(std::launch::async, reader);
```

```
    r.wait();
```

```
    w.wait();
```

```
}
```

The output from the program is:

Writer calling push...

Writer returned from push...

Reader calling pop...

Reader received data: Populated

If the sleep is moved from the reader thread to the writer thread, the pop() member function will be called on an empty queue and an exception will be thrown. The output from the program is now:

Reader calling pop...

Exception caught: Queue is empty

Writer calling push...

Writer returned from push...

Concurrent Queue with Condition Variable

This implementation throws an exception if pop() is called when the queue is empty. This is safe, but not particularly convenient. The caller will have to catch the exception, then try again later, when the queue may have some data on it.

A better solution would be for the pop() call to block until another thread pushes some data onto the queue. This can be done by making it wait on a condition variable. A thread which calls push() will notify the condition variable; the waiting thread will then wake up and call the queue member's pop().

To avoid lost and spurious wakeups, we add a predicate to the wait() call. If the queue is empty, this returns false and the pop() call continues waiting. If the queue is not empty, this returns true and the pop() call will proceed to call the queue member's pop().

The concurrent_queue class will have another member, which is an std::condition_variable.

The member functions will now look like this:

```
void push(T value)
```

```
{  
    std::lock_guard<std::mutex> lg(m);  
    q.push(std::forward<T>(value));  
    cv.notify_one();  
}
```

```
void pop(T& value)  
{  
    // Lock the mutex  
    std::unique_lock<std::mutex> lg(m);  
  
    // Do not pop() an empty queue!  
    cv.wait(lg, [this]() { return !q.empty(); });  
  
    // Save the front element's value  
    value = q.front();  
  
    // Remove the front element  
    q.pop();  
}
```

We now need to use `std::unique_lock` instead of `std::lock_guard` in `pop()`. This is

because the condition variable needs to unlock the mutex.

`q.empty()` will return true if there is no data on the queue. We want to keep waiting until it returns false; to use it in a predicate, we need to invert the return value.

The lambda expression captures the `this` object, to give it access to the queue member.

The same test program can be used, although this time there is no need to put the call to `pop()` inside a try block.

Conclusions

This is rather a simplistic implementation of a concurrent queue. It employs "coarse-grained" locking, in which only one thread can access the queue at any one time. In effect, when a multi-threaded program accesses the queue, it temporarily becomes single-threaded.

Adding the condition variable makes the program more flexible, and it also makes the graining slightly less coarse-grained. Other threads can call different member functions while `pop()` is waiting for data to arrive.

A much more efficient solution would be to implement the queue ourselves, using lock-free programming techniques. The queue would perform atomic operations instead of using mutexes. In high-performance computing, a ring buffer (circular queue) is often used instead of a linear queue. Writing such a data structure, and making it correct and efficient, is unfortunately beyond the scope of this book.

Chapter Forty-One

Thread Pools

Re-using Threads

Thread Creation Overhead

So far, we have created a new thread for every time we start a task. Creating a thread involves a considerable amount of overhead:

- Create an execution stack for the new thread
- Call a function in the operating system's API
- Wait for the operating system to create its internal data to manage the new thread
- Wait for the scheduler to start running the thread
- Perform a context switch to actually run the thread

Starting a thread to execute a task function can take as much as 10,000 times as long as calling the function directly. Is there any way to avoid this overhead, or at least reduce it?

Typing Pools

Before computers were widely used in offices, all correspondence had to be typed by hand. Many companies had a "typing pool". This consisted of a roomful of typists who spent the entire day typing out letters and memos on typewriters. If a manager wanted to send a letter, a handwritten draft was taken to the typing pool and placed in a pile of pending work. (Later on, letters were dictated over telephone or recorded on tape, but the principle is the same.) When a typist completed their current letter, they would take the next letter from the pile and start working on that. In this way, all the typists were kept busy and the

company avoided having to find a new typist for each letter.

Thread Pools

A thread pool works similarly to a typing pool. There is a collection of running threads (typists) and a queue (pending pile) of work to do. When each thread finishes its current task, it takes the next task off the queue and starts executing that.

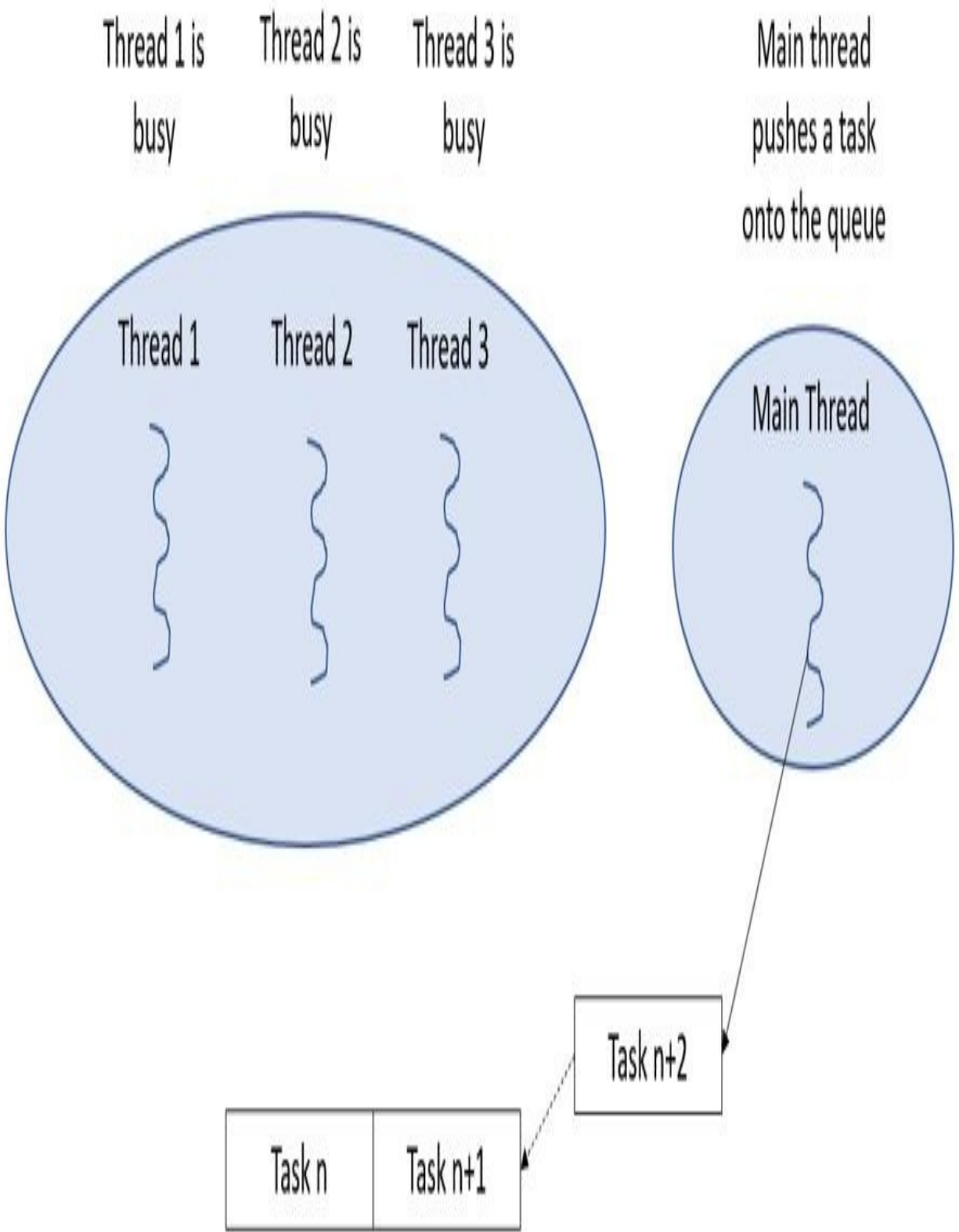
To implement a thread pool, we need a fixed-size container of thread objects, which are ready to start executing tasks.

Usually, the size of this container will be equal to the number of processor cores on the machine; perhaps reduced by two, to allow for the main thread and the operating system. If there are more software threads than processor cores, the performance will be affected by task switching and context switching, and threads will not be able to run all the time. If there are fewer, then the program will not be performing at its maximum efficiency.

The C++ library provides the `std::thread::hardware_concurrency()` function. This returns the maximum number of hardware threads supported by the machine the program is running on, which is the number of processor cores.

We also need a concurrent queue of task function objects.

Before, when a client had a task to be executed, it created a new thread object and passed the task function to the thread's constructor. Now, it pushes the task function onto the queue.

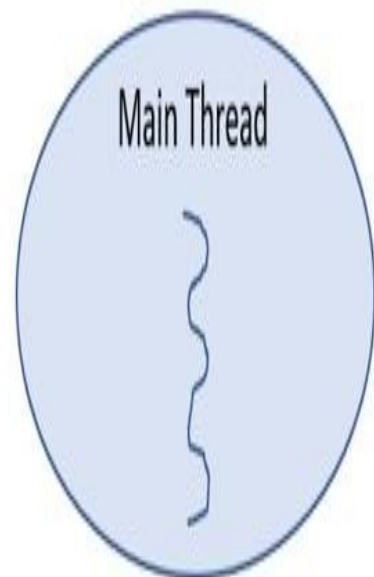
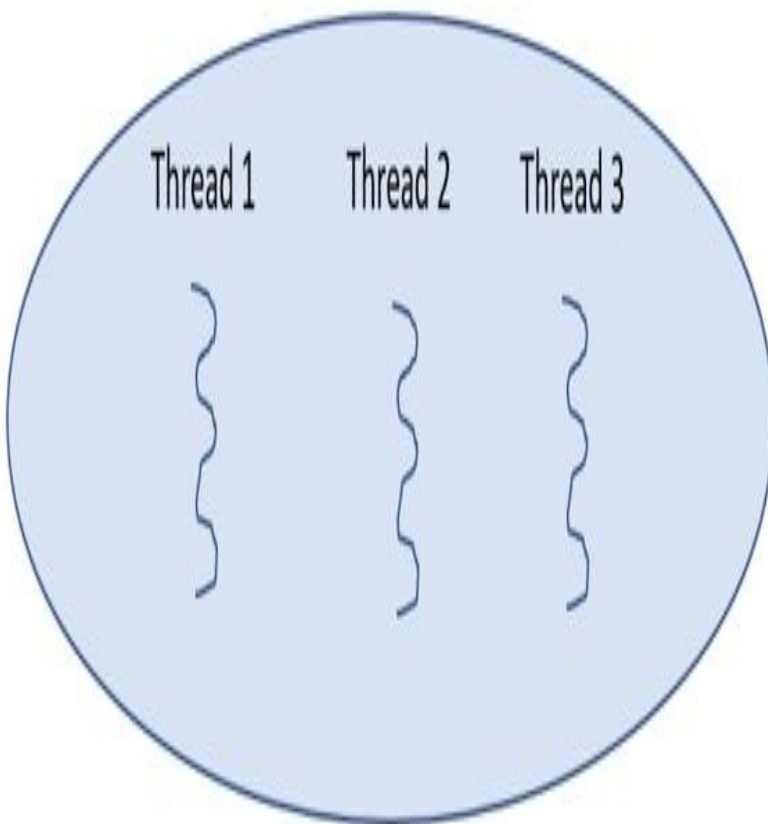


A manager sends a draft letter to the typing pool. It goes into the pile of pending work

Thread 1 is
busy

Thread 2 is
idle

Thread 3 is
busy



Task n	Task n+1	Task n+2
--------	----------	----------

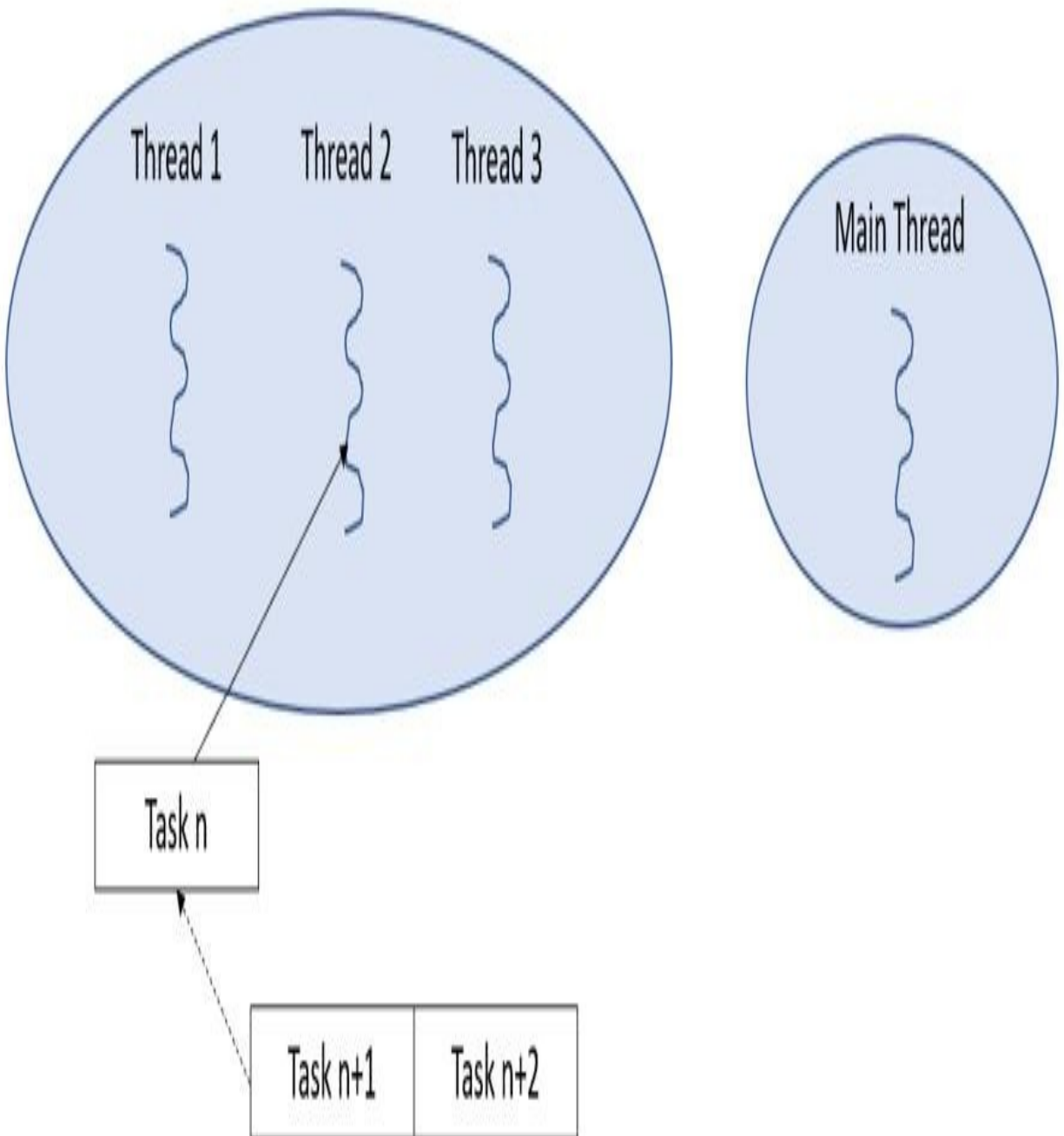
A typist finishes typing a letter and is ready for the next one

A thread object, which was created earlier by the container, will pop the task from the queue and start executing it.

Thread 1 is
busy

Thread 2
pops the
next task

Thread 3 is
busy

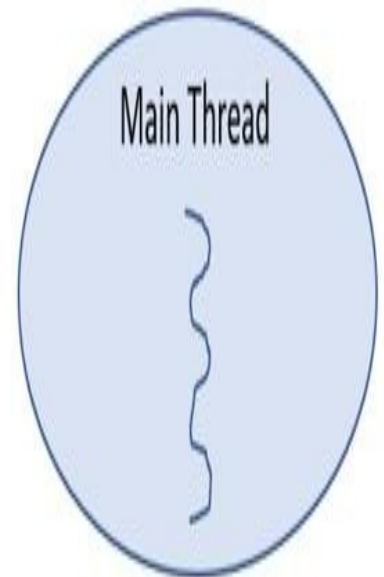
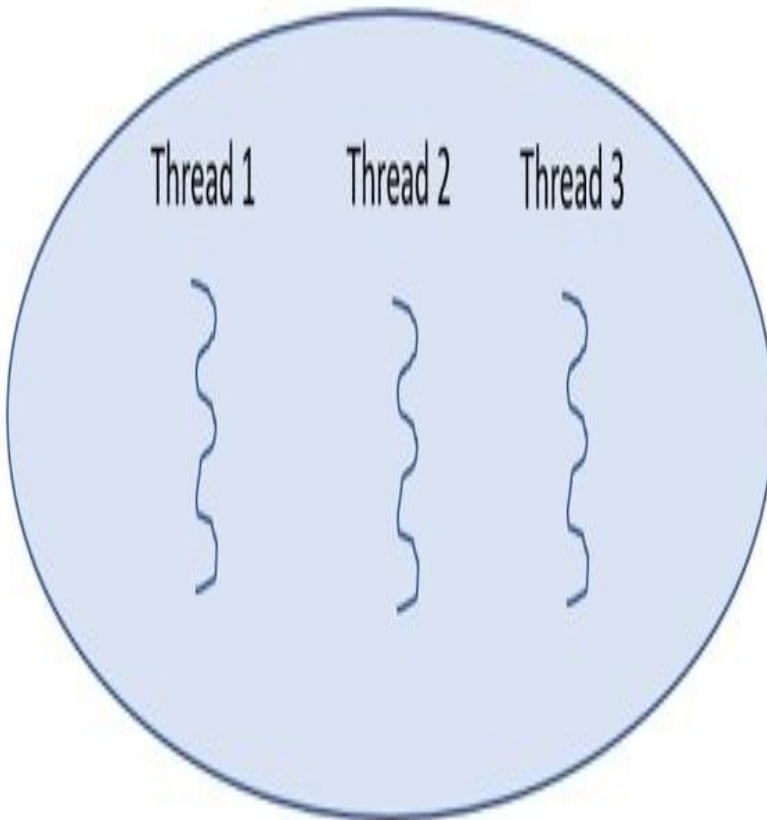


The typist takes the next draft off the pending pile and starts work on it

Thread 1 is
busy

Thread 2 is
busy

Thread 3 is
busy



Task n+1	Task n+2
----------	----------

The typist starts work on the next letter

The next thread to become idle will pop the next task off the queue. And so on.

Advantages of Thread Pools

A thread pool makes scaling very easy. If the program is run on a different machine, which has more processor cores, the thread pool will automatically make use of all the available hardware.

Thread pools make efficient use of resources. All the threads are always busy, constantly popping tasks off the queue and executing them. The only time a thread is idle is if there is no work available for it to do.

If all the processor cores are running a thread, and each thread is always busy, then the computer is working at its full capacity.

Thread pools work best for short and simple tasks, which do one thing, where the time to execute the task function is shorter than the time taken to create a new thread.

Also, the task function should not block. If the task needs to perform input/output, or wait for another thread, or lock a mutex, then that thread's processor core is not being fully utilized. It may be that the delay caused by blocking outweighs the time saved by not starting a new thread.

Disadvantages of Thread Pools

Adding and removing task functions from the queue must be done in a thread-safe way, which adds overhead.

If tasks are added to the queue faster than the threads can process them, the queue can become a "bottleneck" which slows down the entire program.

Chapter Forty-Two

Thread Pool Implementation

How to Recycle Threads

Overview

We will write a class which implements a thread pool. This will need a fixed-size container of `std::thread` objects, and a concurrent queue to store incoming tasks. We can use `std::vector` for the thread container, and we can re-use the concurrent queue implementation from earlier for the queue. These will be private data members of the class.

The class will have a public member function `submit()`. When clients call this, the task function argument will be pushed onto the queue.

Each thread in the container has an entry point function `worker()`, which is an infinite loop. In this loop, the thread calls `pop()` on the queue. We will use the queue implementation with a condition variable, so the call will wait until there is at least one element on the queue.

When the `pop()` call returns, the thread will have a task function. The thread then calls this task function.

Task Function

For simplicity, we will assume that the task functions take no arguments and return nothing. We can use an `std::function` object to store any callable object which meets this requirement. To simplify the code, we create a type alias

```
using Func = std::function<void()>;
```

Thread Pool Class Definition

Our thread pool header will look like this:

```
#ifndef THREAD_POOL_H
#define THREAD_POOL_H

#include "concurrent_queue_cv.h"
#include <atomic>
#include <thread>
#include <functional>
#include <vector>

// Type alias to simplify the code
using Func = std::function<void()>;

class thread_pool {
    // The concurrent queue of task functions
    cq::concurrent_queue<Func> work_queue;

    // The container of threads
    std::vector<std::thread> threads;

    // The entry point function for the threads
```

```

    void worker();

public:
    thread_pool();
    ~thread_pool();

    // Member function to push a task on the queue
    void submit(Func f);
};

#endif //THREAD_POOL_H

```

Thread Pool Class Implementation

The constructor obtains the number of processor cores and then populates the vector with `std::thread` objects. These will use `worker()` as their entry point function. Since we are passing a member function pointer to `std::thread`'s constructor, we must also give the object it will be called on.

The destructor calls `join()` on all the threads in the vector.

```

#include "thread_pool.h"

// Thread entry point function
void thread_pool::worker()
{
    // Infinite loop

```

```
while (true) {  
    Func task;  
  
    // Get the next task from the queue  
    work_queue.pop(task);  
  
    // Call the task function  
    task();  
}  
}  
  
// Thread pool constructor  
thread_pool::thread_pool()  
{  
    // Find the number of hardware threads  
    const unsigned thread_count = std::thread::hardware_concurrency();  
  
    // Create that number of std::thread objects  
    // Use worker() as the entry point  
    // Populate the container  
    for (unsigned i = 0; i < thread_count; ++i)
```

```
        threads.push_back(std::thread{&thread_pool::worker, this});  
    }
```

```
// Destructor
```

```
thread_pool::~thread_pool()  
{  
    // Join all the threads in the container  
    for (auto& t: threads) {  
        t.join();  
    }  
}
```

```
// Function for clients to submit tasks
```

```
void thread_pool::submit(Func f)  
{  
    work_queue.push(f);  
}
```

Thread Pool Class Usage

To use the thread pool, create an object of the class and call `submit()` with a suitable task function as argument.

```
void task()
```

```
{  
    // ...  
}
```

```
thread_pool pool;
```

```
pool.submit(task);
```

In a real program, this would probably be done inside a loop as well. For example, this thread could receive an order, select an appropriate task function to process the order, then send it off to the thread pool to be processed.

This would require modifying the thread pool to work with task functions that take arguments.

Chapter Forty-Three

Conclusion

Summing Up...

Multi-threading

Multi-threading is an important part of modern programming. Used correctly, it can considerably improve the performance of programs. However, there are a number of pitfalls.

Data Races

The most important thing to remember when writing multi-threaded code in C++ is to avoid data races.

A data race occurs when multiple threads make conflicting accesses to the same memory location.

The easiest way to prevent a data race is to protect accesses to the memory location with a mutex lock. This must be done carefully, to avoid the possibility of deadlock.

Deadlock

Deadlock usually arises when a thread has more than one mutex locked, so try to avoid that. If the thread does need to lock multiple mutexes, it should lock them all in a single operation.

Lock-free Programming

Lock-free programming has obvious attractions, compared to using mutexes, but it is very difficult to get right. Even code written by experts can contain hard-to-detect bugs.

Lock-free programming should only be used when code with mutexes is too slow, and only then by programmers who have a deep knowledge and experience of working with processor architectures and cache systems.

And Finally...

You have now reached the end of this book. I would like to thank you for your interest and for staying with it this far. It only remains for me to wish you good luck with your multi-threaded C++ programs.

PS And watch out for data races!

¹ ["C++ Concurrency In Action", Anthony Williams, Manning Publications, 2019, page 315 \(back\)](#)

² ["Design Patterns", Gamma, Helm, Johnson and Vlissides, Addison-Wesley, 1995, p. 129 \(back\)](#)

³ ["C++ Concurrency In Action", Anthony Williams, Manning Publications, 2019, page 55-59 \(back\)](#)

About the Author



James began programming in C++ in 1993. He developed software in C++ for a number of companies in the UK and Continental Europe and now works as a freelance instructor and author. His hobbies include photography, ballroom dancing and walking in the country.

Table of Contents

[Concurrency Introduction](#)

[Concurrency Motivation](#)

[Concurrency Overview](#)

[Concurrent Applications](#)

[A Brief History of C++ Concurrency](#)

[Launching a Thread](#)

[The C++ Thread Class](#)

[Managing Threads](#)

[Multiple Threads](#)

[Data Races](#)

[Working With Shared Data](#)

[Lock Guard](#)

[Unique Lock](#)

[Recursive and Timed Mutexes](#)

[Shared Mutexes](#)

[Shared Data Initialization](#)

[Double-checked Locking](#)

[Thread-local Variables](#)

[Deadlock](#)

[Livelock](#)

[Mutex Conclusion](#)

[Condition Variables](#)

[Condition Variables and Predicates](#)

[Futures and Promises](#)

[Shared Futures](#)

[Atomic Operations Introduction](#)

[Atomic Types](#)

[Atomic Operations](#)

[Spin Locks](#)

[Lock-free Programming](#)

[Asynchronous Programming](#)

[Packaged Tasks](#)

[The std::async\(\) function](#)

[Choosing a Thread Object](#)

[C++ Limitations](#)

[Parallelism Overview](#)

[Execution Policies](#)

[Algorithms and Execution Policies](#)

[New Parallel Algorithms](#)

[Concurrent Queue Implementation](#)

[Thread Pools](#)

[Thread Pool Implementation](#)

[Conclusion](#)

[About the Author](#)

Landmarks

[Cover](#)